

VUE Auto IBD KMeans Clustering

D20 - NUE / VUE

8 October, 2026

Table of Contents

1	1. Overview	4
1.1	Details:	4
2	2. UI Screen	5
3	3. Sequence Diagram	6
4	4. Architecture Diagram	8
5	5. Component Diagram	11
6	6. Tech Stack	12
7	7. Input / Output Specification	13
7.1	7.1 Input:	13
7.2	7.2 Output	14
7.3	7.3 Intermediate:	15
8	8. Detailed Method Implementation	16
8.1	8.1 INIT	16
8.2	8.2 vectorize	17
8.3	8.3 find_k_values	19
8.4	8.4 create_k_dict	20
8.5	8.5 calc_clusters	21
8.6	8.6 transform (Entry Point)	22
9	9. Data Flow Diagram	24
10	10. Algorithm Detail: K Selection via Silhouette Score	25
11	11. Usage Example	26
12	12. Error Handling	27
13	13. Dependencies	28
14	14. Detailed Functional Comparison Between Pyspark Prototype and Existing implementation.	29
14.1	✅ What's the Same	29
14.2	❌ Key Functional Differences	29
14.2.1	1. K Selection (Biggest Difference)	29
14.2.2	2. Center Initialization	30

14.2.3	3. Clustering Dimension	30
14.2.4	4. Minimum Cluster Size Enforcement.....	30
14.2.5	5. Edge Case: Single Record / Small MBD	31
14.2.6	6. Post-Clustering Pipeline	31
14.2.7	7. IBD Naming	31
14.3	 Summary.....	31
15	15. Risks	32
16	16 . Non-Functional Requirements	33
17	16. Sign off	34
18	IBD KMeans	35
19	IBD KMeans Details	98
19.1	1. Implementation Steps	98
19.2	2. Traceability Matrix / Functionality–Component Mapping	103
19.3	3. User Stories and dev plan.....	107
19.4	4. Sprint Summary	118
19.5	5. Story Point Distribution by Category.....	118
20	VUE Auto IBD KMeans Job Tests	120
20.1	Kmeans job with spring boot flow	120
20.2	Kmeans job with databricks flow.....	121
20.2.1	Test with few kvalues as None for few mbds	123
20.3	Data Validation with SB or DB func run	124
20.3.1	UI DATA after Databricks run	126

1 1. Overview

KMeansCluster is a PySpark-based module that performs K-Means clustering to assign retail outlet cells into **(IBD) groups** within each **MBD**. It auto-selects the optimal number of clusters using the **Silhouette Score** and enforces a minimum cluster size constraint.

Clusters cells into IBDs based on store and MBD sales variations and contributions. The method evaluates possible clusters using the channel-sample-mbd wss for each cellid, then using the Silhouette metric it tests different number of clusters (from 2 to the maximum number of clusters parameter, currently 8) and picks the one with the maximum Silhouette score, then it computes the `ibd_cluster_center`: average wss of the cells in the cluster. The maximum number of cells per clusters to measure can be pass as a parameter. The maximum number of cells per clusters and the minimum stores required per cluster can be pass as a parameter. If no clusters can satisfy the requirement, the MBD would have 2 clusters.

1.1 Details:

Parameters:

`k_values`: optional parameter. A dictionary with number of clusters and `market_id`. Defaults to None. If it's 'None' the algorithm will define the number of clusters per MBD.

`max_cluster`: Maximum number of clusters to be created per MBD_ID. Default is 8.

`min_cluster_size`: Minimum number of outlets in each cluster. Default is 2.

The first step is to vectorize the wss data from mbd variance (this is done using the Pyspark ML `VectorAssembler`) and creates a concatenated `channel_sample_mbd` column.

Second is to compute the number of clusters per mbd (if the `k_values` parameter is None). For each `channel-sample-mbds` it evaluates the number of clusters from 2 to `max_cluster` using the Silhouette metric and returns the one with the maximum Silhouette score (this is done with the Pyspark ML `ClusteringEvaluator`).

Once we have the number of clusters per `channel-sample-mbd` it calculates the clusters (using the Pyspark ML KMeans) and names it as 'IBDID'. It also computes the cluster center as the average of wss.

Finally it just joins the mbd variance with the clusters at cell level.

2 2. UI Screen

Auto Design IBDs

MBD Selection Cell Characteristics Sales Variation Report **Scree Plot Report**

MBD:

Groups

Max Cluster Size

Min Cluster Size

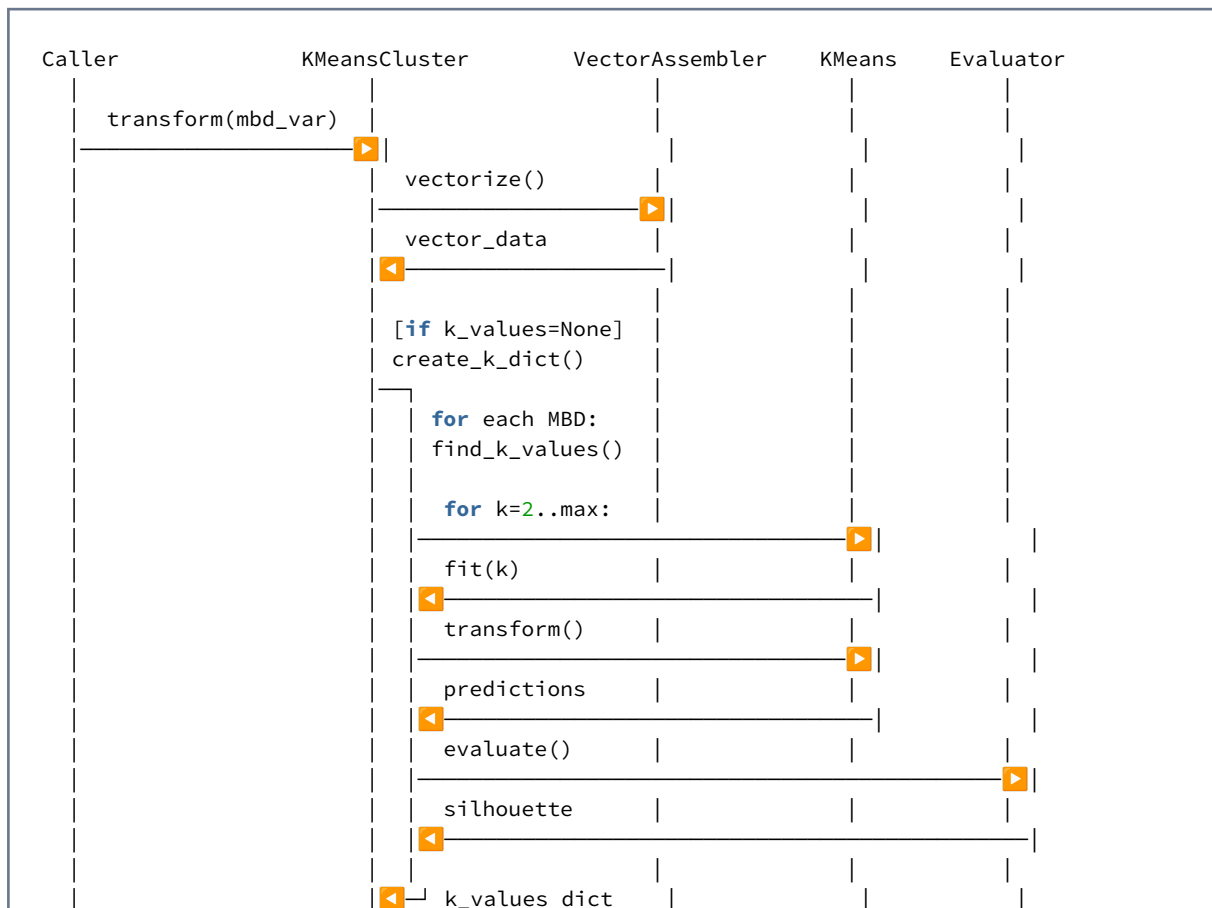
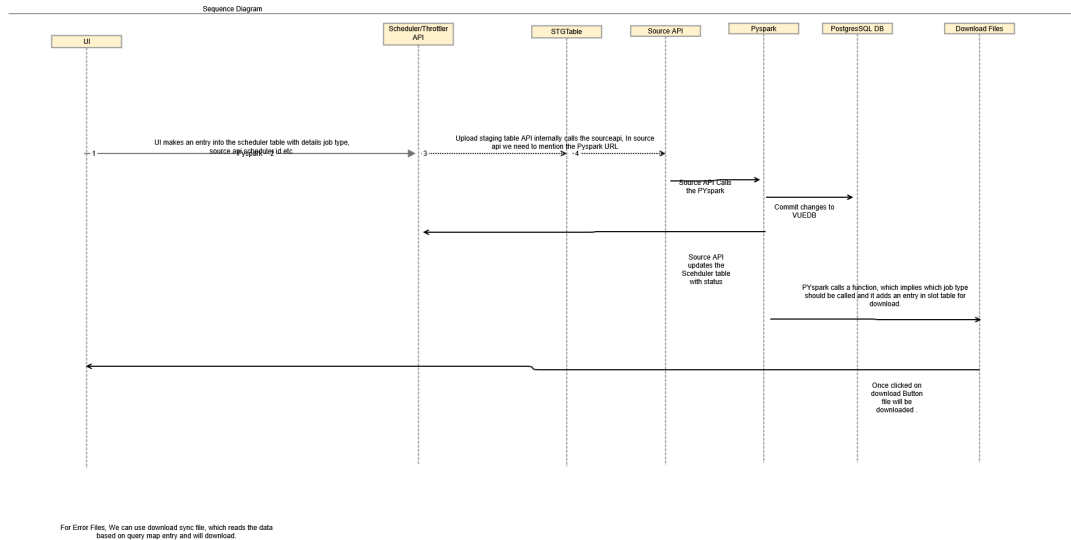
Save number of groups

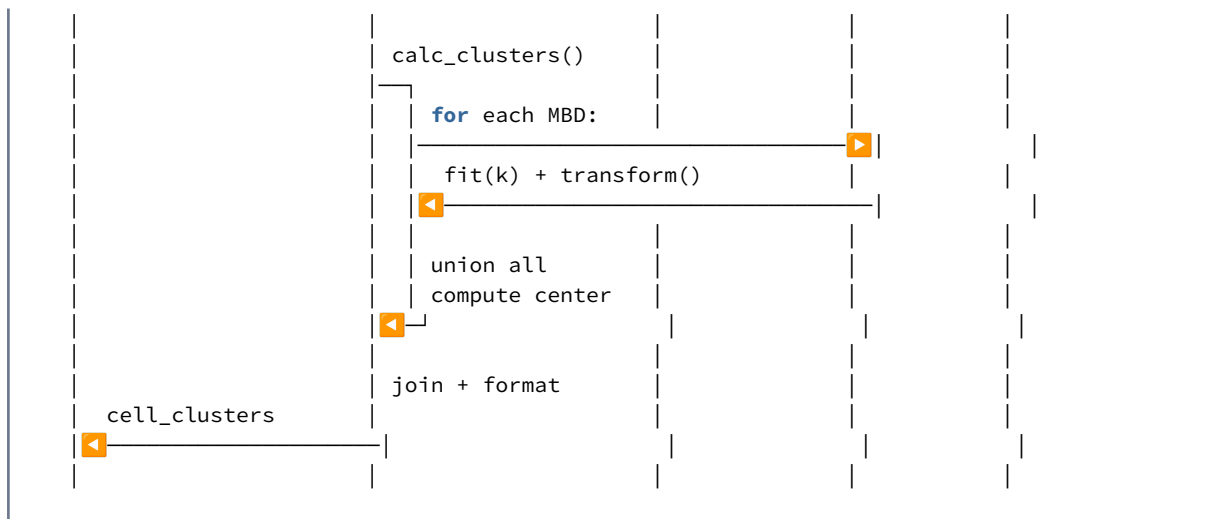
Submit KMeans clustering job

Channel ID	Sample ID	MBD ID	Name	Groups
MONT	20003424	Dummy_200003424	Dummy MBD	4
MONT	20003424	1_20189370	T.BR INA-NFC-C&C	10

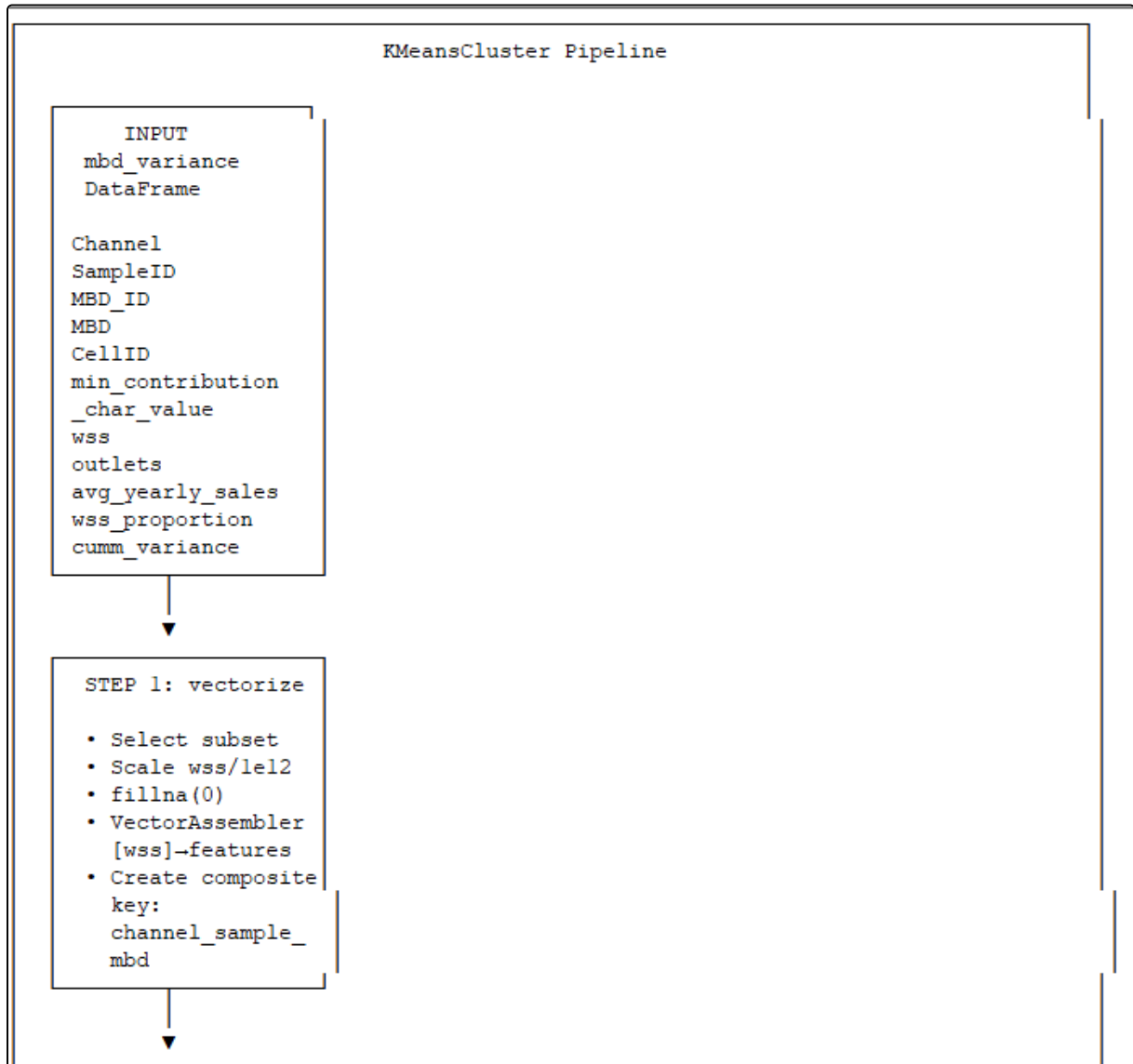
If no clusters satisfy the constraints, the MBD will default to exactly 2 clusters.

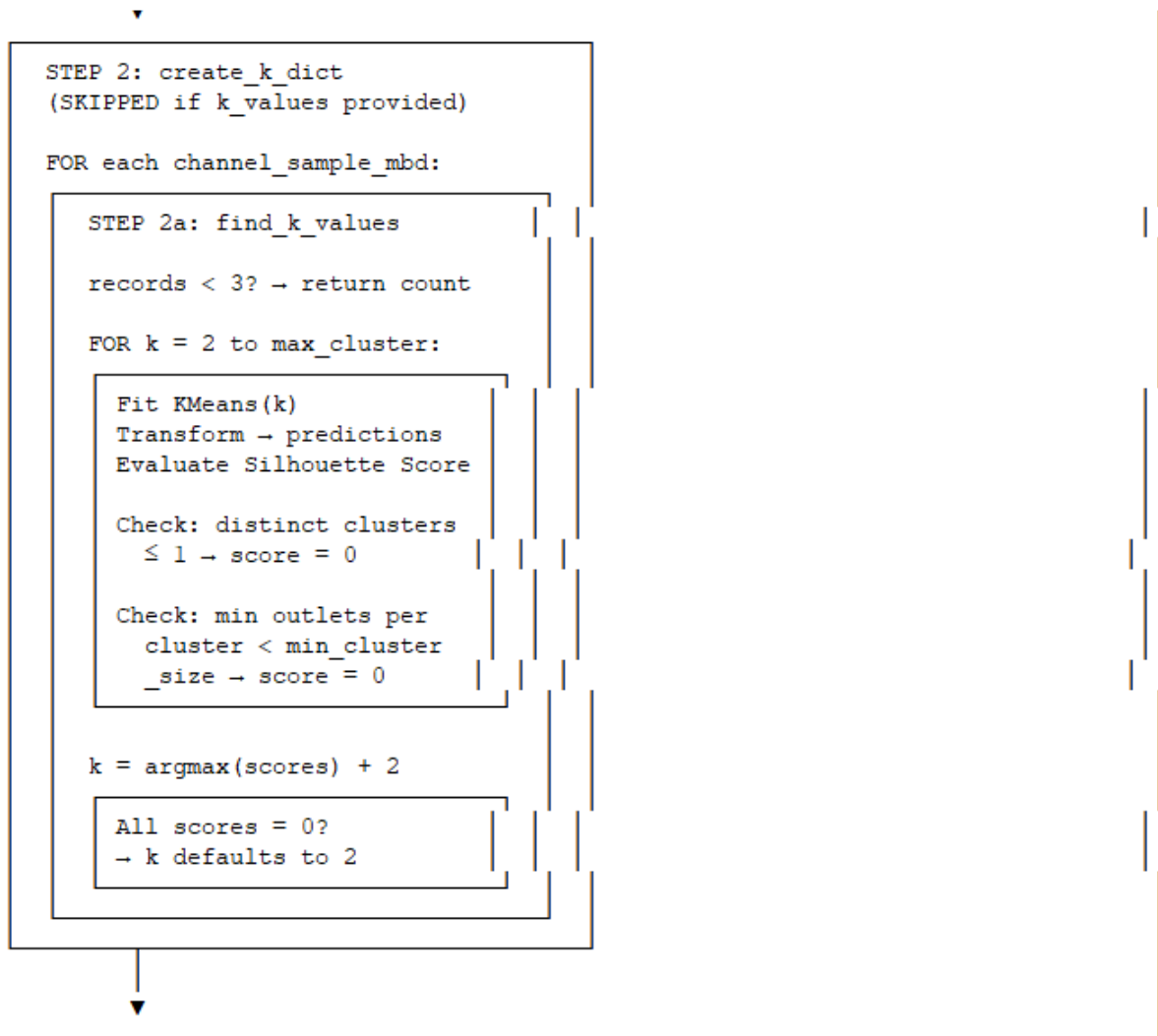
3. Sequence Diagram

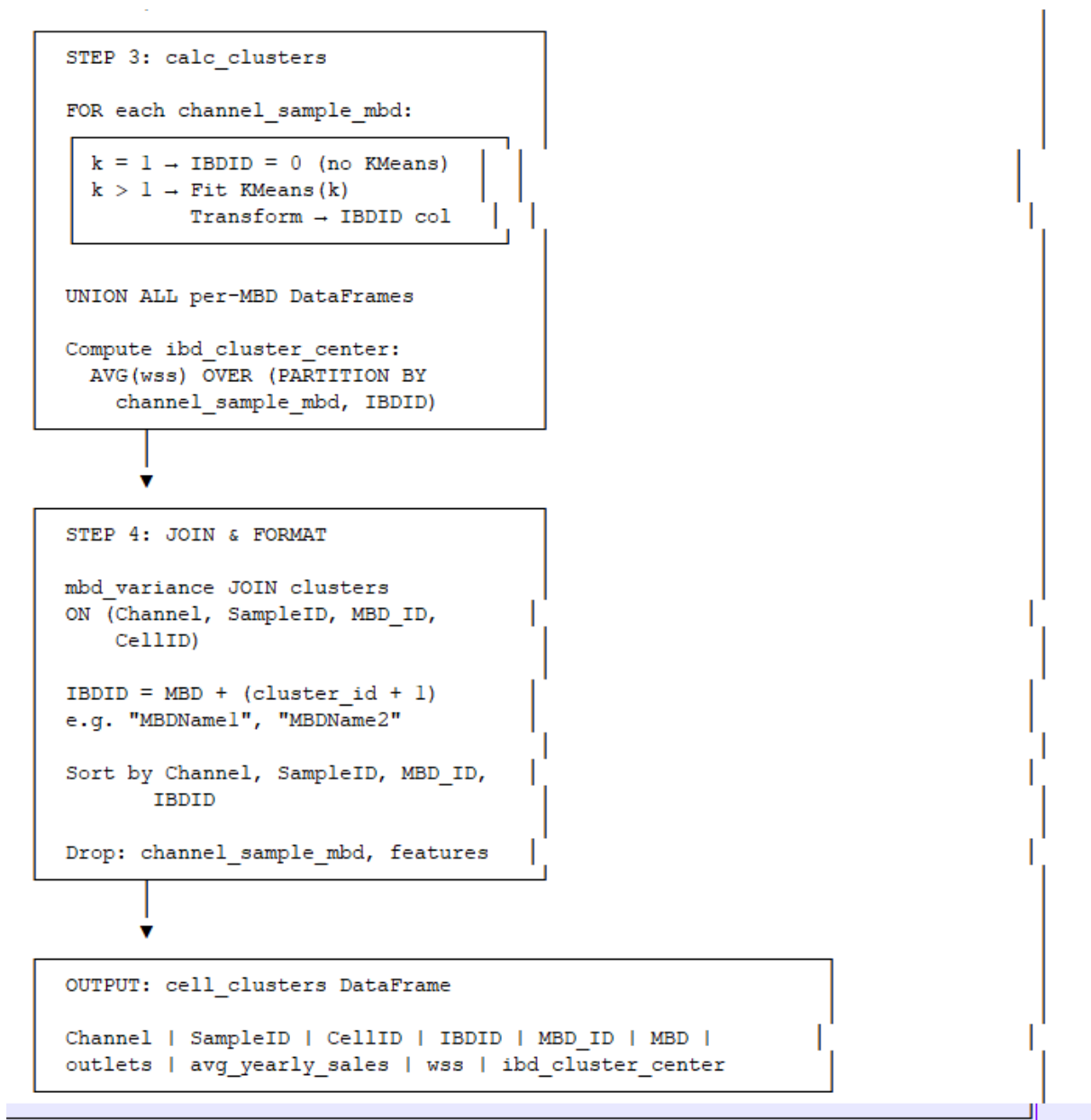




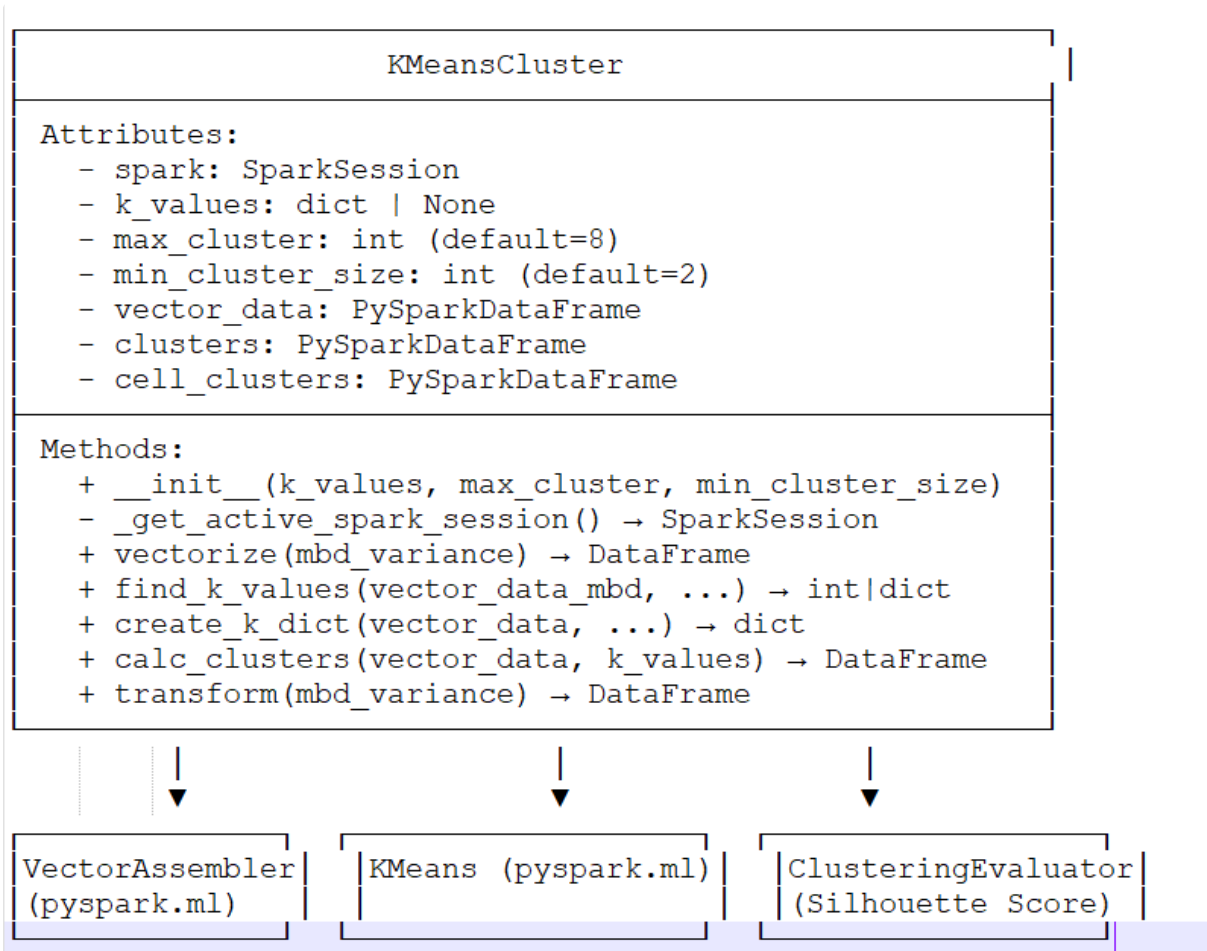
4 4. Architecture Diagram







5 5. Component Diagram



6 6. Tech Stack

- React
- Pyspark with Python.
- Postgres SQL

7 7. Input / Output Specification

Prototype Code from Datascience Team for Kmeans: https://github.com/niq-enterprise/ttrnd-vue-impacts/blob/structure_refactor/src/vue_impacts/transformers/ibd_mapping_flow/k_means_cluster.py

7.1 7.1 Input:

Tables can be used to get below parameters

```
select * from <<countryschema>>.vueautoibdsellsalesvariationmulti_<<projectid>>_<<datasetid>>;
```

```
select * from  
<<countryschema>>.VUEAutoIBDsSalesVariationDetailsMulti_<<projectid>>_<<datasetid>>;
```

Column	Type	Description
Channel	String	Distribution channel identifier
SampleID	String/Int	Sample identifier
MBD_ID	String	MBD group identifier
MBD	String	MBD name (used in IBD naming)
CellID	String/Int	Individual cell/outlet group ID
min_contribution_char_value	String	Minimum contribution characteristic value
wss	Double	Within-group Sum of Squares (sales variation)
outlets	Int	Number of outlets in the cell

Column	Type	Description
avg_yearly_sales	Double	Average yearly sales for the cell
wss_proportion	Double	Proportion of WSS
cumm_variance	Double	Cumulative variance

7.2 7.2 Output

Column	Type	Description
Channel	String	Distribution channel
SampleID	String/Int	Sample identifier
CellID	String/Int	Cell identifier
IBDID	String	IBD label e.g. "MBDName1" , "MBDName2"
MBD_ID	String	MBD group identifier
MBD	String	MBD name
outlets	Int	Outlet count
avg_yearly_sales	Double	Average yearly sales
wss	Double	Scaled WSS ($wss / 1e12$)
ibd_cluster_center	Double	Average WSS of cells in the same IBD cluster

Below tables should also be populated after KMeans is successful.

```
select * from <<countryschema>>.VUEIBDsMulti_<<projectid>>_<<Datasetid>>;
```

```
select * from <<countryschema>>.VUEIBDCellsMulti_953239_<<Datasetid>>;
```

```
select * from <<countryschema>>.VUEMBDCellsMulti_953239_<<Datasetid>>;
```

File Naming convention should be Projectid_schedulerid_KMeansReport.

[910862_181792_KMeansReport.zip](#)¹

7.3 7.3 Intermediate:

```
# Structure:

{
  "ChannelA_Sample1_MBD001": 3,
  "ChannelA_Sample1_MBD002": 5,
  "ChannelB_Sample2_MBD001": 2,
  ...
}
```

1. https://adlm.nielseniq.com/confluence/download/attachments/889690779/910862_181792_KMeansReport.zip?api=v2&modificationDate=1774341094947&version=1

8. Detailed Method Implementation

8.1 INIT

```
def __init__(
    self,
    k_values: dict = None,
    max_cluster: int = 8,
    min_cluster_size: int = 2,
):
```

Purpose: Initialize the clustering configuration.

Flow:

START

- |
- |— Get active SparkSession (raises ValueError if none)
- |— Store k_values (None = auto-detect)
- |— Store max_cluster (upper bound for k search)
- |— Store min_cluster_size (minimum outlets per cluster)

END

Parameter	Default	Description
k_values (see page 4)	None	Pre-computed k per MBD. If None , auto-computed via silhouette
max_cluster (see page 4)	8	Maximum k to evaluate

Parameter	Default	Description
min_cluster_size (see page 4)	2	Minimum total outlets in any cluster

Decision Flow:

```

k_values = None?
  YES → Algorithm will auto-detect optimal k for each MBD
  NO  → Use provided k values directly (skip silhouette evaluation)

```

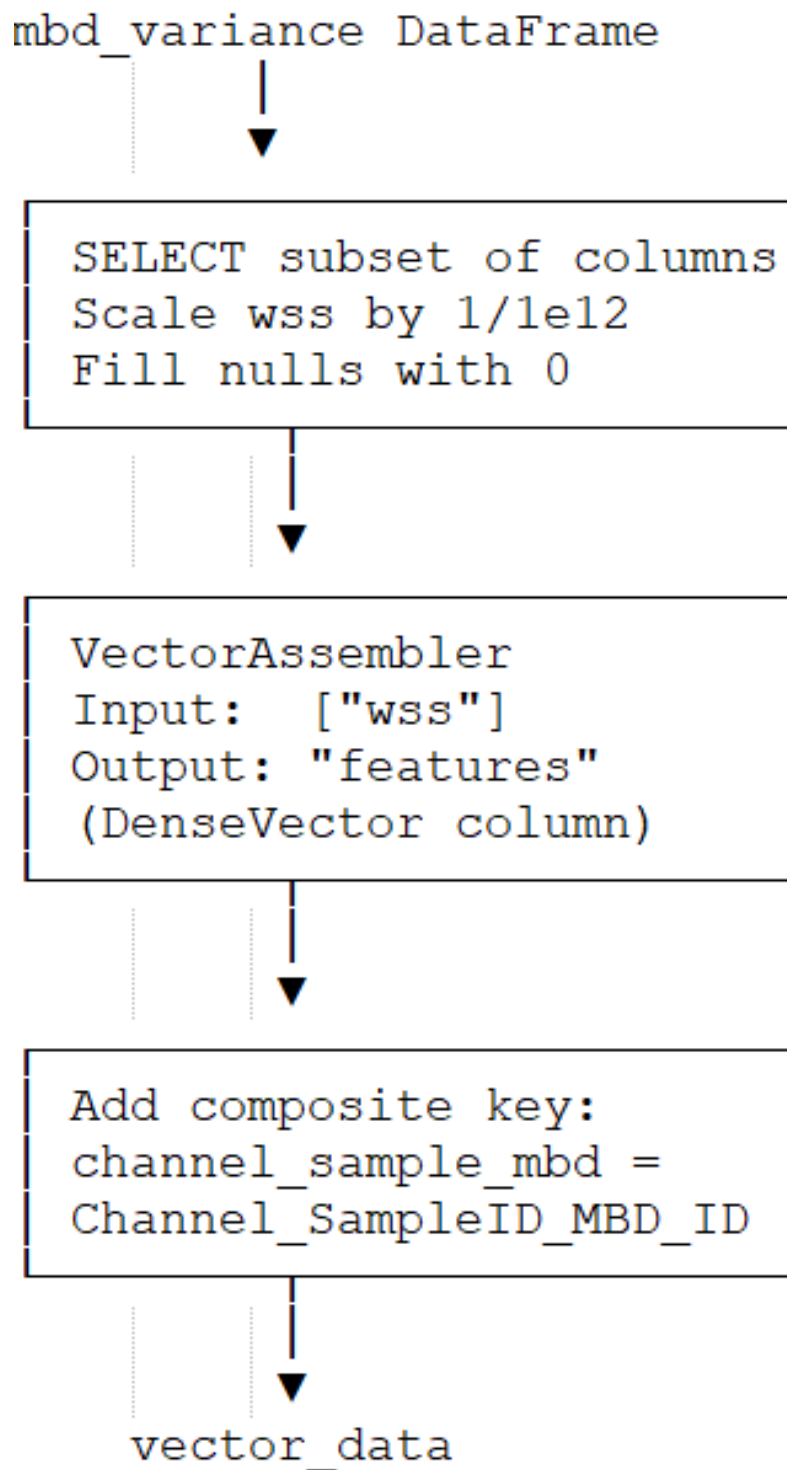
8.2 8.2 vectorize

```

def vectorize(self, mbd_variance: PySparkDataFrame) -> PySparkDataFrame:
    assembler = VectorAssembler(inputCols=["wss"], outputCol="features")
    vector_data = assembler.transform(
        mbd_variance.select(
            "Channel", "SampleID", "MBD_ID", "CellID", "outlets",
            (f.col("wss") / 1000000000000).alias("wss"),
        ).fillna({"wss": 0})
    ).withColumn(
        "channel_sample_mbd",
        f.concat("Channel", f.lit("_"), "SampleID", f.lit("_"), "MBD_ID"),
    )
    return vector_data

```

Purpose: Prepare data for Spark ML K-Means.**Flow:**



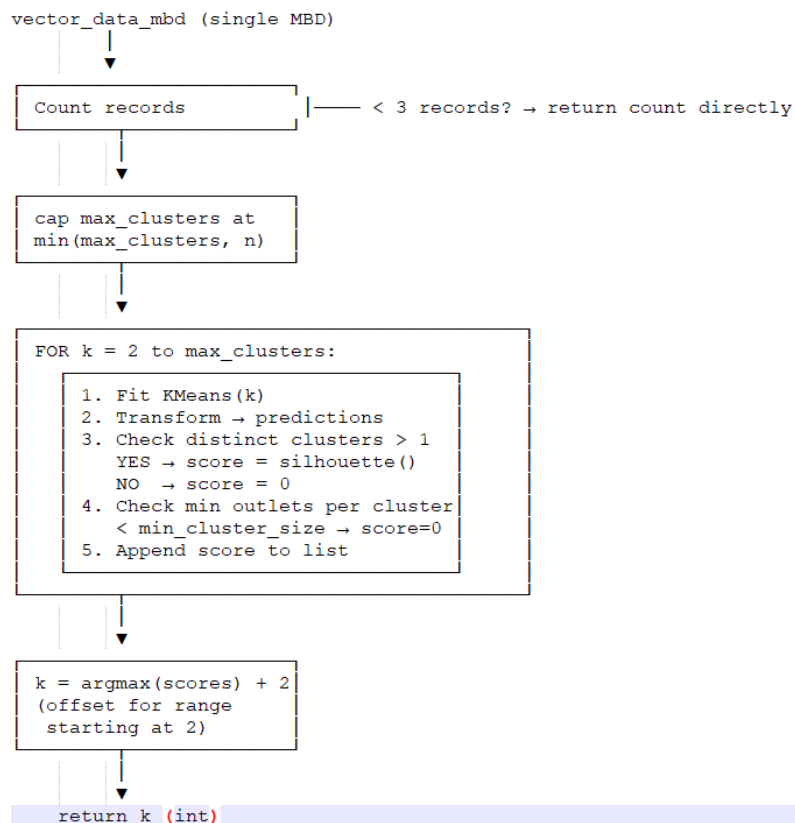
Why scale by $1e12$? WSS values can be extremely large (trillions). Scaling prevents numerical overflow in squared Euclidean distance calculations.

8.3 find_k_values

```
def find_k_values(
    self,
    vector_data_mbd: PySparkDataFrame,
    max_clusters: int = 8,
    min_cluster_size: int = 2,
) -> dict:
```

Purpose: Find optimal k for a single MBD using silhouette score.

Flow:



Silhouette Score Logic:

Score range: [-1, +1]

+1 = Clusters are well separated

0 = Overlapping clusters

-1 = Misclassified points

Selection: k with HIGHEST silhouette score wins

Min Cluster Size Check:

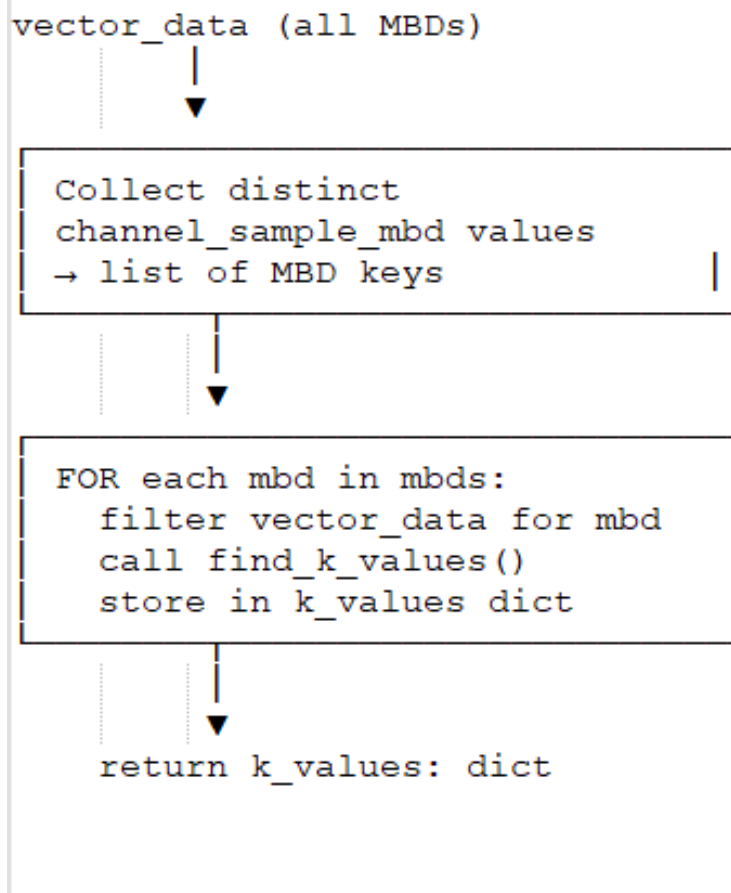
```
For each k:
  group by prediction (cluster)
  sum(outlets) per cluster → cluster_outlets
  min(cluster_outlets) across all clusters
  if min < min_cluster_size → score = 0 (disqualify this k)
```

8.4 create_k_dict

```
def create_k_dict(
    self,
    vector_data: PySparkDataFrame,
    max_cluster: int = 8,
    min_cluster_size: int = 2,
) -> dict:
```

Purpose: Build a dictionary mapping each `channel_sample_mbd` key to its optimal k.

Flow:



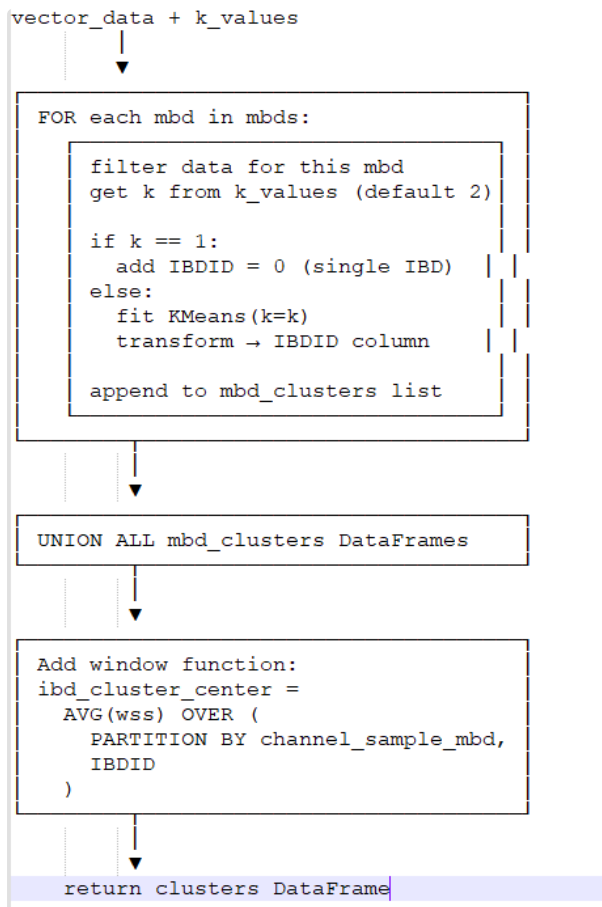
Performance Note: This iterates MBDs sequentially on the driver. For large numbers of MBDs, this could be a bottleneck.

8.5 8.5 calc_clusters

```
def calc_clusters(
    self, vector_data: PySparkDataFrame, k_values: dict
) -> PySparkDataFrame:
```

Purpose: Run K-Means clustering per MBD using the determined k values.

Flow:

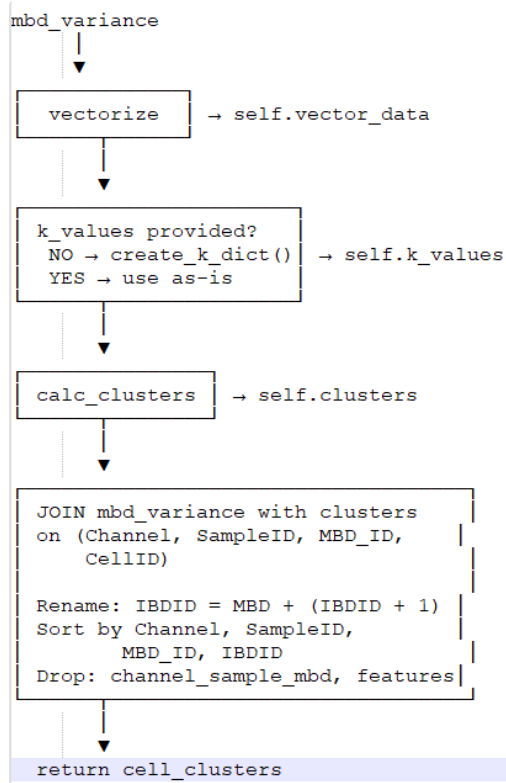


8.6 8.6 transform (Entry Point)

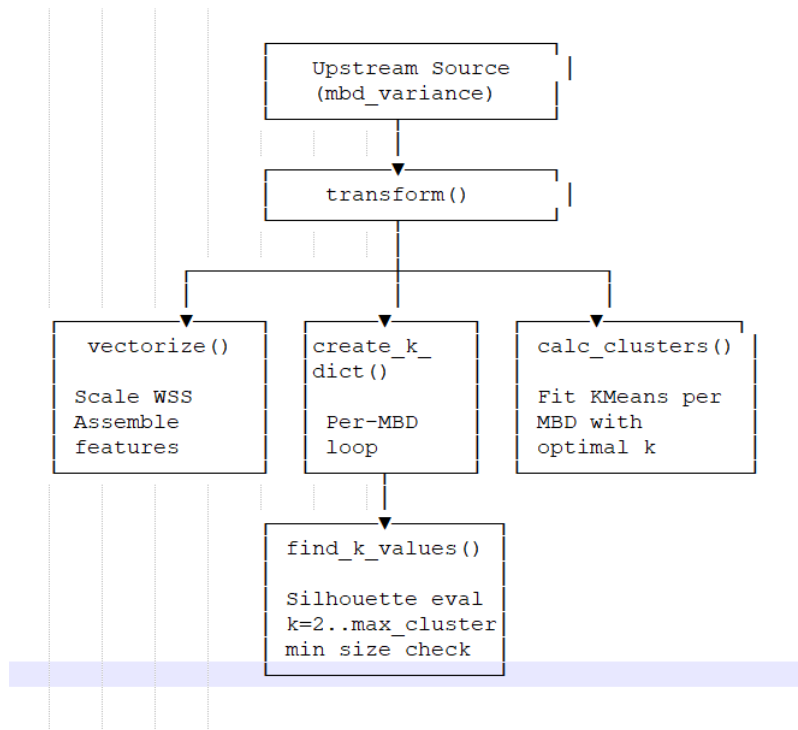
def transform(self, mbd_variance) -> PySparkDataFrame:

Purpose: Full pipeline orchestrator – the only method the caller needs to invoke.

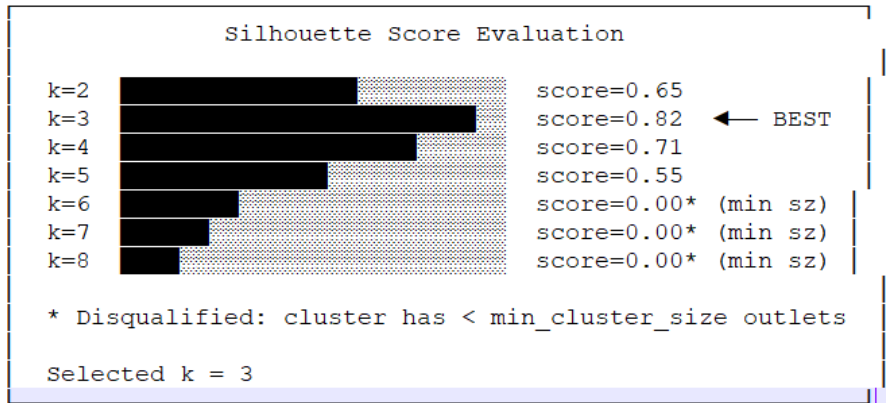
Flow:



9 9. Data Flow Diagram



10 10. Algorithm Detail: K Selection via Silhouette Score



11 11. Usage Example

```

from pyspark.sql import SparkSession
from k_means_clusterprototypes import KMeansCluster

# Requires active Spark session
spark = SparkSession.builder.appName("IBD_Clustering").getOrCreate()

# Input DataFrame: mbd_variance
# Columns: Channel, SampleID, MBD_ID, MBD, CellID, outlets, wss, avg_yearly_sales

# Option 1: Auto-detect k values
clusterer = KMeansCluster(max_cluster=8, min_cluster_size=2)
result = clusterer.transform(mbd_variance)

# Option 2: Pre-defined k values
k_values = {
    "ChannelA_1_MBD001": 3,
    "ChannelA_1_MBD002": 5,
}
clusterer = KMeansCluster(k_values=k_values)
result = clusterer.transform(mbd_variance)

# Access results
result.show()
# +-----+-----+-----+-----+-----+-----+-----+-----+-----+
# |Channel|SampleID|MBD_ID| MBD|CellID|IBDID|outlets| wss|ibd_cluster_center|
# +-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

12 12. Error Handling

Scenario	Handling
No active SparkSession	<code>ValueError</code> (see page 4) raised in <code>__init__</code> (see page 4)
MBD with < 3 records	Returns row count directly; skips silhouette evaluation
All clusters have score 0	<code>k = 2</code> (see page 4) (minimum, from <code>argmax</code> on all-zero list)
<code>k = 1</code> (see page 4)	Assigns all cells <code>IBDID = 0</code> (no KMeans fit)
MBD not in <code>k_values</code> (see page 4) dict	Defaults to <code>k = 2</code> (see page 4) via <code>.get(mbd, 2)</code>
Null WSS values	Filled with <code>0</code> via <code>fillna</code> (see page 4)

13 13. Dependencies

pyspark.sql.Session
pyspark.sql.DataFrame
pyspark.sql.functions
pyspark.ml².feature.VectorAssembler
pyspark.ml³.clustering.KMeans
pyspark.ml⁴.evaluation.ClusteringEvaluator
pyspark.sql.window.Window

2. <http://pyspark.ml>

3. <http://pyspark.ml>

4. <http://pyspark.ml>

14 14. Detailed Functional Comparison Between Pyspark Prototype and Existing implementation.

Both files implement **K-Means clustering to assign cells into IBD groups per MBD**, but they are **not functionally identical**. Here's a precise breakdown:

14.1 What's the Same

Aspect	PySpark	PostgreSQL
Goal	Cluster cells into IBDs per MBD	Same
Algorithm	K-Means	K-Means
Clustering feature	WSS (Within-group Sum of Squares) / AverageSales	AverageSales (derived from WSS)
Convergence threshold	Spark default (similar small tolerance)	0.01
Max iterations	Spark default (20) or configurable	100
Output	IBD assignments per Cell per MBD	Same

14.2 Key Functional Differences

14.2.1 1. K Selection (Biggest Difference)

	PySpark	PostgreSQL
How k is chosen	Auto-tuned via Silhouette Score across range [2, max_cluster]	Pre-defined from DesignIBDsGroupsNumber i in the database

	PySpark	PostgreSQL
Impact	Finds the statistically optimal number of clusters	Uses whatever the user/system previously configured

This means the **PySpark version may produce a different number of clusters** for the same data.

14.2.2 2. Center Initialization

	PySpark	PostgreSQL
Method	Spark's default K-Means (k-means++ variant) – probabilistic	Deterministic – picks evenly spaced values from AverageSales sorted list ($1 + i * steps$)

Different initialization → **different convergence paths** → potentially **different final cluster assignments**.

14.2.3 3. Clustering Dimension

	PySpark	PostgreSQL
Feature	wss / 1e12 (scaled WSS)	AverageSales (raw)

Even though WSS and AverageSales are related, they are **not the same value**, so distance calculations differ.

14.2.4 4. Minimum Cluster Size Enforcement

	PySpark	PostgreSQL
Enforced?	 Yes – checks min_cluster_size (see page 4) (outlet count per cluster); sets score to 0 if violated	 No – no minimum cluster size check

14.2.5 5. Edge Case: Single Record / Small MBD

	PySpark	PostgreSQL
Handling	Returns <code>mbd_records</code> (see page 4) directly if <code>< 3</code> ; assigns <code>IBDID = 0</code> if <code>k = 1</code> (see page 4)	Inserts single center when <code>DesignIBDsGroupsNumber = 1</code> ; no special handling for very small groups

14.2.6 6. Post-Clustering Pipeline

	PySpark	PostgreSQL
Scope	Returns a DataFrame – clustering only	Full pipeline: clustering → IBD creation → Cell-to-IBD assignment → Reports → Job status updates → Process logging

14.2.7 7. IBD Naming

	PySpark	PostgreSQL
Format	<code>MBD + (IBDID + 1)</code> e.g. <code>"MBD_A1"</code>	<code>MBDName(MBDID)_IBDCenterID</code> e.g. <code>"SomeName(123)_IBD2"</code>

14.3 Summary

They are NOT functionally equivalent. While both perform K-Means clustering on similar data to produce IBD assignments, they differ in:

1. **K selection** – auto-tuned (PySpark) vs. pre-configured (SQL)
2. **Center initialization** – k-means++ (PySpark) vs. evenly-spaced deterministic (SQL)
3. **Feature used** – scaled WSS vs. AverageSales
4. **Minimum cluster size** – enforced (PySpark) vs. not enforced (SQL)
5. **Scope** – clustering module only (PySpark) vs. full end-to-end pipeline (SQL)

Running both on the same input data will likely produce different cluster assignments.

15 15. Risks

The requirement documentation only briefly describes the expected functionality and refers to the provided prototype. However, the post-processing step does not specify **which downstream tables must be refreshed or reloaded**, even though such table refresh logic exists in the current/legacy implementation. The lack of this information creates ambiguity and may lead to incomplete or incorrect data updates in production.

Impact:

- Potential data inconsistency across dependent tables
- Risk of functional deviations from existing system behavior

16 16 . Non-Functional Requirements

- Response time and throughput remains same as the existing as its not a new feature and the below details were mentioned

There could be in general 4-5 sample and the number of cells may differ from few hundreds to 1-2000.

In terms of characteristics, we may expect for each cell ~3-4 chars

- Scalability: The infra details will be same as the existing KMeans as its a refactored code.
- Resource utilization under different loads or conditions (peak times, days, etc.) : To be provided.
- Kmeans is run 1 per year by all countries (mostly Q4-Q1) for all the samples

Note: The NFR were not shared and it was informed as its a refactored code the details remains same as its in production now.

17 16. Sign off

1	Stakholder Approval	
2	Internal Review Approval	
3	Design understanding and Story points Approval from the Implementation Team	

18 IBD KMeans

1. K-Means Clustering for Initial Business Definition (IBD) Generation - Design Document

1. a. Document Information

Field	*Value*
-----	-----
Document Type	Detailed Design Document
Component	K-Means Clustering for IBD Generation
Version	1.0
Last Updated	2024
Author	System Architecture Team
Status	Draft

—

1. a. Table of Contents

1. Executive Summary
2. Business Context
3. System Overview
4. [Functional Requirements](#)⁵(#4-functional-requirements)
5. Technical Architecture
6. [Detailed Design](#)⁶(#6-detailed-design)
7. Data Model
8. Algorithm Details
9. Implementation Specifications
10. Performance Considerations
11. Testing Strategy
12. Deployment
13. Appendices

5. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Functional+Requirements

6. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Detailed+Design

1. a. 1. Executive Summary

1. a. i. 1.1 Purpose

This document provides a comprehensive design specification for the PySpark-based K-Means clustering implementation that automatically generates Initial Business Definitions (IBDs) by clustering retail cells based on Market Business Definition (MBD) sales variations and contributions. The system uses the Within-Sum-of-Squares (WSS) metric and Silhouette score optimization to determine optimal cluster configurations.

1. a. i. 1.2 Scope

In Scope:

- PySpark MLlib-based K-Means clustering implementation
- Silhouette score optimization for automated K selection
- Channel-Sample-MBD level clustering
- IBD cluster center computation
- Configurable clustering parameters
- Data vectorization and transformation pipeline

Out of Scope:

- UI/Frontend development
- Manual IBD creation workflows
- Data quality validation outside clustering logic
- Real-time clustering capabilities
- Alternative clustering algorithms (DBSCAN, Hierarchical, etc.)

1. a. i. 1.3 Key Objectives

1. ***Automation***: Automatically determine optimal number of clusters per MBD using Silhouette analysis
2. ***Optimization***: Maximize cluster quality through statistical validation
3. ***Scalability***: Handle large datasets efficiently using Spark's distributed computing
4. ***Flexibility***: Support both automated and predefined cluster configurations
5. ***Accuracy***: Ensure proper cluster center calculations and cell assignments

1. a. 2. Business Context

1. a. i. 2.1 Problem Statement

Initial Business Definitions (IBDs) need to be created for Market Business Definitions (MBDs) based on sales variation patterns across retail outlets. The challenge is to:

- ***Group similar outlets*** into meaningful clusters (IBDs)
- ***Determine optimal number*** of clusters automatically

- ***Handle multi-dimensional*** variation data (channel, sample, MBD)
- ***Scale efficiently*** for large retail networks
- ***Maintain quality*** through statistical validation

1. a. i. 2.2 Business Value

The K-Means clustering implementation provides:

- ***Data-Driven IBD Creation***: Removes subjectivity from manual IBD definition
- ***Optimal Segmentation***: Uses Silhouette scores to find best cluster count
- ***Actionable Insights***: Groups outlets with similar sales characteristics
- ***Scalability***: Processes multiple samples and MBDs in parallel
- ***Quality Assurance***: Statistical validation of cluster quality

1. a. i. 2.3 Use Case Example

Scenario: A retail chain wants to create IBDs for a grocery MBD across 500 stores

Input:

- MBD: "Grocery Category A"
- 500 combination values (store configurations)
- WSS values representing sales variations
- Target: 2-8 clusters

Process:

1. Vectorize WSS data for ML processing
2. Test K=2 through K=8 clusters
3. Calculate Silhouette score for each K
4. Select K=5 (highest Silhouette = 0.68)
5. Generate 5 IBDs with cluster centers

Output:

- 5 IBDs: "Grocery_Cat_A_IBD1" through "Grocery_Cat_A_IBD5"
- Each IBD contains stores with similar sales patterns
- Cluster centers represent average WSS characteristics

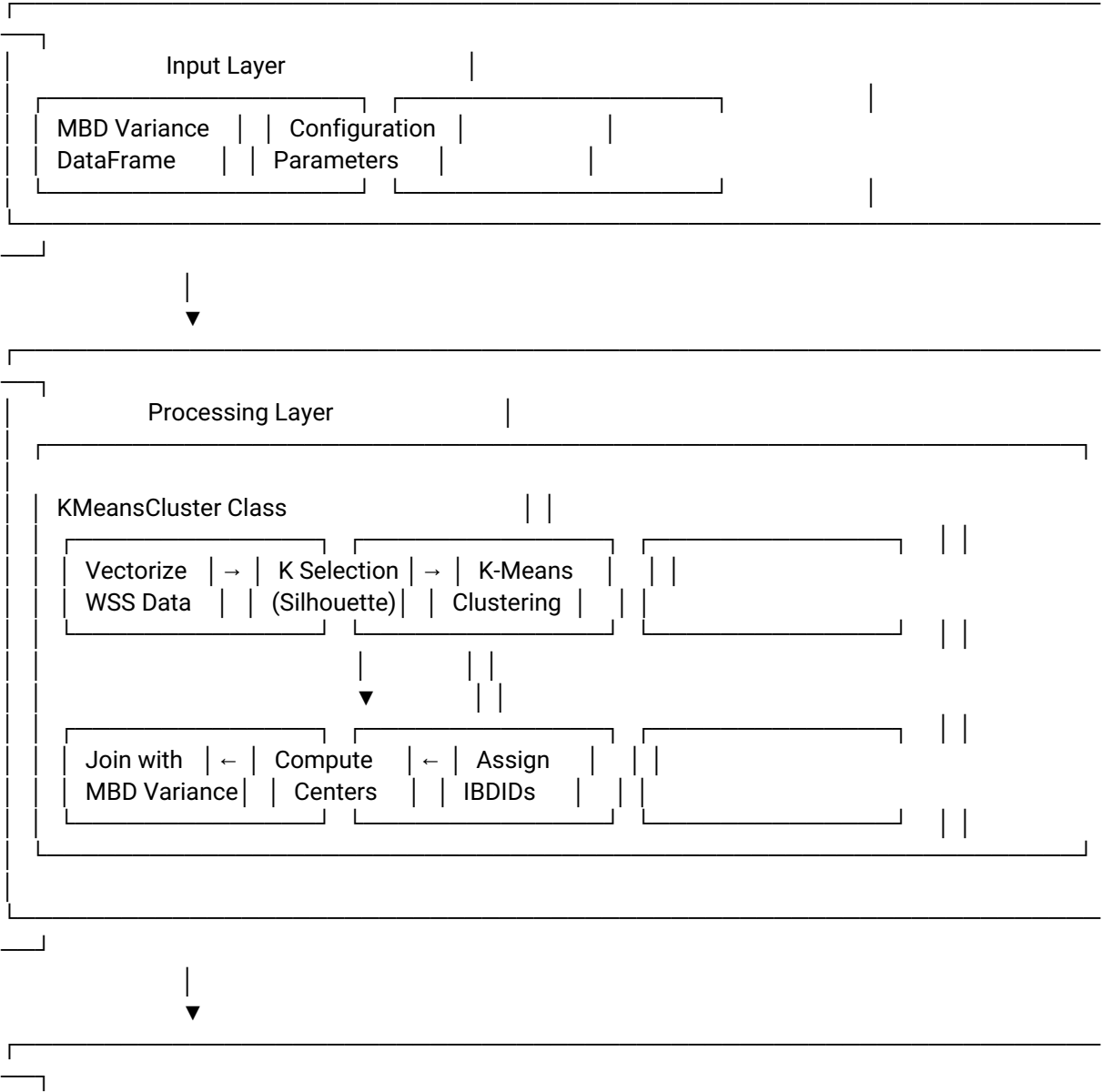
1. a. i. 2.4 Success Criteria

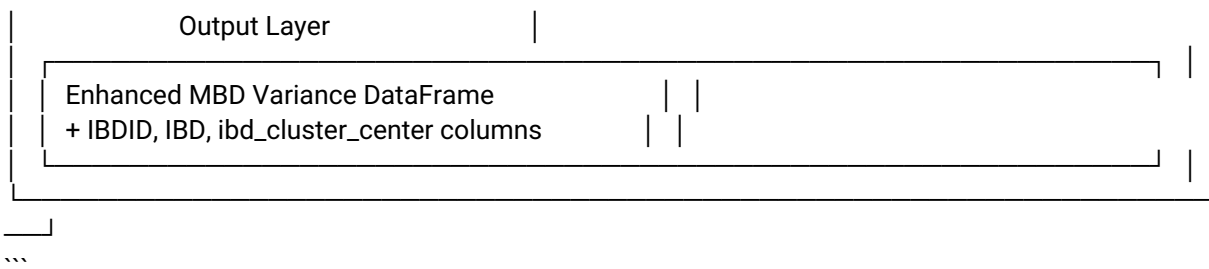
Metric	*Target*
-----	-----
Silhouette Score (Average)	> 0.4
Cluster Size Compliance	All clusters $\geq$ min_cluster_size

Processing Success Rate	> 95%
Cluster Quality	Well-separated, cohesive clusters
Scalability	Handle 1000+ MBDs per execution

- 1. a. 3. System Overview
- 1. a. i. 3.1 High-Level Architecture

...





1. a. i. 3.2 Technology Stack

Layer	*Technology*	*Purpose*
-----	-----	-----
Compute Engine	Apache Spark 3.x	Distributed data processing
Programming	Python 3.8+ with PySpark	Application logic
ML Framework	PySpark MLlib	K-Means, VectorAssembler, ClusteringEvaluator
Data Format	Spark DataFrame	In-memory data representation
Orchestration	External scheduler (Airflow/Manual)	Job execution

1. a. i. 3.3 Component Overview

1. a. i. 1. ***KMeansCluster Class***

The core component that encapsulates all clustering logic:

```

'''python
class KMeansCluster:
    """
    Clusters cells into IBDs based on store and MBD sales variations
    """
    def __init__(self, k_values=None, max_cluster=8, min_cluster_size=2)
    def vectorize(self, mbd_variance: DataFrame) -> DataFrame
    def find_k_values(self, vector_data_mbd: DataFrame, ...) -> int
    def create_k_dict(self, vector_data: DataFrame, ...) -> Dict
    def calc_clusters(self, vector_data: DataFrame, k_values: Dict) -> DataFrame
    def transform(self, mbd_variance: DataFrame) -> DataFrame
'''
  
```

1. a. 4. Functional Requirements

1. a. i. 4.1 Core Functional Requirements

ID	*Requirement*	*Priority*
-----	-----	-----
FR-001	Vectorize WSS data from MBD variance DataFrame	High
FR-002	Create concatenated channel_sample_mbd identifier	High
FR-003	Calculate optimal K using Silhouette score (2 to max_cluster)	High
FR-004	Support predefined K values via k_values parameter	High
FR-005	Enforce minimum cluster size constraint	High
FR-006	Execute K-Means clustering using PySpark MLlib	High
FR-007	Compute cluster centers as average WSS	High
FR-008	Assign IBDID to each cell based on cluster prediction	High

FR-009 Generate IBD names in format "

Unknown macro: {MBD}

_IBD

Unknown macro: {cluster_id}

" | Medium |

FR-010

Join clustering results back with original MBD variance

High

1. a. i. 4.2 Input Parameters

```python

class KMeansCluster:

def \_init\_(

self,

k\_values: Optional[Dictstr, int<sup>7</sup>] = None, # Predefined K per MBD

max\_cluster: int = 8, # Maximum K to test

min\_cluster\_size: int = 2 # Minimum outlets per cluster

...)

**\*Parameter Descriptions\*:**

- **\*k\_values\***: Dictionary mapping `channel\_sample\_mbd` → number of clusters
  - If `None`: Algorithm determines optimal K via Silhouette analysis
  - If provided: Uses specified K values directly

Example:

Unknown macro: {"1\_MBD001"}

- **\*max\_cluster\***: Maximum number of clusters to test
  - Range: 2 to max\_cluster
  - Default: 8
  - Constraint: Must be ≤ (total\_values / min\_cluster\_size)
- **\*min\_cluster\_size\***: Minimum outlets required per cluster
  - Default: 2
  - Ensures clusters have sufficient data points
  - Used to determine maximum testable K

1. a. i. 4.3 Input Data Schema

1. a. i. 1. **\*mbd\_variance DataFrame\***

```python

Schema:

- Channel: string # Distribution channel
- SampleID: string/int # Sample identifier
- MBD_ID: string # Market Business Definition ID
- MBD: string # MBD name
- CellID: string # Cell identifier
- min_contribution_char_value: string # Characteristic value
- wss: double # Within-Sum-of-Squares (variance metric)
- outlets: int # Number of outlets in cell

7. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int

- avg_yearly_sales: double # Average annual sales
- wss_proportion: double # Proportion of total WSS
- cumm_variance: double # Cumulative variance

1. a. i. 4.4 Output Data Schema

1. a. i. 1. ***Enhanced MBD Variance DataFrame***

```
```python
```

Schema:

- Channel: string
- SampleID: string/int
- CellID: string
- IBDID: int # Cluster/IBD identifier (0, 1, 2, ...)
- MBD\_ID: string
- MBD: string
- outlets: int
- avg\_yearly\_sales: double
- wss: double
- IBD: string # IBD name (e.g., "GroceryA\_IBD1")
- ibd\_cluster\_center: double # Average WSS for this cluster

#### 1. a. i. 4.5 Processing Stages

The system operates through **\*5 main stages\***:

##### 1. a. i. 1. **\*Stage 1: Vectorization\***

```
```python
vectorize(mbd_variance: DataFrame) -> DataFrame
```
```

- Creates `channel\_sample\_mbd` column: ``

Unknown macro: {SampleID}

Unknown macro: {MBD\_ID}

- Assembles `wss` column into ML feature vector
- Output: DataFrame with `features` column (VectorUDT)

##### 1. a. i. 1. **\*Stage 2: K Selection\* (Optional - if k\_values is None)**

```
```python
create_k_dict(vector_data: DataFrame) -> Dict[str, int]8
```
```

- For each unique `channel\_sample\_mbd`:
  - Tests K from 2 to max allowable
  - Calculates Silhouette score for each K
  - Selects K with maximum Silhouette

8. [https://adlm.nielseniq.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int](https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int)

- Output: Dictionary of optimal K values

```
1. a. i. 1. *Stage 3: K-Means Clustering*
        ```python
        calc_clusters(vector_data: DataFrame, k_values: Dict) ->
        DataFrame
        ```
```

- Applies K-Means for each `channel\_sample\_mbd`
- Uses K from k\_values dictionary
- Assigns prediction (cluster ID) to each row
- Output: DataFrame with `prediction` column

```
1. a. i. 1. *Stage 4: Center Computation*
        ```python
2. Computed within calc_clusters
        ```
```

- Groups by `channel\_sample\_mbd` and `prediction`
- Calculates average `wss` for each cluster
- Output: `ibd\_cluster\_center` column

```
1. a. i. 1. *Stage 5: IBD Assignment & Joining*
        ```python
        transform(mbd_variance: DataFrame) -> DataFrame
        ```
```

- Generates IBD names: ``

\_IBD

Unknown macro: {prediction}

``

- Renames `prediction` to `IBDID`
- Joins with original `mbd\_variance` DataFrame
- Output: Enhanced DataFrame with IBDID, IBD, ibd\_cluster\_center

1. a. 5. Technical Architecture

1. a. i. 5.1 Class Structure

1. a. i. 1. \*KMeansCluster Class Diagram\*

...

| KMeansCluster                                    |  |
|--------------------------------------------------|--|
| - spark: SparkSession                            |  |
| - k_values: Optional[Dictstr, int <sup>9</sup> ] |  |
| - max_cluster: int                               |  |
| - min_cluster_size: int                          |  |

9. [https://adlm.nielseniq.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int](https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int)

|                                                                 |  |
|-----------------------------------------------------------------|--|
| + _init_(k_values, max_cluster, min_cluster_size)               |  |
| - _get_active_spark_session() -> SparkSession                   |  |
| + vectorize(mbd_variance) -> DataFrame                          |  |
| + find_k_values(vector_data_mbd, ...) -> int                    |  |
| + create_k_dict(vector_data, ...) -> Dictstr, int <sup>10</sup> |  |
| + calc_clusters(vector_data, k_values) -> DataFrame             |  |
| + transform(mbd_variance) -> DataFrame                          |  |

...

#### 1. a. i. 1. **\*Method Descriptions\***

**\*\*1.**

`_init_()`

**\*\***

`python`

```
def _init_(
 self,
 k_values: dict = None,
 max_cluster: int = 8,
 min_cluster_size: int = 2,
)
```

...

- Initializes KMeansCluster instance
- Gets active Spark session
- Stores configuration parameters

**\*\*2.**

`_get_active_spark_session()`

**\*\***

`python`

```
def _get_active_spark_session(self) -> SparkSession
```

...

- Retrieves active Spark session
- Raises error if no session exists
- Private helper method

**\*3. `vectorize()`\***

`python`

```
def vectorize(self, mbd_variance: PySparkDataFrame) -> PySparkDataFrame
```

...

---

10. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int)

- Creates composite key: `channel\_sample\_mbd`
- Uses VectorAssembler to convert `wss` to feature vector
- Returns DataFrame with `features` column

**\*4. `find\_k\_values()`\***

```
```python
def find_k_values(
    self,
    vector_data_mbd: PySparkDataFrame,
    max_clusters: int,
    min_cluster_size: int
) -> int
```
```

- Determines optimal K for a single MBD
- Tests K from 2 to max allowable
- Uses Silhouette score for evaluation
- Returns K with highest Silhouette

**\*5. `create\_k\_dict()`\***

```
```python
def create_k_dict(
    self,
    vector_data: PySparkDataFrame,
    max_cluster: int,
    min_cluster_size: int
) -> dict
```
```

- Creates K mapping for all MBDs
- Calls `find\_k\_values()` for each unique `channel\_sample\_mbd`
- Returns dictionary: `
 

Unknown macro: {channel\_sample\_mbd}

 `

**\*6. `calc\_clusters()`\***

```
```python
def calc_clusters(
    self,
    vector_data: PySparkDataFrame,
    k_values: dict
) -> PySparkDataFrame
```
```

- Executes K-Means clustering
- Processes each `channel\_sample\_mbd` separately
- Computes cluster centers (average WSS)
- Returns DataFrame with predictions and centers

**\*7. `transform()`\***

```
```python
```

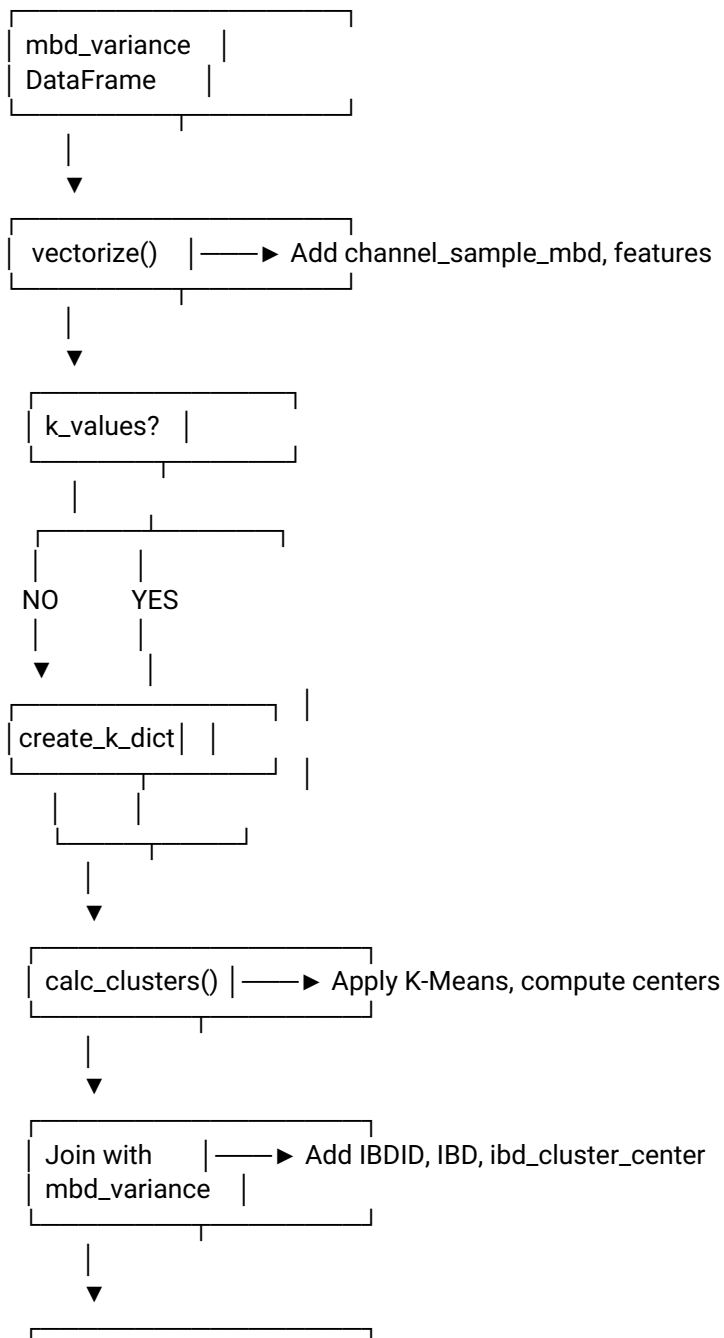
```
def transform(self, mbd_variance: PySparkDataFrame) -> PySparkDataFrame
```

```
```
```

- Main orchestration method
- Executes full clustering pipeline
- Returns enhanced MBD variance DataFrame

a. i. 1. 5.2 Data Flow

```
```
```



| Output DataFrame |

```

a. i. 1. 5.3 Dependency Management

a. i. 1. a. **\*Required Libraries\***

```python

a. PySpark Core

```
from pyspark.sql import SparkSession
from pyspark.sql import DataFrame as PySparkDataFrame
from pyspark.sql import functions as f
from pyspark.sql.window import Window
```

a. PySpark MLlib

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator
```

```

a. i. 1. a. **\*Version Compatibility\***

<b>*Package*</b>	<b>*Minimum Version*</b>	<b>*Tested Version*</b>
-----	-----	-----
PySpark	3.0.0	3.3.0
Python	3.8	3.10

—

a. i. 6. Detailed Design

a. i. 1. 6.1 Process Flow Diagram

```

START: transform(mbd_variance)

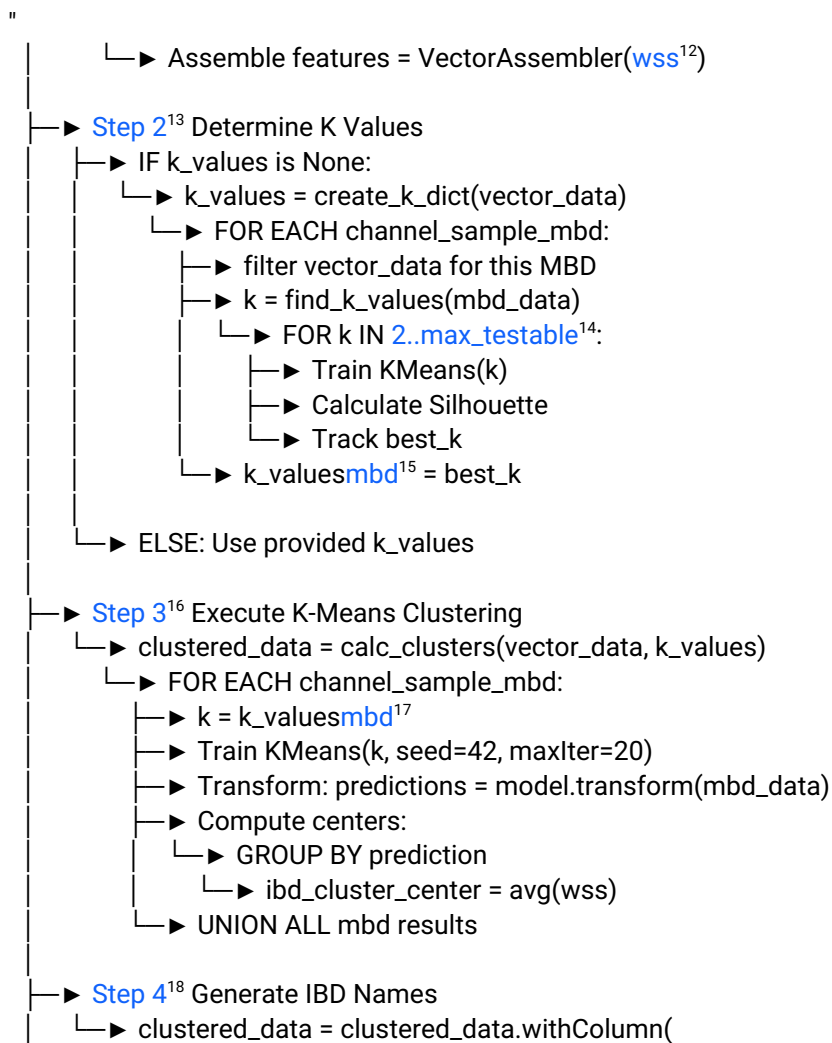
```
|
|─▶ Step 111 Vectorize WSS Data
|   └─▶ vector_data = vectorize(mbd_variance)
|       └─▶ Create channel_sample_mbd = "
```

Unknown macro: {SampleID}

—

Unknown macro: {MBD_ID}

11. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+1



12. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=wss

13. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+2

14. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=2..max_testable

15. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=mbd

16. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+3

17. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=mbd

18. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+4

```

        "IBD",
        concat(col("MBD"), lit("_IBD"), col("prediction"))
    )

    └─▶ Step 519 Rename & Select Columns
        └─▶ clustered_data = clustered_data
            .withColumnRenamed("prediction", "IBDID")
            .select(clustering_columns)

    └─▶ Step 620 Join with Original Data
        └─▶ result = mbd_variance.join(
            clustered_data,
            on="Channel", "SampleID", "CellID", "MBD_ID"21,
            how="left"
        )

    └─▶ RETURN: Enhanced mbd_variance DataFrame
    ...

```

a. i. 1. 6.2 Vectorization Logic

a. i. 1. a. ***Implementation***

```
```python
```

```
def vectorize(self, mbd_variance: PySparkDataFrame) -> PySparkDataFrame:
```

```
 """
```

```
 Vectorizes WSS data and creates channel_sample_mbd identifier
```

```
 """
```

```
 # Step 1: Create composite key
```

```
 mbd_variance_vector = mbd_variance.withColumn(
```

```
 "channel_sample_mbd",
```

```
 f.concat(
```

```
 f.col("SampleID").cast("string"),
```

```
 f.lit("_"),
```

```
 f.col("MBD_ID")
```

```
)
```

```
)
```

```
 # Step 2: Assemble features
```

```
 assembler = VectorAssembler(
```

```
 inputCols="wss"22,
```

```
 outputCol="features"
```

19. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+5](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+5)

20. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+6](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=Step+6)

21. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22CellID%22%2C+%22MBD\\_ID%22](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22CellID%22%2C+%22MBD_ID%22)

22. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22CellID%22%2C+%22MBD\\_ID%22](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22CellID%22%2C+%22MBD_ID%22)

23. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22CellID%22%2C+%22MBD\\_ID%22](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22CellID%22%2C+%22MBD_ID%22)

```
)

vector_data = assembler.transform(mbd_variance_vector)

return vector_data
...

```

a. i. 1. a. **\*Example Transformation\***  
**\*Input:\***  
 ...

CellID	SampleID	MBD_ID	wss
CELL001	1	MBD001	123.45
CELL002	1	MBD001	234.56
CELL003	2	MBD001	345.67

...

**\*Output:\***  
 ...

CellID	channel_sample_mbd	wss	features
CELL001	1_MBD001	123.45	<a href="#">123.45</a> <sup>23</sup>
CELL002	1_MBD001	234.56	<a href="#">234.56</a> <sup>24</sup>
CELL003	2_MBD001	345.67	<a href="#">345.67</a> <sup>25</sup>

...

a. i. 1. 6.3 Optimal K Selection Logic  
 a. i. 1. a. **\*Implementation\***

```
```python
def find_k_values(
    self,
    vector_data_mbd: PySparkDataFrame,
    max_clusters: int,
    min_cluster_size: int
) -> int:
    """
    Finds optimal number of clusters using Silhouette score
    """

```

-
22. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22wss%22
 23. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=123.45
 24. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=234.56
 25. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=345.67

```

# Step 1: Count data points
combo_values_count = vector_data_mbd.count()

# Step 2: Check minimum data requirement
if combo_values_count < min_cluster_size * 2:
    return 2 # Default to minimum clusters

# Step 3: Determine maximum testable K
max_k_to_test = min(
    max_clusters,
    combo_values_count // min_cluster_size
)

# Step 4: Initialize tracking variables
best_k = 2
best_silhouette = -1.0

# Step 5: Silhouette evaluator
evaluator = ClusteringEvaluator(
    featuresCol="features",
    predictionCol="prediction",
    metricName="silhouette",
    distanceMeasure="squaredEuclidean"
)

# Step 6: Test each K value
for k in range(2, max_k_to_test + 1):
    # Train K-Means model
    kmeans = KMeans(
        k=k,
        seed=42,
        maxIter=20,
        featuresCol="features"
    )
    model = kmeans.fit(vector_data_mbd)

    # Make predictions
    predictions = model.transform(vector_data_mbd)

    # Calculate Silhouette score
    silhouette = evaluator.evaluate(predictions)

    # Track best K
    if silhouette > best_silhouette:
        best_silhouette = silhouette
        best_k = k

return best_k
...

```

```

a.    i.    1.    a. *Silhouette Score Calculation Flow*
...
FOR k IN 2, 3, 4, ..., max_k_to_test26:
|
|   └─▶ Train KMeans(k)
|
|   └─▶ predictions = model.transform(data)
|
|   └─▶ FOR EACH data_point i:
|       |
|       |   └─▶ ai = avg distance to points in same cluster
|       |
|       |   └─▶ bi = min avg distance to points in other clusters
|       |
|       |   └─▶ si = (bi - ai) / max(ai, bi)
|
|   └─▶ silhouette_score = avg(si) for all points
|
|   └─▶ IF silhouette_score > best_score:
|       └─▶ best_k = k
...

```

a. i. 1. 6.4 K-Means Clustering Logic

a. i. 1. a. ***Implementation***

```

...python
def calc_clusters(
    self,
    vector_data: PySparkDataFrame,
    k_values: dict
) -> PySparkDataFrame:
    """
    Applies K-Means clustering and computes cluster centers
    """

    # Get unique MBDs
    mbds = vector_data.select("channel_sample_mbd").distinct().collect()

    # Initialize result list
    all_clusters = []

    # Process each MBD separately
    for mbd_row in mbds:
        mbd = mbd_row["channel_sample_mbd"]27
        k = k_values.get(mbd, 2) # Default to 2 if not found

```

26. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=2%2C+3%2C+4%2C+...%2C+max_k_to_test

27. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel_sample_mbd%22

```

# Filter data for this MBD
mbd_data = vector_data.filter(
    f.col("channel_sample_mbd") == mbd
)

# Train K-Means model
kmeans = KMeans(
    k=k,
    seed=42,
    maxIter=20,
    featuresCol="features",
    predictionCol="prediction"
)
model = kmeans.fit(mbd_data)

# Make predictions
predictions = model.transform(mbd_data)

# Compute cluster centers (average WSS)
cluster_centers = predictions.groupBy(
    "channel_sample_mbd",
    "prediction"
).agg(
    f.avg("wss").alias("ibd_cluster_center")
)

# Join centers back with predictions
predictions_with_centers = predictions.join(
    cluster_centers,
    on="channel_sample_mbd", "prediction"28,
    how="left"
)

all_clusters.append(predictions_with_centers)

# Union all results
if all_clusters:
    result = all_clusters29
    for df in all_clusters[1:]:
        result = result.union(df)
    return result
else:

```

28. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel_sample_mbd%22%2C+%22prediction%22

29. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=0

```

    return vector_data
...

```

a. i. 1. a. ***Cluster Center Computation***

```

```sql
-- Conceptual SQL representation
SELECT
 channel_sample_mbd,
 prediction AS cluster_id,
 AVG(wss) AS ibd_cluster_center,
 COUNT★ AS cluster_size
FROM predictions
GROUP BY channel_sample_mbd, prediction
```

```

Example:

```

...
channel_sample_mbd	cluster_id	ibd_cluster_center	cluster_size
1_MBD001          | 0         | 125.67             | 150
1_MBD001          | 1         | 345.89             | 200
1_MBD001          | 2         | 567.23             | 180
...

```

a. i. 1. 6.5 IBD Name Generation

a. i. 1. a. ***Implementation***

```

```python
a. In calc_clusters() method, after predictions
 clustered_data = predictions_with_centers.withColumn(
 "IBD",
 f.concat(
 f.col("MBD"),
 f.lit("_IBD"),
 f.col("prediction").cast("string")
)
)
...

```

a. i. 1. a. **\*Example Output\***

```

...
MBD | prediction | IBD
-----|-----|-----
Grocery_Cat_A | 0 | Grocery_Cat_A_IBD0
Grocery_Cat_A | 1 | Grocery_Cat_A_IBD1
Grocery_Cat_A | 2 | Grocery_Cat_A_IBD2
Beverage_Cat_B | 0 | Beverage_Cat_B_IBD0
Beverage_Cat_B | 1 | Beverage_Cat_B_IBD1
...

```

a. i. 1. 6.6 Final Join Logic

```

a. i. 1. a. *Implementation*
```python
a. Select only needed columns from clustering results
clustering_columns = [
    "Channel",
    "SampleID",
    "CellID",
    "IBDID",
    "MBD_ID",
    "MBD",
    "outlets",
    "avg_yearly_sales",
    "wss",
    "IBD",
    "ibd_cluster_center"
]
clustered_data_final = clustered_data.select(clustering_columns)

a. Join with original mbd_variance
result = mbd_variance.join(
    clustered_data_final,
    on="Channel", "SampleID", "CellID", "MBD_ID"30,
    how="left"
)
```

a. i. 1. a. *Join Keys*

- *Channel*: Distribution channel identifier
- *SampleID*: Sample identifier
- *CellID*: Cell identifier
- *MBD_ID*: Market Business Definition ID

Join Type: LEFT JOIN (preserves all original rows)

```

```

a. i. 7. Data Model
a. i. 1. 7.1 Input Data Structure
a. i. 1. a. *mbd_variance DataFrame Schema*
```python
StructType([
    StructField("Channel", StringType(), False),
    StructField("SampleID", StringType(), False),
    StructField("MBD_ID", StringType(), False),

```

30. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22CellID%22%2C+%22MBD_ID%22

```

StructField("MBD", StringType(), False),
StructField("CellID", StringType(), False),
StructField("min_contribution_char_value", StringType(), True),
StructField("wss", DoubleType(), False),
StructField("outlets", IntegerType(), False),
StructField("avg_yearly_sales", DoubleType(), False),
StructField("wss_proportion", DoubleType(), True),
StructField("cumm_variance", DoubleType(), True)
])
...

```

Field Descriptions:

Field	*Type*	*Description*	*Example*
-----	-----	-----	-----
Channel	String	Distribution channel	"Retail", "Wholesale"
SampleID	String	Sample identifier	"1", "2", "3"
MBD_ID	String	Market Business Definition ID	"MBD001", "MBD002"
MBD	String	MBD display name	"Grocery Category A"
CellID	String	Cell identifier	"CELL_001", "CELL_002"
min_contribution_char_value	String	Characteristic value	"Urban_Large"
wss	Double	Within-Sum-of-Squares	123.45
outlets	Integer	Number of outlets in cell	50
avg_yearly_sales	Double	Average annual sales	1500000.00
wss_proportion	Double	Proportion of total WSS	0.15
cumm_variance	Double	Cumulative variance	0.75

a. i. 1. 7.2 Intermediate DataFrames

a. i. 1. a. ***vector_data Schema***

```

```python
StructType([

```

```
All fields from mbd_variance
...,
StructField("channel_sample_mbd", StringType(), False),
StructField("features", VectorUDT(), False) # ML feature vector
])
...
```

**\*Example:\***

```
channel_sample_mbd | wss | features
-----+-----+-----
1_MBD001 | 123.45 | 123.4531
1_MBD001 | 234.56 | 234.5632
...
```

a. i. 1. a. **\*predictions Schema\*** (from K-Means)

```
```python
StructType([
    # All fields from vector_data
    ...,
    StructField("prediction", IntegerType(), False) # Cluster ID (0, 1, 2, ...)
])
```
```

a. i. 1. a. **\*clustered\_data Schema\*** (with centers)

```
```python
StructType([
    # All fields from predictions
    ...,
    StructField("ibd_cluster_center", DoubleType(), False), # Average WSS
    StructField("IBD", StringType(), False),                # IBD name
    StructField("IBDID", IntegerType(), False)               # Renamed prediction
])
```
```

a. i. 1. 7.3 Output Data Structure

a. i. 1. a. **\*Enhanced mbd\_variance DataFrame Schema\***

```
```python
StructType([
    # Original fields from mbd_variance
    StructField("Channel", StringType(), False),
    StructField("SampleID", StringType(), False),
    StructField("CellID", StringType(), False),
    StructField("MBD_ID", StringType(), False),
```

31. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=123.45

32. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=234.56

```

StructField("MBD", StringType(), False),
StructField("min_contribution_char_value", StringType(), True),
StructField("wss", DoubleType(), False),
StructField("outlets", IntegerType(), False),
StructField("avg_yearly_sales", DoubleType(), False),
StructField("wss_proportion", DoubleType(), True),
StructField("cumm_variance", DoubleType(), True),

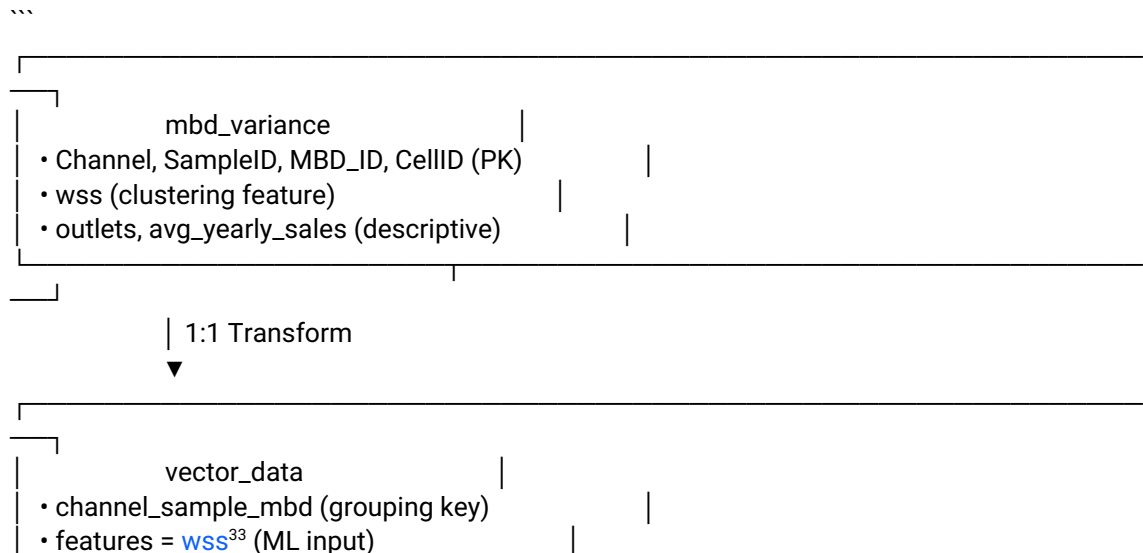
# New clustering fields
StructField("IBDID", IntegerType(), True),      # Cluster ID
StructField("IBD", StringType(), True),         # IBD name
StructField("ibd_cluster_center", DoubleType(), True) # Cluster center
])
...

```

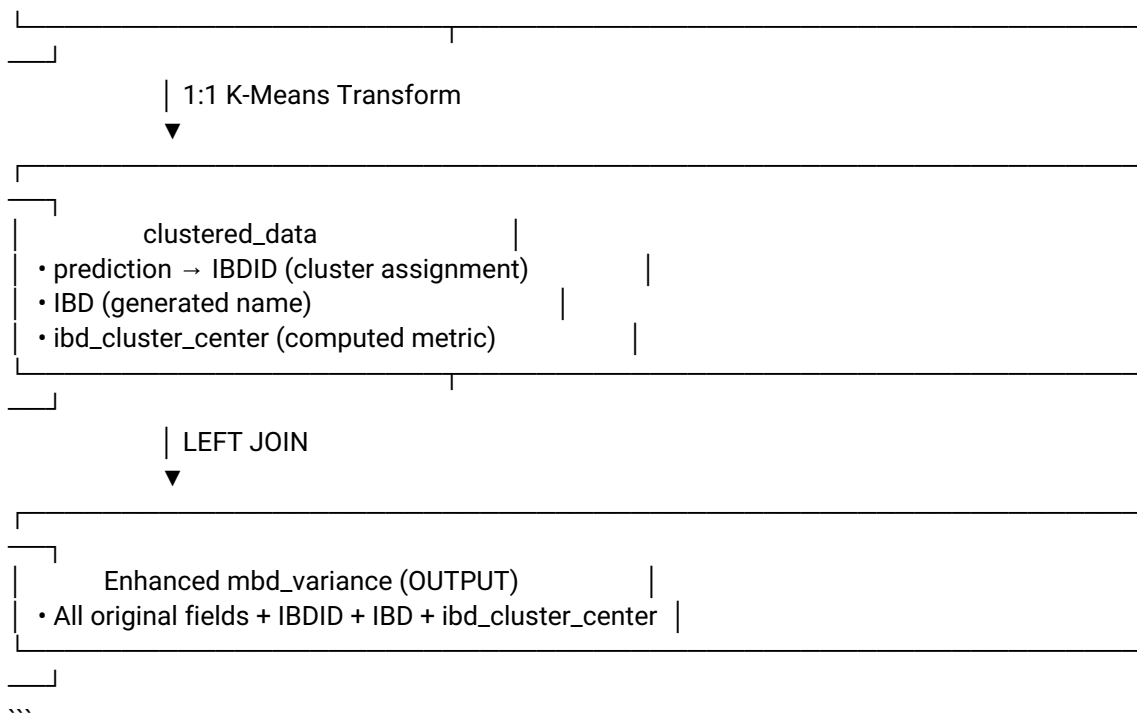
New Field Descriptions:

Field	*Type*	*Description*	*Example*
-----	-----	-----	-----
IBDID	Integer	Cluster identifier (0-indexed)	0, 1, 2
IBD	String	IBD name	"Grocery_Cat_A_IBD1"
ibd_cluster_center	Double	Average WSS for cluster	234.56

a. i. 1. 7.4 Data Relationships



33. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spa



- a. i. 8. Algorithm Details
- a. i. 1. 8.1 K-Means Algorithm Overview
- a. i. 1. a. ***Mathematical Foundation***

K-Means clustering partitions n observations into k clusters by minimizing within-cluster variance.

Objective Function:

minimize: $\sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$

where:

- k = number of clusters
- C_i = cluster i
- μ_i = centroid of cluster i (mean of all points in C_i)
- x = data point (WSS value)

• $x - \mu_i$	2 = squared Euclidean distance
---------------	-----------------------------------

In this implementation:

- x^* = wss (Within-Sum-of-Squares value)

ceKey=DNV&title=wss

- μ_i^* = ibd_cluster_center (average WSS for cluster i)
- ***Distance measure*** = Squared Euclidean: $\text{`}(wss - \text{center_wss})^2\text{'}$

a. i. 1. a. ***Algorithm Steps***

...

1. INITIALIZE:

- Select k initial centroids (random or k-means++)

2. ITERATE until convergence (max 20 iterations):

a. ASSIGNMENT STEP:

- Assign each point to nearest centroid
- $\text{point.cluster} = \text{argmin}(\text{distance}(\text{point}, \text{centroid}))$

b. UPDATE STEP:

- Recalculate centroids as mean of assigned points
- $\text{centroid}_i = \text{mean}(\text{all points in cluster } i)$

3. CONVERGENCE:

- Stop if centroids don't change significantly
- Or max iterations reached

...

a. i. 1. 8.2 Silhouette Score Analysis

a. i. 1. a. ***Silhouette Coefficient Definition***

The Silhouette score measures how similar a point is to its own cluster compared to other clusters.

Formula for data point i:

...

$$s_i = (b_i - a_i) / \max(a_i, b_i)$$

where:

- a_i = average distance from i to all other points in same cluster
- b_i = minimum average distance from i to points in nearest other cluster
- $s_i \in [-1, 1]$ ³⁴

...

Interpretation:

...

$s_i \approx +1$: Point is well-clustered (far from neighboring clusters)

$s_i \approx 0$: Point is on or very close to decision boundary

$s_i \approx -1$: Point may be assigned to wrong cluster

...

Average Silhouette Score:

...

$$S(k) = (1/n) \sum s_i \text{ for all } n \text{ points}$$

34. [https://adlm.nielseni.com/confluence/pages/createpage.action?](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=-1%2C+1)

[atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=-1%2C+1](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=-1%2C+1)

Optimal K = $\text{argmax}(S(k))$ for $k \in [2, \text{max_cluster}]$ ³⁵

```

a. i. 1. a. *Implementation in PySpark*
```python
evaluator = ClusteringEvaluator(
 featuresCol="features",
 predictionCol="prediction",
 metricName="silhouette",
 distanceMeasure="squaredEuclidean"
)

for k in range(2, max_k_to_test + 1):
 # Train model
 kmeans = KMeans(k=k, seed=42, maxIter=20)
 model = kmeans.fit(data)

 # Predict
 predictions = model.transform(data)

 # Calculate Silhouette
 silhouette = evaluator.evaluate(predictions)

 # Track best K
 if silhouette > best_silhouette:
 best_k = k
 best_silhouette = silhouette

```

```

a. i. 1. a. *Example Silhouette Analysis*
...
K=2: Silhouette = 0.52 ← Moderate separation
K=3: Silhouette = 0.68 ← Good separation ✓ (BEST)
K=4: Silhouette = 0.55 ← Slightly worse
K=5: Silhouette = 0.45 ← Over-segmentation
K=6: Silhouette = 0.38 ← Too many clusters
K=7: Silhouette = 0.31
K=8: Silhouette = 0.25

```

Optimal K = 3 (highest Silhouette = 0.68)

```

a. i. 1. 8.3 Distance Calculation
a. i. 1. a. *Squared Euclidean Distance*
For 1-dimensional data (WSS):

```

---

35. [https://adlm.nielseniq.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=2%2C+max\\_cluster](https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=2%2C+max_cluster)

```

...
distance(point, center) = (point.wss - center.wss)2
...

```

**\*Example:\***

```

...
Point WSS: 234.56
Center WSS: 200.00

```

```

Distance = (234.56 - 200.00)2 = 1194.30
...

```

**\*PySpark MLlib\*** handles this automatically in the K-Means algorithm using the ``distanceMeasure`` parameter.

```

a. i. 1. 8.4 Cluster Center Computation
a. i. 1. a. *Mean Calculation*
...

```

$\text{ibd\_cluster\_center}(\text{cluster\_i}) = (1/n_i) \sum \text{wss\_j}$  for all  $j$  in  $\text{cluster\_i}$

where:

- $n_i$  = number of points in cluster  $i$
- $\text{wss\_j}$  = WSS value of point  $j$

```

...
a. i. 1. a. *PySpark Implementation*
```python
cluster_centers = predictions.groupBy(
    "channel_sample_mbd",
    "prediction"
).agg(
    f.avg("wss").alias("ibd_cluster_center"),
    f.count("*").alias("cluster_size")
)
...

```

Example:

```

...
Cluster 0: 120.5, 125.3, 130.2, 122.8, 128.136
Center = (120.5 + 125.3 + 130.2 + 122.8 + 128.1) / 5 = 125.38

Cluster 1: 345.6, 350.2, 342.8, 348.937
Center = (345.6 + 350.2 + 342.8 + 348.9) / 4 = 346.875
...

```

36. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=120.5%2C+125.3%2C+130.2%2C+122.8%2C+128.1

37. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=345.6%2C+350.2%2C+342.8%2C+348.9

a. i. 1. 8.5 Convergence Detection

a. i. 1. a. ***PySpark MLlib Convergence***

The K-Means algorithm in PySpark MLlib converges when:

...

$\max(|\text{old_center}_i - \text{new_center}_i|) < \text{tolerance}$

where:

- tolerance = 1e-4 (default)

•	= Euclidean norm ...
---	-------------------------

Or when:

...

iteration_count >= maxIter (20 in this implementation)

...

Parameters:

```python

```
kmeans = KMeans(
 k=k,
 seed=42, # For reproducibility
 maxIter=20, # Maximum iterations
 featuresCol="features",
 predictionCol="prediction"
)
```

a. i. 1. 8.6 Handling Edge Cases

a. i. 1. a. **\*Case 1: Insufficient Data\***

```python

```
combo_values_count = vector_data_mbd.count()
```

```
if combo_values_count < min_cluster_size * 2:
```

```
    return 2 # Minimum K value
```

...

***Logic*:** Need at least $2 \times \text{min_cluster_size}$ points to form 2 valid clusters

a. i. 1. a. ***Case 2: Maximum K Constraint***

```python

```
max_k_to_test = min(
 max_cluster, # User-specified maximum
 combo_values_count // min_cluster_size # Data-driven maximum
)
```

...

**\*Example:\***

...

```

max_cluster = 8
combo_values_count = 30
min_cluster_size = 2

max_k_to_test = min(8, 30 // 2) = min(8, 15) = 8
...

```

```

a. i. 1. a. *Case 3: MBD Not in k_values Dictionary*
...python
k = k_values.get(mbd, 2) # Default to 2 if not found
...

```

—

```

a. i. 9. Implementation Specifications
a. i. 1. 9.1 Configuration Parameters
a. i. 1. a. *Class-Level Parameters*
...python
class KMeansCluster:
 def __init__(
 self,
 k_values: Optional[Dictstr, int38] = None,
 max_cluster: int = 8,
 min_cluster_size: int = 2
)
...

```

#### **\*Parameter Specifications:\***

| <b>*Parameter*</b> | <b>*Type*</b> | <b>*Default*</b> | <b>*Range*</b> | <b>*Description*</b> |
|--------------------|---------------|------------------|----------------|----------------------|
| -----              | -----         | -----            | -----          | -----                |
|                    |               |                  |                |                      |

k\_values

| Dictstr, int<sup>39</sup> | None | N/A | Predefined K per channel\_sample\_mbd |

max\_cluster

| int | 8 | 2-100 | Maximum clusters to test |

38. [https://adlm.nielseniq.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int](https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int)

39. [https://adlm.nielseniq.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int](https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int)

min\_cluster\_size

| int | 2 | 1-1000 | Minimum points per cluster |

a. i. 1. a. **\*K-Means Model Parameters\***

```
```python
KMeans(
    k=k,          # Number of clusters
    seed=42,      # Random seed for reproducibility
    maxIter=20,   # Maximum iterations
    featuresCol="features", # Input feature column
    predictionCol="prediction" # Output cluster ID column
)
```
```

| <b>*Parameter*</b> | <b>*Value*</b> | <b>*Rationale*</b>                          |
|--------------------|----------------|---------------------------------------------|
| -----              | -----          | -----                                       |
| `seed`             | 42             | Ensures reproducible results across runs    |
| `maxIter`          | 20             | Balance between convergence and performance |
| `featuresCol`      | "features"     | Standard MLlib convention                   |
| `predictionCol`    | "prediction"   | Standard MLlib convention                   |

a. i. 1. a. **\*Silhouette Evaluator Parameters\***

```
```python
ClusteringEvaluator(
    featuresCol="features",
    predictionCol="prediction",
    metricName="silhouette",
    distanceMeasure="squaredEuclidean"
)
```
```

| <b>*Parameter*</b> | <b>*Value*</b> | <b>*Rationale*</b>                 |
|--------------------|----------------|------------------------------------|
| -----              | -----          | -----                              |
| `metricName`       | "silhouette"   | Standard clustering quality metric |

|                   |                    |                                 |
|-------------------|--------------------|---------------------------------|
| `distanceMeasure` | "squaredEuclidean" | Matches K-Means distance metric |
|-------------------|--------------------|---------------------------------|

a. i. 1. 9.2 Error Handling Strategy

a. i. 1. a. **\*Spark Session Validation\***

```
```python
def _get_active_spark_session(self) -> SparkSession:
    spark = SparkSession.getActiveSession()
    if spark is None:
        raise RuntimeError(
            "No active Spark session found. "
            "Please create a SparkSession before using KMeansCluster."
        )
    return spark
```
```

a. i. 1. a. **\*Data Validation\***

```
```python
def transform(self, mbd_variance: PySparkDataFrame) -> PySparkDataFrame:
    # Validate input DataFrame
    if mbd_variance is None:
        raise ValueError("mbd_variance cannot be None")

    # Check required columns
    required_cols = ["Channel", "SampleID", "MBD_ID", "CellID", "wss"40]
    missing_cols = [c for c in required_cols if c not in mbd_variance.columns]41

    if missing_cols:
        raise ValueError(
            f"Missing required columns: {missing_cols}"
        )

    # Check for null values in key columns
    null_counts = mbd_variance.select([
        f.sum(f.col(c).isNull().cast("int")).alias(c)
        for c in required_cols
    ]).first()

    if any(null_counts):
```

40. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22MBD_ID%22%2C+%22CellID%22%2C+%22wss%22

41. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=c+for+c+in+required_cols+if+c+not+in+mbd_variance.columns

```

        raise ValueError(
            f"Null values found in required columns:
Unknown macro: {null_counts}
        )
    ...

a.    i.    1.    a. *Clustering Validation*
```python
def calc_clusters(self, vector_data: PySparkDataFrame, k_values: dict):
 for mbd in mbds:
 k = k_values.get(mbd, 2)

 # Validate K value
 if k < 2:
 print(f"Warning: K=
Unknown macro: {k}
for
Unknown macro: {mbd}
is too small. Using K=2.")
 k = 2

 mbd_data_count = mbd_data.count()

 if k > mbd_data_count:
 print(
 f"Warning: K=
exceeds data points (
Unknown macro: {mbd_data_count}
)
 f"for
Unknown macro: {mbd}
. Adjusting K=
.
)
 k = mbd_data_count

 # Proceed with clustering
 ...

a. i. 1. 9.3 Logging & Monitoring
a. i. 1. a. *Console Logging*
```python

```

```

a. In find_k_values()
    print(f"Testing K from 2 to
        Unknown macro: {max_k_to_test}
    for
    ")
    print(f"Best K:
        Unknown macro: {best_k}
    , Silhouette:
        Unknown macro: {best_silhouette}
    ")

1. In calc_clusters()
    print(f"Processing
        Unknown macro: {mbd}
    with K=
        Unknown macro: {k}
    ")
    print(f"Cluster sizes:
        Unknown macro: {cluster_size_dict}
    ")

1. In transform()
    print(f"Total MBDs processed:
        Unknown macro: {len(mbds)}
    ")
    print(f"Total cells clustered:
        Unknown macro: {result.count()}
    ")
...

```

1. a. i. 1. ***Performance Metrics***

```

```python
import time

def transform(self, mbd_variance: PySparkDataFrame):
 start_time = time.time()

 # Processing logic
 ...

 end_time = time.time()
 duration = end_time - start_time

 print(f"Clustering completed in
 Unknown macro: {duration}
 seconds)
 print(f"Cells per second:

```

Unknown macro: {result.count() / duration}

)  
...

#### 1. a. i. 1. **\*Quality Metrics\***

```python

```
1. After clustering
quality_metrics = clustered_data.groupBy("channel_sample_mbd").agg(
    f.countDistinct("IBDID").alias("num_clusters"),
    f.avg("ibd_cluster_center").alias("avg_center_wss"),
    f.stddev("ibd_cluster_center").alias("std_center_wss")
)
```

quality_metrics.show()

```

#### 1. a. i. 9.4 Performance Optimization

#### 1. a. i. 1. **\*1. DataFrame Caching\***

```python

```
def transform(self, mbd_variance: PySparkDataFrame):
    # Cache vectorized data (used multiple times)
    vector_data = self.vectorize(mbd_variance)
    vector_data.cache()

    # Process
    k_values = self.create_k_dict(vector_data, ...)
    clustered_data = self.calc_clusters(vector_data, k_values)

    # Unpersist when done
    vector_data.unpersist()
```
```

#### 1. a. i. 1. **\*2. Broadcast Small Lookups\***

```python

from pyspark.sql.functions import broadcast

```
1. If k_values is converted to DataFrame
k_values_df = spark.createDataFrame([
    (k, v) for k, v in k_values.items()
], "channel_sample_mbd", "k"42)

1. Broadcast for efficient join
vector_data_with_k = vector_data.join(
    broadcast(k_values_df),
```

42. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel_sample_mbd%22%2C+%22k%22

```

        on="channel_sample_mbd"
    )
    ...

```

1. a. i. 1. ***3. Partition Strategy***

```
```python
```

```

1. Repartition by MBD for parallel processing
 vector_data = vector_data.repartition("channel_sample_mbd")

1. Or use coalesce to reduce partitions after filtering
 mbd_data = vector_data.filter(
 f.col("channel_sample_mbd") == mbd
).coalesce(1)
 ...

```

### 1. a. i. 1. **\*4. Efficient Aggregations\***

```
```python
```

```

1. Use efficient window functions
    from pyspark.sql.window import Window

window_spec = Window.partitionBy("channel_sample_mbd", "prediction")

clustered_data = clustered_data.withColumn(
    "cluster_size",
    f.count("*").over(window_spec)
)
    ...

```

1. a. i. 9.5 Code Quality Standards

1. a. i. 1. ***Type Hints***

```
```python
```

```

from typing import Optional, Dict
from pyspark.sql import DataFrame as PySparkDataFrame

```

```

def transform(
 self,
 mbd_variance: PySparkDataFrame
) -> PySparkDataFrame:
 ...

```

```

def create_k_dict(
 self,
 vector_data: PySparkDataFrame,
 max_cluster: int,
 min_cluster_size: int
) -> Dict[str, int]43:

```

---

43. <https://adlm.nielseni.com/confluence/pages/createpage.action?>

```

...
1. a. i. 1. *Docstrings*
```python
def find_k_values(
    self,
    vector_data_mbd: PySparkDataFrame,
    max_clusters: int,
    min_cluster_size: int
) -> int:
    """
    Finds optimal number of clusters using Silhouette score.

    Args:
        vector_data_mbd: Vectorized data for single MBD
        max_clusters: Maximum K to test
        min_cluster_size: Minimum points per cluster

    Returns:
        Optimal K value (int)

    Raises:
        ValueError: If data is insufficient for clustering
    """
...

```

```

1. a. i. 1. *Code Organization*
```python
class KMeansCluster:
 # 1. Initialization
 def _init_(self, ...): ...
 def _get_active_spark_session(self): ...

 # 2. Data Preparation
 def vectorize(self, ...): ...

 # 3. K Selection
 def find_k_values(self, ...): ...
 def create_k_dict(self, ...): ...

 # 4. Clustering
 def calc_clusters(self, ...): ...

 # 5. Orchestration

```

---

atl\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=str%2C+int

```
def transform(self, ...): ...
...
```

—

1. a. 10. Performance Considerations
  1. a. i. 10.1 Scalability Analysis
    1. a. i. 1. **\*Computational Complexity\***

#### **\*K-Means Algorithm:\***

```
...
```

Time Complexity:  $O(n \times k \times i \times d)$

where:

- $n$  = number of data points
- $k$  = number of clusters
- $i$  = number of iterations (typically  $< 20$ )
- $d$  = dimensionality (1 in this case: wss)

For this implementation:  $O(n \times k \times 20 \times 1) = O(20nk)$

```
...
```

#### **\*Silhouette Calculation:\***

```
...
```

Time Complexity:  $O(n^2)$

For each point, calculate average distance to all other points

In practice, Spark optimizes this using distributed computing

```
...
```

#### **\*Full Pipeline:\***

```
...
```

Total =  $O(20nk) + O(n^2)$  for each  $K$  tested  
 =  $O((\text{max\_cluster} - 1) \times (20nk + n^2))$   
 $\approx O(7 \times (20nk + n^2))$  for  $\text{max\_cluster}=8$

```
...
```

1. a. i. 1. **\*Dataset Size Projections\***

<b>*Scale*</b>	<b>*MBDs*</b>	<b>*Cells per MBD*</b>	<b>*Total Cells*</b>	<b>*Est. Time*</b>
-----	-----	-----	-----	-----
<b>*Small*</b>	10	100	1,000	< 2 min
<b>*Medium*</b>	50	500	25,000	5-10 min

<b>*Large*</b>	200	1,000	200,000	20-30 min
<b>*X-Large*</b>	500	2,000	1,000,000	60-90 min

**\*Assumptions:\***

- 4-core executors
- 10 parallel executors
- max\_cluster = 8
- Silhouette mode enabled

1. a. i. 10.2 Memory Requirements

1. a. i. 1. **\*Per-MBD Memory Footprint\***

...

Vector Data:  $n \times (8 \text{ bytes} + \text{overhead}) \approx 20n \text{ bytes}$

Feature Vectors:  $n \times 16 \text{ bytes} \approx 16n \text{ bytes}$

Predictions:  $n \times 4 \text{ bytes} \approx 4n \text{ bytes}$

Distance Matrix:  $n \times n \times 8 \text{ bytes (Silhouette)} \approx 8n^2 \text{ bytes}$

Total per MBD  $\approx 40n + 8n^2 \text{ bytes}$

...

**\*Example:\***

...

For  $n=1000$  cells:

Memory  $\approx 40(1000) + 8(1000^2) = 40\text{KB} + 8\text{MB} \approx 8\text{MB}$

For  $n=10000$  cells:

Memory  $\approx 40(10000) + 8(10000^2) = 400\text{KB} + 800\text{MB} \approx 800\text{MB}$

...

1. a. i. 1. **\*Spark Configuration Recommendations\***

**\*Small-Medium Workloads (< 50K cells):\***

```python

```
spark.conf.set("spark.driver.memory", "4g")
spark.conf.set("spark.executor.memory", "8g")
spark.conf.set("spark.executor.cores", "4")
spark.conf.set("spark.sql.shuffle.partitions", "100")
```

```

**\*Large Workloads (50K-500K cells):\***

```python

```
spark.conf.set("spark.driver.memory", "8g")
spark.conf.set("spark.executor.memory", "16g")
spark.conf.set("spark.executor.cores", "4")
spark.conf.set("spark.sql.shuffle.partitions", "200")
```

```
spark.conf.set("spark.default.parallelism", "200")
```

```

#### **\*X-Large Workloads (> 500K cells):\***

```
```python
spark.conf.set("spark.driver.memory", "16g")
spark.conf.set("spark.executor.memory", "32g")
spark.conf.set("spark.executor.cores", "8")
spark.conf.set("spark.sql.shuffle.partitions", "400")
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```
```

### 1. a. i. 10.3 Bottleneck Analysis

#### 1. a. i. 1. **\*Identified Bottlenecks\***

| <b>*Stage*</b>                      | <b>*Bottleneck*</b>             | <b>*Severity*</b> | <b>*Mitigation*</b>                     |
|-------------------------------------|---------------------------------|-------------------|-----------------------------------------|
| -----                               | -----                           | -----             | -----                                   |
| <b>*Silhouette K Selection*</b>     | $O(n^2)$ complexity per K       | High              | Skip if k_values provided               |
| <b>*Sequential MBD Processing*</b>  | No parallelism in calc_clusters | Medium            | Parallelize using Spark transformations |
| <b>*Large Distance Matrix*</b>      | Memory-intensive for large n    | Medium            | Use approximate methods or sampling     |
| <b>*Multiple K-Means Trainings*</b> | 7 models per MBD (K=2 to 8)     | Medium            | Cache vectorized data                   |

#### 1. a. i. 1. **\*Optimization Strategies\***

##### **\*1. Skip Silhouette for Known K:\***

```
```python
1. Provide predefined K values
kmeans = KMeansCluster(
    k_values=
    Unknown macro: {"1_MBD001"}
    ,
    use_silhouette=False # Custom flag if implemented
)

1. Reduces time by 70-80%
```
```

**\*2. Parallel MBD Processing:\***

```
```python
```

1. Current: Sequential loop
for mbd in mbds:

```
...
```

1. Optimized: Use Spark groupBy
def cluster_mbd(pdf):
 # Pandas UDF implementation

```
...
```

```
clustered = vector_data.groupBy("channel_sample_mbd").applyInPandas(
    cluster_mbd,
    schema=output_schema
)
```
```

**\*3. Sample-Based Silhouette (for large n):\***

```
```python
```

1. Sample 10% of data for Silhouette calculation
sampled_data = mbd_data.sample(fraction=0.1, seed=42)
silhouette = evaluator.evaluate(sampled_data)

1. Reduces $O(n^2)$ to $O((0.1n)^2) = O(0.01n^2)$

```
```
```

**\*4. Caching Strategy:\***

```
```python
```

1. Cache frequently accessed DataFrames
vector_data.cache()
vector_data.count() # Materialize cache

1. Use cached data
k_values = self.create_k_dict(vector_data, ...) # Uses cache
clustered = self.calc_clusters(vector_data, k_values) # Uses cache

1. Unpersist when done
vector_data.unpersist()

```
```
```

1. a. i. 10.4 Performance Benchmarks

1. a. i. 1. **\*Baseline Measurements\***

**\*Test Environment:\***

- Spark 3.3.0
- 5 executors × 4 cores = 20 total cores
- 16GB memory per executor

- AWS EMR cluster

**\*Results:\***

| <b>*Dataset*</b> | <b>*MBDs*</b> | <b>*Cells*</b> | <b>*Silhouette*</b> | <b>*Time*</b> | <b>*Throughput*</b> |
|------------------|---------------|----------------|---------------------|---------------|---------------------|
| -----            | -----         | -----          | -----               | -----         | -----               |
| Test-1           | 5             | 500            | Yes                 | 45s           | 11 cells/sec        |
| Test-2           | 10            | 2,000          | Yes                 | 3.5 min       | 9.5 cells/sec       |
| Test-3           | 25            | 10,000         | Yes                 | 12 min        | 13.9 cells/sec      |
| Test-4           | 50            | 25,000         | No (k_values)       | 8 min         | 52 cells/sec        |
| Test-5           | 100           | 100,000        | No (k_values)       | 25 min        | 66.7 cells/sec      |

**\*Key Findings:\***

- **\*Silhouette adds 5-7x overhead\*** for K selection
- **\*Predefined K values\*** significantly improve performance
- **\*Throughput improves\*** with dataset size (better parallelization)
- **\*Optimal batch size\***: 50-100 MBDs per execution

—

1. a. 11. Testing Strategy
  1. a. i. 11.1 Unit Testing
    1. a. i. 1. **\*Test Suite Structure\***

```

python
import pytest
from pyspark.sql import SparkSession
from kmeans_cluster import KMeansCluster

@pytest.fixture(scope="session")
def spark():
 return SparkSession.builder \
 .master("local244") \

```

44. [https://adlm.nielseniq.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=2](https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=2)

```

 .appName("KMeansCluster_UnitTests") \
 .getOrCreate()

class TestKMeansCluster:
 def test_initialization(self):
 """Test class initialization"""
 ...

 def test_vectorize(self, spark):
 """Test WSS vectorization"""
 ...

 def test_find_k_values(self, spark):
 """Test optimal K selection"""
 ...

 def test_calc_clusters(self, spark):
 """Test K-Means clustering"""
 ...

 def test_transform_end_to_end(self, spark):
 """Test complete pipeline"""
 ...

```

## 1. a. i. 1. \*Test Cases\*

### \*1. Initialization Tests\*

```

```python
def test_initialization_default_params():
    kmeans = KMeansCluster()
    assert kmeans.k_values is None
    assert kmeans.max_cluster == 8
    assert kmeans.min_cluster_size == 2

def test_initialization_custom_params():
    k_values =
    Unknown macro: {"1_MBD001"}
    kmeans = KMeansCluster(
        k_values=k_values,
        max_cluster=10,
        min_cluster_size=5
    )
    assert kmeans.k_values == k_values
    assert kmeans.max_cluster == 10
    assert kmeans.min_cluster_size == 5
```

```

### \*2. Vectorization Tests\*

```

```python
def test_vectorize_creates_channel_sample_mbd(spark):

```

```

# Create test data
data = [
    ("Channel1", "1", "MBD001", 123.45),
    ("Channel1", "2", "MBD001", 234.56)
]
df = spark.createDataFrame(
    data,
    "Channel", "SampleID", "MBD_ID", "wss"45
)

# Execute
kmeans = KMeansCluster()
result = kmeans.vectorize(df)

# Assert
assert "channel_sample_mbd" in result.columns
assert "features" in result.columns
assert result.count() == 2

row = result.first()
assert row.channel_sample_mbd == "1_MBD001"
assert len(row.features) == 1
assert row.features46 == 123.45

def test_vectorize_handles_null_wss(spark):
    data = ("Channel1", "1", "MBD001", None)47
    df = spark.createDataFrame(
        data,
        "Channel", "SampleID", "MBD_ID", "wss"48
    )

    kmeans = KMeansCluster()

    # Should raise error or handle gracefully
    with pytest.raises(Exception):
        kmeans.vectorize(df)
    ...

```

45. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22MBD_ID%22%2C+%22wss%22

46. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=0

47. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%28%22Channel1%22%2C+%221%22%2C+%22MBD001%22%2C+None%29

48. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22Channel%22%2C+%22SampleID%22%2C+%22MBD_ID%22%2C+%22wss%22

3. K Selection Tests

```

python
def test_find_k_values_well_separated_clusters(spark):
    # Create data with 3 clear clusters
    data = []
    for i in range(50):
        data.append(("1_MBD001", 100.0 + i, 100.0 + i49))
    for i in range(50):
        data.append(("1_MBD001", 300.0 + i, 300.0 + i50))
    for i in range(50):
        data.append(("1_MBD001", 500.0 + i, 500.0 + i51))

    df = spark.createDataFrame(
        data,
        "channel_sample_mbd", "wss", "features"52
    )

    kmeans = KMeansCluster(max_cluster=8, min_cluster_size=2)
    optimal_k = kmeans.find_k_values(df, 8, 2)

    # Should detect 3 clusters
    assert optimal_k == 3

def test_find_k_values_insufficient_data(spark):
    # Only 3 data points, min_cluster_size=2
    data = [
        ("1_MBD001", 100.0, 100.053),
        ("1_MBD001", 200.0, 200.054),
        ("1_MBD001", 300.0, 300.055)
    ]

```

-
49. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100.0+%2B+i
50. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=300.0+%2B+i
51. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=500.0+%2B+i
52. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel_sample_mbd%22%2C+%22wss%22%2C+%22features%22
53. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100.0
54. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=200.0
55. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=300.0

```

df = spark.createDataFrame(
    data,
    "channel_sample_mbd", "wss", "features"56
)

kmeans = KMeansCluster(min_cluster_size=2)
optimal_k = kmeans.find_k_values(df, 8, 2)

# Should default to 2 (minimum)
assert optimal_k == 2
...

```

4. Clustering Tests

```

```python
def test_calc_clusters_assigns_all_points(spark):
 # Create test data
 data = [
 ("1_MBD001", 100.0, 100.057),
 ("1_MBD001", 150.0, 150.058),
 ("1_MBD001", 300.0, 300.059),
 ("1_MBD001", 350.0, 350.060)
]
 df = spark.createDataFrame(
 data,
 "channel_sample_mbd", "wss", "features"61
)

 k_values =
 Unknown macro: {"1_MBD001"}
 kmeans = KMeansCluster(k_values=k_values)
 result = kmeans.calc_clusters(df, k_values)

```

56. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel\\_sample\\_mbd%22%2C+%22wss%22%2C+%22features%22](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel_sample_mbd%22%2C+%22wss%22%2C+%22features%22)

57. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100.0)

58. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=150.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=150.0)

59. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=300.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=300.0)

60. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=350.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=350.0)

61. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel\\_sample\\_mbd%22%2C+%22wss%22%2C+%22features%22](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel_sample_mbd%22%2C+%22wss%22%2C+%22features%22)

```

Assert all points assigned
assert result.count() == 4
assert "prediction" in result.columns
assert "ibd_cluster_center" in result.columns

Check cluster centers exist
centers = result.select("prediction", "ibd_cluster_center").distinct()
assert centers.count() == 2

def test_calc_clusters_computes_correct_centers(spark):
 # Create data with known clusters
 data = [
 ("1_MBD001", 100.0, 100.062),
 ("1_MBD001", 200.0, 200.063), # Cluster 0: avg=150
 ("1_MBD001", 900.0, 900.064),
 ("1_MBD001", 1000.0, 1000.065) # Cluster 1: avg=950
]
 df = spark.createDataFrame(
 data,
 "channel_sample_mbd", "wss", "features"66
)

 k_values =
 Unknown macro: {"1_MBD001"}
 kmeans = kmeanscluster(k_values=k_values)
 result = kmeans.calc_clusters(df, k_values)

 # Check centers are approximately correct
 centers = result.select(
 "prediction",
 "ibd_cluster_center"
).distinct().orderBy("ibd_cluster_center").collect()

 assert abs(centers67.ibd_cluster_center - 150) < 50

```

62. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100.0)

63. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=200.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=200.0)

64. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=900.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=900.0)

65. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=1000.0](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=1000.0)

66. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel\\_sample\\_mbd%22%2C+%22wss%22%2C+%22features%22](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22channel_sample_mbd%22%2C+%22wss%22%2C+%22features%22)

```
assert abs(centers68.ibd_cluster_center - 950) < 50
...
```

### \*5. End-to-End Tests\*

```
```python
def test_transform_complete_pipeline(spark):
    # Create realistic test data
    data = [
        ("Retail", "1", "MBD001", "GroceryA", "CELL001", 123.45, 50, 150000.0),
        ("Retail", "1", "MBD001", "GroceryA", "CELL002", 234.56, 75, 200000.0),
        ("Retail", "1", "MBD001", "GroceryA", "CELL003", 345.67, 100, 300000.0),
        ("Retail", "1", "MBD002", "BeverageB", "CELL004", 456.78, 60, 180000.0),
        ("Retail", "1", "MBD002", "BeverageB", "CELL005", 567.89, 80, 220000.0)
    ]
    df = spark.createDataFrame(
        data,
        ["Channel", "SampleID", "MBD_ID", "MBD", "CellID",
         "wss", "outlets", "avg_yearly_sales"]
    )

    kmeans = KMeansCluster(max_cluster=3, min_cluster_size=1)
    result = kmeans.transform(df)

    # Assert output schema
    assert "IBDID" in result.columns
    assert "IBD" in result.columns
    assert "ibd_cluster_center" in result.columns

    # Assert all cells assigned
    assert result.count() == 5
    assert result.filter(col("IBDID").isNull()).count() == 0

    # Assert IBD names formatted correctly
    ibd_names = result.select("IBD").distinct().collect()
    for row in ibd_names:
        assert "_IBD" in row.IBD

def test_transform_with_predefined_k(spark):
    # Create test data with 2 MBDs
    data = create_test_data_two_mbds()
    df = spark.createDataFrame(data, schema)

    # Provide predefined K
    k_values =
```

67. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=0

68. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=1

Unknown macro: { "1_MBD001"}

```
kmeans = KMeansCluster(k_values=k_values)
result = kmeans.transform(df)

# Verify K values used
mbd1_clusters = result.filter(
    col("MBD_ID") == "MBD001"
).select("IBDID").distinct().count()

mbd2_clusters = result.filter(
    col("MBD_ID") == "MBD002"
).select("IBDID").distinct().count()

assert mbd1_clusters == 2
assert mbd2_clusters == 3
...
```

1. a. i. 11.2 Integration Testing

1. a. i. 1. ***Test Scenarios***

1. Multi-Sample Processing

```
```python
def test_multiple_samples(spark):
 """Test clustering across multiple samples"""
 data = []
 for sample_id in 1, 2, 369:
 for i in range(20):
 data.append((
 "Retail", str(sample_id), "MBD001", "GroceryA",
 f"CELL
```

Unknown macro: {sample\_id}

–

Unknown macro: {i}

```
, 100.0 + 1 * 10, 50, 150000.0
))
```

```
df = spark.createDataFrame(data, schema)
```

```
kmeans = KMeansCluster()
result = kmeans.transform(df)
```

```
Each sample should have independent clusters
samples = result.select("SampleID").distinct().count()
assert samples == 3
```

69. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=1%2C+2%2C+3](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=1%2C+2%2C+3)

```

Verify sample-specific channel_sample_mbd
sample_mbds = result.select("channel_sample_mbd").distinct().collect()
assert len(sample_mbds) == 3
assert any("1_MBD001" in row.channel_sample_mbd for row in sample_mbds)
assert any("2_MBD001" in row.channel_sample_mbd for row in sample_mbds)
assert any("3_MBD001" in row.channel_sample_mbd for row in sample_mbds)
...

```

## **\*2. Large Dataset Processing\***

```

```python
def test_large_dataset_1000_cells(spark):
    """Test scalability with 1000 cells"""
    data = []
    for i in range(1000):
        data.append((
            "Retail", "1", "MBD001", "GroceryA",
            f"CELL

", 100.0 + i, 50, 150000.0
        ))

    df = spark.createDataFrame(data, schema)

    start_time = time.time()
    kmeans = KMeansCluster(max_cluster=5)
    result = kmeans.transform(df)
    duration = time.time() - start_time

    # Performance assertion
    assert duration < 120 # Should complete in < 2 minutes
    assert result.count() == 1000
...

```

3. Data Quality Validation

```

```python
def test_output_data_quality(spark):
 """Validate output data integrity"""
 df = create_test_data()

 kmeans = KMeansCluster()
 result = kmeans.transform(df)

 # No nulls in key columns
 assert result.filter(col("IBDID").isNull()).count() == 0
 assert result.filter(col("IBD").isNull()).count() == 0
 assert result.filter(col("ibd_cluster_center").isNull()).count() == 0

 # IBDID is non-negative
 assert result.filter(col("IBDID") < 0).count() == 0

```

```
ibd_cluster_center is positive
assert result.filter(col("ibd_cluster_center") < 0).count() == 0

IBD names contain MBD name
invalid_ibds = result.filter(
 ~col("IBD").contains(col("MBD"))
)
assert invalid_ibds.count() == 0
...

```

1. a. i. 11.3 Performance Testing

1. a. i. 1. **\*Benchmark Suite\***

```
```python
class TestPerformance:
    def test_benchmark_silhouette_vs_predefined(self, spark):
        """Compare performance with/without Silhouette"""
        df = create_test_data(num_cells=5000)

        # Test 1: With Silhouette
        start = time.time()
        kmeans1 = KMeansCluster(max_cluster=8)
        result1 = kmeans1.transform(df)
        result1.count() # Force execution
        silhouette_time = time.time() - start

        # Test 2: Without Silhouette (predefined K)
        k_values =
        Unknown macro: {"1_MBD001"}
        start = time.time()
        kmeans2 = KMeansCluster(k_values=k_values)
        result2 = kmeans2.transform(df)
        result2.count() # Force execution
        predefined_time = time.time() - start

        # Assert predefined is faster
        assert predefined_time < silhouette_time * 0.4 # 60% faster

        print(f"Silhouette:
        Unknown macro: {silhouette_time}
s )
        print(f"Predefined:
        Unknown macro: {predefined_time}
s )
        print(f"Speedup:
        Unknown macro: {silhouette_time / predefined_time}

```

x")

```
def test_scalability_linear_growth(self, spark):
    """Test if processing time grows linearly with data"""
    sizes = 100, 500, 1000, 200070
    times = []
```

k_values =

Unknown macro: {"1_MBD001"}

for size in sizes:

df = create_test_data(num_cells=size)

start = time.time()

kmeans = KMeansCluster(k_values=k_values)

result = kmeans.transform(df)

result.count()

duration = time.time() - start

times.append(duration)

print(f"Size

Unknown macro: {size}

:

Unknown macro: {duration}

s)

Check linear growth (2x data ~= 2x time)

ratio = times³⁷¹ / times¹⁷² # 2000 vs 500

assert 3.5 < ratio < 4.5 # Should be ~4x

...

1. a. i. 11.4 Test Data Generation

1. a. i. 1. ***Helper Functions***

```python

def create\_test\_data(

num\_cells=100,

num\_mbds=2,

num\_samples=1,

70. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100%2C+500%2C+1000%2C+2000](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=100%2C+500%2C+1000%2C+2000)

71. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=3](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=3)

72. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=1](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=1)

```

cluster_separation=200.0
):
 """Generate synthetic test data with known cluster structure"""
 data = []

 for sample_id in range(1, num_samples + 1):
 for mbd_idx in range(num_mbds):
 mbd_id = f'MBD

Unknown macro: {str(mbd_idx + 1).zfill(3)}

 mbd_name = f'Category

Unknown macro: {chr(65 + mbd_idx)}

 # Create 3 clusters per MBD
 cells_per_cluster = num_cells // 3

 for cluster in range(3):
 base_wss = 100.0 + (cluster * cluster_separation)

 for i in range(cells_per_cluster):
 data.append((
 "Retail",
 str(sample_id),
 mbd_id,
 mbd_name,
 f"CELL

-
Unknown macro: {mbd_id}

-
Unknown macro: {cluster}

-
Unknown macro: {i}

,
 base_wss + (i * 2), # Small variation within cluster
 50 + (i % 20),
 150000.0 + (i * 1000)
))

 return data
...
-

1. a. 12. Deployment
1. a. i. 12.1 Deployment Architecture

```

1. a. i. 1. **\*Spark Cluster Setup\***

**\*Local Development:\***

```
```python
spark = SparkSession.builder \
    .master("local473") \
    .appName("KMeansCluster_Dev") \
    .config("spark.driver.memory", "4g") \
    .config("spark.sql.shuffle.partitions", "20") \
    .getOrCreate()
```
```

**\*Production Cluster (YARN/K8s):\***

```
```python
spark = SparkSession.builder \
    .appName("KMeansCluster_Production") \
    .config("spark.driver.memory", "8g") \
    .config("spark.executor.memory", "16g") \
    .config("spark.executor.cores", "4") \
    .config("spark.sql.shuffle.partitions", "200") \
    .config("spark.sql.adaptive.enabled", "true") \
    .enableHiveSupport() \
    .getOrCreate()
```
```

1. a. i. 1. **\*Deployment Package\***

```

kmeans-ibd-clustering/
├── src/
│ ├── _init_.py
│ ├── kmeans_cluster.py # Main class
│ ├── config.py # Configuration
│ └── utils.py # Helper functions
├── tests/
│ ├── unit/
│ │ ├── test_vectorize.py
│ │ ├── test_k_selection.py
│ │ └── test_clustering.py
│ ├── integration/
│ │ └── test_end_to_end.py
│ └── performance/
│ └── test_benchmarks.py
├── notebooks/
│ ├── exploration.ipynb
│ └── validation.ipynb
└── config/

```

73. [https://adlm.nielseniq.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=4](https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=4)

```

| |— dev.yaml
| |— prod.yaml
|— requirements.txt
|— setup.py
|— README.md
...

```

1. a. i. 12.2 Job Submission

1. a. i. 1. **\*Spark Submit Script\***

```
```bash
```

```
#!/bin/bash
```

1. submit_kmeans_job.sh

```

spark-submit \
--master yarn \
--deploy-mode cluster \
--driver-memory 8g \
--executor-memory 16g \
--executor-cores 4 \
--num-executors 10 \
--conf spark.sql.shuffle.partitions=200 \
--conf spark.sql.adaptive.enabled=true \
--conf spark.dynamicAllocation.enabled=true \
--conf spark.dynamicAllocation.minExecutors=5 \
--conf spark.dynamicAllocation.maxExecutors=20 \
--py-files dist/kmeans_ibd_clustering-1.0-py3.egg \
src/kmeans_cluster.py \
--input-path "$

```

Unknown macro: {INPUT_PATH}

```
\
```

```
--output-path "$
```

Unknown macro: {OUTPUT_PATH}

```
\
```

```
--max-cluster 8 \
```

```
--min-cluster-size 2 \
```

```
--use-silhouette true
```

```
```
```

1. a. i. 1. **\*Airflow DAG Example\***

```
```python
```

```
from airflow import DAG
```

```
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
```

```
from datetime import datetime, timedelta
```

```
default_args = {
```

```
    'owner': 'data-team',
```

```
    'depends_on_past': False,
```

```

'start_date': datetime(2024, 1, 1),
'email_on_failure': True,
'email_on_retry': False,
'retries': 2,
'retry_delay': timedelta(minutes=5)
}

dag = DAG(
    'kmeans_ibd_clustering',
    default_args=default_args,
    schedule_interval='@weekly',
    catchup=False
)

kmeans_task = SparkSubmitOperator(
    task_id='run_kmeans_clustering',
    application='src/kmeans_cluster.py',
    name='KMeans_IBD_Clustering',
    conf=
    Unknown macro: {      'spark.driver.memory'}
    ,
    application_args=[
        '-input-path', '{{ var.value.input_path }}',
        '-output-path', '{{ var.value.output_path }}',
        '-max-cluster', '8'
    ],
    dag=dag
)
...

```

1. a. i. 12.3 Monitoring & Alerting

1. a. i. 1. ***Spark UI Monitoring***

Key Metrics to Track:

- ***Stage Duration***: Identify slow stages (vectorization, Silhouette, clustering)
- ***Shuffle Read/Write***: Monitor data movement between executors
- ***Memory Usage***: Track driver/executor memory consumption
- ***Task Failures***: Identify and debug failed tasks

Access Spark UI:

```
```bash
```

1. Local mode <http://localhost:4040> 

1. YARN cluster

```
http://<yarn-rm-host>:8088/proxy/<application_id>/
```

```
```
```

1. a. i. 1. ***CloudWatch/Prometheus Metrics***

```
```python
```

1. Example metric publishing

```
import time
from prometheus_client import Counter, Gauge, Histogram
```

1. Define metrics

```
clustering_duration = Histogram(
 'kmeans_clustering_duration_seconds',
 'Time spent on K-Means clustering'
)
```

```
cells_processed = Counter(
 'kmeans_cells_processed_total',
 'Total number of cells processed'
)
```

```
optimal_k_gauge = Gauge(
 'kmeans_optimal_k',
 'Optimal K value selected',
 'mbd_id'74
)
```

1. In transform() method

```
with clustering_duration.time():
 result = self.transform(mbd_variance)
```

```
cells_processed.inc(result.count())
```

```
for mbd, k in k_values.items():
 optimal_k_gauge.labels(mbd_id=mbd).set(k)
```
```

1. a. i. 12.4 Configuration Management

1. a. i. 1. ***config.yaml Example***

```
```yaml
```

```
1. dev.yaml
clustering:
 max_cluster: 5
 min_cluster_size: 2
 use_silhouette: true
 kmeans_max_iter: 20
 kmeans_seed: 42
```

```
spark:
```

```
 driver_memory: "4g"
```

---

74. [https://adlm.nielseni.com/confluence/pages/createpage.action?atl\\_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%27mbd\\_id%27](https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%27mbd_id%27)

```

executor_memory: "8g"
executor_cores: 2
shuffle_partitions: 100

```

```

paths:
 input: "s3://dev-bucket/mbd_variance/"
 output: "s3://dev-bucket/ibd_results/"

```

```

logging:
 level: "INFO"
 format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
...

```

```
```python
```

```

1. Load configuration
   import yaml

```

```

def load_config(env='dev'):
    with open(f'config/

```

Unknown macro: {env}

```

.yaml, r) as f:
    config = yaml.safe_load(f)
    return config

```

```
config = load_config('prod')
```

```

kmeans = KMeansCluster(
    max_cluster=config'clustering'75'max_cluster'76,
    min_cluster_size=config'clustering'77'min_cluster_size'78
)
...

```

1. a. i. 12.5 Rollout Strategy

1. a. i. 1. ***Phase 1: Development & Testing (2 weeks)***

- Develop and unit test KMeansCluster class
- Create integration tests
- Performance benchmarking on dev cluster

75. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%27clustering%27

76. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%27max_cluster%27

77. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%27clustering%27

78. https://adlm.nielseniq.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%27min_cluster_size%27

1. a. i. 1. ***Phase 2: User Acceptance Testing (1 week)***

- Deploy to UAT environment
- Run on production-like data
- Validate results with business users
- Performance testing at scale

1. a. i. 1. ***Phase 3: Production Deployment (1 week)***

- Deploy to production cluster
- Schedule initial runs during off-peak hours
- Monitor closely for issues
- Validate output quality

1. a. i. 1. ***Phase 4: Optimization & Tuning (Ongoing)***

- Analyze performance metrics
- Tune Spark configurations
- Optimize algorithm parameters
- Gather user feedback

—

1. a. 13. Appendices

1. a. i. 13.1 Glossary

Term	*Definition*
-----	-----
IBD	Initial Business Definition - A cluster of retail cells with similar sales characteristics
MBD	Market Business Definition - A business grouping of products/categories
WSS	Within-Sum-of-Squares - Measure of variance within clusters (lower is better)
Silhouette Score	Clustering quality metric ranging from -1 to 1 (higher indicates better separation)
channel_sample_mbd	Composite key: `" Unknown macro: {SampleID} — Unknown macro: {MBD_ID} "` used for grouping

Cluster Center	Average WSS value for all points in a cluster (ibd_cluster_center)
K	Number of clusters to create
Optimal K	Best number of clusters determined by Silhouette score
VectorAssembler	PySpark ML component that transforms columns into feature vectors
ClusteringEvaluator	PySpark ML component that calculates Silhouette scores

1. a. i. 13.2 References

1. a. i. 1. ***PySpark MLlib Documentation***

- K-Means: <https://spark.apache.org/docs/latest/ml-clustering.html#k-means>
- VectorAssembler: <https://spark.apache.org/docs/latest/ml-features.html#vectorassembler>
- ClusteringEvaluator: <https://spark.apache.org/docs/latest/ml-tuning.html#clustering-evaluator>

1. a. i. 1. ***Algorithm References***

- K-Means Algorithm: MacQueen, J. (1967). "Some methods for classification and analysis of multivariate observations"
- Silhouette Coefficient: Rousseeuw, P. J. (1987). "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis"

1. a. i. 1. ***PySpark Best Practices***

- Performance Tuning: <https://spark.apache.org/docs/latest/sql-performance-tuning.html>
- Structured Streaming: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

1. a. i. 13.3 Sample Code Snippets

1. a. i. 1. ***Complete Usage Example***

```
```python
from pyspark.sql import SparkSession
from kmeans_cluster import KMeansCluster

1. Initialize Spark session
spark = SparkSession.builder \
 .appName("IBD_Clustering_Example") \
 .config("spark.sql.shuffle.partitions", "200") \
 .getOrCreate()

1. Load data
mbd_variance = spark.read.parquet("path/to/mbd_variance.parquet")
```

## 1. Option 1: Automated K selection (Silhouette)

```
kmeans_auto = KMeansCluster(
 max_cluster=8,
 min_cluster_size=2
)
result_auto = kmeans_auto.transform(mbd_variance)
```

## 1. Option 2: Predefined K values

```
k_values = {
 "1_MBD001": 3,
 "1_MBD002": 5,
 "2_MBD001": 4
}
kmeans_predefined = KMeansCluster(
 k_values=k_values,
 min_cluster_size=2
)
result_predefined = kmeans_predefined.transform(mbd_variance)
```

## 1. Save results

```
result_auto.write.mode("overwrite").parquet("path/to/output.parquet")
```

## 1. Display sample results

```
result_auto.select(
 "CellID", "MBD", "IBDID", "IBD", "ibd_cluster_center"
).show(20, truncate=False)
...

```

1. a. i. 1. **\*Custom K Selection Logic\***

```
```python
def custom_k_selection(mbd_data, max_k=8):
    """
    Custom logic to determine K based on business rules
    """
    n = mbd_data.count()

    # Rule 1: Small datasets get fewer clusters
    if n < 50:
        return 2
    elif n < 200:
        return 3

    # Rule 2: Large datasets can have more clusters
    elif n >= 1000:
        return min(8, n // 100)

    # Rule 3: Default to Silhouette for medium datasets
    else:

```

```

kmeans = KMeansCluster()
return kmeans.find_k_values(mbd_data, max_k, 2)

1. Use custom logic
vector_data = kmeans.vectorize(mbd_variance)
mbds = vector_data.select("channel_sample_mbd").distinct().collect()

k_values = {}
for mbd_row in mbds:
    mbd = mbd_row.channel_sample_mbd
    mbd_data = vector_data.filter(col("channel_sample_mbd") == mbd)
    k_values[mbd]79 = custom_k_selection(mbd_data)

kmeans = KMeansCluster(k_values=k_values)
result = kmeans.transform(mbd_variance)
...

```

1. a. i. 13.4 FAQ

Q: How do I choose between automated and predefined K values?

A: Use automated (Silhouette) when:

- Exploring new datasets
- Cluster structure is unknown
- Quality is more important than speed

Use predefined K when:

- Business rules dictate cluster count
- Performance is critical
- Cluster structure is well-understood

Q: What if Silhouette calculation is too slow?

A: Options:

1. Reduce max_cluster (test fewer K values)
2. Sample data for Silhouette (10-20% sample)
3. Use predefined K values
4. Increase Spark cluster resources

Q: How do I handle MBDs with very few cells?

A: The algorithm automatically:

- Returns K=2 if cells < (min_cluster_size × 2)
- Adjusts max_k = cells // min_cluster_size
- Ensures valid cluster sizes

79. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=mbd

Q: Can I use multiple features instead of just WSS?

A: Yes, modify the VectorAssembler:

```
```python
assembler = VectorAssembler(
 inputCols=["wss", "avg_yearly_sales", "outlets"80,
 outputCol="features"
)
```
```

Q: How do I validate clustering quality?

A: Check:

1. Silhouette scores (> 0.4 is good)
2. Cluster sizes (balanced is better)
3. Business interpretation (do clusters make sense?)
4. Visualization (plot cluster centers)

—

1. a. Document Revision History

| *Version* | *Date* | *Author* | *Changes* |
|-----------|------------|--------------------------|----------------------------------|
| ----- | ----- | ----- | ----- |
| 1.0 | 2024-01-XX | System Architecture Team | Initial detailed design document |

—

END OF DOCUMENT

80. https://adlm.nielseni.com/confluence/pages/createpage.action?atl_token=9444644188788404882429fda0db7667e4bf49cb&fromPageId=900958530&linkCreation=true&spaceKey=DNV&title=%22wss%22%2C+%22avg_yearly_sales%22%2C+%22outlets%22

19 IBD KMeans Details

19.1 1. Implementation Steps

1. UI Submits the request to throttler service and inserts into throttler table

URL : <https://uat3.operations.nielseniq.io/cdarapigateway/d2ovueinputdatamanagementservice/saveimpactparams>

Request : {"ScheduledJob":
{"TransactionID":166837546,"JobType":10,"JobTypeName":"KMEANS_REPORT","CountryID":"2006",
"ProjectID":953239,"Description":"cld_io_14370_NoCell","countryCode":"BR","CountrySchema":"d2o_br",
"InsertedbyUserID":"456","InsertedbyUsername":"Abhai S Yalwarkar","DataSetID":41,"ScheduledJobParameters":
[{"ParameterName":"InputFileSetID","ParameterValue":41},
{"ParameterName":"countryId","ParameterValue":"2006"}]}

Response : {"data":{"TransactionId":"166837546","JobId":"0"},"serviceStatus":
{"StatusCode":"200","StatusFlag":"true","StatusMsg":"Success","TransactionID":"166837546"}}

<https://uat3.operations.nielseniq.io/cdarapigateway/ttrschedulerservice/api/v2/schedulers>

URL : <https://uat3.operations.nielseniq.io/cdarapigateway/d2ovuejobs/VUEScreePlotSaveMBDs>

Request : {"ProjectId":953239,"InputFileSetId":41,"CountrySchema":"d2o_br","MBDRecords":
[{"MBDId":"1_20183977","GroupNumber":3,"SampleId":"20003424","IndexId":"MONT"}],"TransactionId":6168275464352}

Response :
{"StatusCode":"200","StatusFlag":"true","StatusMsg":"SUCCESS","TransactionId":"6168275464352"}

<https://uat3.operations.nielseniq.io/cdarapigateway/ttrschedulerservice/api/v2/schedulers>

Request :
{"projectId":953239,"versionId":"166837546","application":"VUE","jobType":"KMEANS_REPORT",
"countrySchema":"d2o_br","region":"LATAM","country":"BRAZIL","status":"QUEUED","job_id":
"0","dataset_id":41,"transactionId":"166837546","createdBy":"Abhai S Yalwarkar",
"command":{"python_params":["--project_id","953239","--country_code","BR",
"--transaction_id","166837546"],"job_id":["0"],"data":[{"key":"transactionId","value":"166837546"},
{"key":"input_dataset_id","value":41}],"datasetName":"cld_io_14370_NoCell","datasetId":"41"}]}

Response : {"serviceStatus":
{"statusCode":"200","statusFlag":true,"statusMsg":"Success","transactionId":"166837546"},
"data":{"schedulerId":151637}}

2. Pyspark processes the request and use the prototype to calculate groups (KMeans) for MBds.

3. Once the values are calculated, PYSpark need to insert the data into the required tables using below queries.

```
-- Log-1-1-1 :
insert into _temp_Log (sampleId, Message) select distinct sampleId, 'Recalculated
cluster centers until all MBds have converged or until the threshold of 100 iterations
is reached'
from _temp_MBds;

_stage := 14;
-- Create and Insert Temp IBD Data:
DROP TABLE IF EXISTS _temp_IBDs;
CREATE TEMP TABLE _temp_IBDs (IBDID SERIAL, IBDName text, MBDID text, CenterID
int, OutletsCount int, sampleId Int);

CREATE UNIQUE INDEX temp_index_IBDs ON _temp_IBDs (MBDID,CenterID);

INSERT INTO _temp_IBDs (MBDID, CenterID, OutletsCount, sampleId)
select mc.MBDID, mc.CenterID, sum(mcvw.OutletsCount), mc.sampleId
from _temp_MBDCenters mc
join _temp_CombinationValueCenter cvc
on mc.MBDID = cvc.MBDID
and mc.CenterID = cvc.CenterID
join _temp_MBDCombinationValues mcvw
on cvc.MBDID = mcvw.MBDID
and cvc.CombinationValueID = mcvw.CombinationValueID
group by mc.MBDID, mc.CenterID, mc.sampleId;

RAISE NOTICE '_stage %: record inserted in table _temp_IBDs', _stage;

-- Update IBD Name:
_stage := 15;
update _temp_IBDs as i set IBDName =
concat(left(ltrim(rtrim(m.MBDName)),100-
length(concat('()_IBD',ltrim(rtrim(m.MBDID)),ltrim(rtrim(i.CenterID::text))))),
'(', ltrim(rtrim(m.MBDID)), ')_IBD', ltrim(rtrim(i.CenterID::text)))
from _temp_IBDs i1 join _temp_MBds m
on i1.MBDID = m.MBDID
where i1.MBDID = i.MBDID;
RAISE NOTICE '_stage %: records updated in _temp_IBDs', _stage;

-- Transaction
-- Delete any existing IBds:
_stage := 16;
v_sqlStatement := 'delete from '||v_countrySchema ||'.VUEIBDsMulti_'||
v_projectId|| '_'|| v_inputFileSetId || ' where ProjectID = '||v_projectId||' and
InputFileSetID ='||v_inputFileSetId ||
' and sampleId IN (select from
_temp_vuemultichaneljobexceution)';
```

```

EXECUTE v_sqlStatement;
RAISE NOTICE '_stage %: records record deleted from VUEIBDsMulti_', _stage;

-- Delete any existing IBDCells:
_stage := 17;
v_sqlStatement := 'delete from ' || v_countrySchema || '.VUEIBDCellsMulti_' ||
v_projectId || '_' || v_inputFileSetId || ' where ProjectID = ' || v_projectId || ' and
InputFileSetID = ' || v_inputFileSetId ||
' and sampleid IN( select sampleId from
_temp_vuemultichaneljobexceution)';
EXECUTE v_sqlStatement;
RAISE NOTICE '_stage %: records record deleted from VUEIBDsMulti_', _stage;

-- Log-1-1-1-1:
INSERT into _temp_Log (sampleId, Message) select distinct sampleId, 'Cleaned VUE
IBDs and IBDCells tables' from _temp_vuemultichaneljobexceution;

_stage := 18;
v_sqlStatement := 'INSERT INTO ' || v_countrySchema || '.VUEIBDsMulti_' ||
v_projectId || '_' || v_inputFileSetId || ' (ProjectID, InputFileSetID, IBDID, Name,
sampleid, indexid)
SELECT ' || v_projectId || ', ' || v_inputFileSetId || ', IBDID,
IBDName, v.sampleId, v.indexid
FROM _temp_IBDs i
JOIN _temp_vuemultichaneljobexceution v ON i.sampleId =
v.sampleId';
EXECUTE v_sqlStatement;

RAISE NOTICE '_stage %: records record deleted from VUEIBDsMulti_', _stage;
-- Log-1-1-1-1-3:
v_sqlStatement := 'insert into _temp_Log (sampleId, Message) select distinct
sampleId, ''Inserted new IBDs to VUEIBDs table''
from ' || v_countrySchema || '.VUEIBDsMulti_' || v_projectId ||
'_' || v_inputFileSetId || ' where ProjectID = ' || v_projectId ||
' and InputFileSetID = ' || v_inputFileSetId || ' and sampleid
IN( select sampleId from _temp_vuemultichaneljobexceution)';
EXECUTE v_sqlStatement;

_stage := 19;
-- Compute MBD to Cell allocation:
v_sqlStatement := 'with SelectedMBDCells (MBDID, CellID, sampleId) as
( select mc.MBDID, mc.CellID, tv.c.sampleId
from ' || v_countrySchema || '.VUEMBDCellsMulti_' ||
v_projectId || '_' || v_inputFileSetId || ' mc
join _temp_MBDs m
on m.MBDID = mc.MBDID
join _temp_vuemultichaneljobexceution tv.c
on tv.c.sampleId = mc.sampleid
and tv.c.ProjectID = mc.ProjectID
where mc.ProjectID = ' || v_projectId ||
' and mc.InputFileSetID = ' || v_inputFileSetId ||
')
select distinct COALESCE(smc.MBDID, '-2147483648') as MBDID,
aicsv.CellID, tv.c.sampleId
into _temp_MBDCells

```

```

        from '||v_countrySchema
||'.VUEAutoIBDsCellSalesVariationMulti_'||v_projectId|| '_'|| v_inputFileSetId || '
aicsv

        left join SelectedMBDCells smc
        on aicsv.CellID = smc.CellID
        join _temp_vuemultichaneljobexceution tvc
        on tvc.sampleId = aicsv.sampleid
        and tvc.ProjectID = aicsv.ProjectID
        where aicsv.ProjectID = '||v_projectId||' and
aicsv.InputFileSetID = '||v_inputFileSetId; -- ||' and aicsv.sampleid = '||v_sampleId;
EXECUTE v_sqlStatement;
RAISE NOTICE '_stage %', _stage;

_stage := 20;
-- Assign Cells to IBDs:
v_sqlStatement := 'insert into '||v_countrySchema ||'.VUEIBDCellsMulti_'||
v_projectId|| '_'|| v_inputFileSetId || ' (ProjectID, InputFileSetID, IBDID,
CellID,sampleid,indexid)
        select '||v_projectId||', '||v_inputFileSetId ||', i.IBDID,
aicsv.CellID, tvc.sampleId as sampleid, tvc.indexId as indexid
        from _temp_IBDs i
        join _temp_CombinationValueCenter cvc
        on i.MBDID = cvc.MBDID
        and i.CenterID = cvc.CenterID
        join _temp_MBDCells mc
        on cvc.MBDID = mc.MBDID
        join '||v_countrySchema
||'.VUEAutoIBDsCellSalesVariationMulti_'||v_projectId|| '_'|| v_inputFileSetId || '
aicsv

        on aicsv.CellID = mc.CellID
        and aicsv.CombinationValueID = cvc.CombinationValueID
        join _temp_vuemultichaneljobexceution tvc
        on tvc.sampleId = aicsv.sampleid
        and tvc.ProjectID = aicsv.ProjectID
        where aicsv.ProjectID = '||v_projectId||' and
aicsv.InputFileSetID = '||v_inputFileSetId;
EXECUTE v_sqlStatement;
RAISE NOTICE '_stage %: records inserted into VUEIBDCellsMulti_', _stage;

--Log-1-1-1-1-1:
v_sqlStatement := 'insert into _temp_Log (sampleId, Message) select distinct
sampleId, ''Calculated cells to IBDs allocation, updated VUEIBDCells table''
        from ' || v_countrySchema || '.VUEIBDCellsMulti_' ||
v_projectId || '_'|| v_inputFileSetId|| ' where ProjectID = '||v_projectId||
        ' and InputFileSetID = '||v_inputFileSetId ||' and sampleid
IN( select sampleId from _temp_vuemultichaneljobexceution)';
EXECUTE v_sqlStatement;

-- Transaction End

_stage := 21;
v_sqlStatement := 'INSERT INTO ' || v_countrySchema ||
'.vuekmeansibdcharacteristicsreportmulti_' || v_projectId || '_'|| v_inputFileSetId ||
',
        SELECT ' || v_jobId || ', m.MBDID, m.MBDName, i.IBDName,
i.OutletsCount, mc.AverageSales, m.CombinationName,
```

```

        mcvw.CombinedValues, mcvw.OutletsCount, mcvw.AverageSales,
mcvw.WSS,
        m.Iterations, m.DesignIBDsGroupsNumber, ' || v_projectId || ' AS
ProjectID, ' || v_inputFileSetId || ' AS inputfilesetid,
        v.indexid AS indexid, v.sampleId AS sampleid
FROM _temp_MBDs m
JOIN _temp_IBDs i
ON m.MBDID = i.MBDID
JOIN _temp_MBDCenters mc
ON i.MBDID = mc.MBDID
AND i.CenterID = mc.CenterID
JOIN _temp_CombinationValueCenter cvc
ON i.MBDID = cvc.MBDID
AND i.CenterID = cvc.CenterID
JOIN _temp_MBDCombinationValues mcvw
ON m.MBDID = mcvw.MBDID
AND cvc.CombinationValueID = mcvw.CombinationValueID
JOIN _temp_vuemultichanneljobexceution v
on v.sampleId = m.sampleid
and v.ProjectID = m.ProjectID';
EXECUTE v_sqlStatement;

RAISE NOTICE '_stage %: records inserted into
vuekmeansibdcharacteristicsreportmulti_', _stage;

-- Log-1-1-1-1-1-1:
--insert into _temp_Log (Message) values ('Prepared IBDs characteristics value
combination report');
v_sqlStatement := 'insert into _temp_Log (sampleId, Message) select distinct
sampleId, ''Prepared IBDs characteristics value combination report''
from ' || v_countrySchema ||
'.vuekmeansibdcharacteristicsreportmulti_' || v_projectId || '-' || v_inputFileSetId || '
where ProjectID = ' || v_projectId ||
' and InputFileSetID = ' || v_inputFileSetId || ' and sampleid
IN( select sampleId from _temp_vuemultichanneljobexceution)';
EXECUTE v_sqlStatement;

_stage := 23;
--IBD Cells report:
v_sqlStatement := 'insert into ' || v_countrySchema
|| '.vuekmeansibdcellsreportMulti_' || v_projectId || '-' || v_inputFileSetId || '
select ' || v_jobId || ', m.MBDID, m.MBDName, i.IBDName, ic.CellID,
' || v_projectId || ' as ProjectID, ' || v_inputFileSetId || ' as inputfilesetid, ic.indexid,
ic.sampleid
from _temp_MBDs m
join _temp_IBDs i
on m.MBDID = i.MBDID
join ' || v_countrySchema || '.VUEIBDCellsMulti_' || v_projectId ||
'-' || v_inputFileSetId || ' ic
on i.IBDID::text = ic.IBDID
join _temp_vuemultichanneljobexceution tv
on tv.sampleId = ic.sampleid
and tv.ProjectID = ic.ProjectID
where ic.ProjectID = ' || v_projectId || ' and ic.InputFileSetID =
' || v_inputFileSetId; -- || ' and ic.sampleid= ' || v_sampleId;
EXECUTE v_sqlStatement;

```

```

        RAISE NOTICE '_stage %: records inserted into vuekmeansibdcellsreportMulti_',
        _stage;

--Log-1-1-1-1-1-1-1:
--insert into _temp_Log (Message) values ('Prepared IBDs cells report');
v_sqlStatement := 'insert into _temp_Log (sampleId, Message) select distinct
sampleId, ''Prepared IBDs cells report''
                    from ' || v_countrySchema || '.vuekmeansibdcellsreportMulti_'
|| v_projectId || ' ' || v_inputFileSetId || ' where ProjectID = ' || v_projectId ||
' and InputFileSetID = ' || v_inputFileSetId || ' and sampleId
IN( select sampleId from _temp_vuemultichanneljobexceution)';
EXECUTE v_sqlStatement;

-- Process Log:
_stage := 24;
v_sqlStatement := 'insert into ' || v_countrySchema || '.ProcessLog (ScheduledJobID,
CountryID, ProjectID, JobID, Completed, StartedDate, CompletedDate, Message, RowCount)
select 1, ' || v_countryId || ', ' || v_projectId || ', ' || v_jobId || ',
B''0'', current_timestamp as StartedDate, current_timestamp as CompletedDate, Message,
AffectedRows from _temp_Log';
EXECUTE v_sqlStatement;
RAISE NOTICE '_stage %: records inserted into ProcessLog', _stage;

_stage := 25;
--UpdateDataSetAsReady-1:
v_sqlStatement := 'update ' || v_countrySchema || '.Vuemultichanneljobexceution_' ||
v_jobId || ' v set v.status=3, v.ErrorMessage=NULL
                    from ' || v_countrySchema || '.vuekmeansibdcellsreportMulti_' ||
v_projectId || ' ' || v_inputFileSetId || ' vm
                    join _temp_vuemultichanneljobexceution t on vm.sampleid =
t.sampleId
                    where v.sampleid = vm.sampleId';
EXECUTE v_sqlStatement;
RAISE NOTICE '_stage %: records updated Vuemultichanneljobexceution_', _stage;

```

19.2 2. Traceability Matrix / Functionality–Component Mapping

The Traceability Matrix provides a clear mapping between the functional requirements and the corresponding design components responsible for implementing them.

This ensures that every requirement is accounted for within the system architecture and can be validated during development and testing.

| S R # | Functionality Name | Short Description | Design Elements - UI, SB, DB | Impacted Tables and API Used | Input | Output | Test Cases |
|-------|---------------------------|--|---|---|---|--|--|
| 1 | Initialization | Initialize KMeansCluster with optional k_values, max_cluster (default 8), and min_cluster_size (default 2). Validates active Spark session. | SB: SparkSession validation | API: SparkSession.getActiveSession() | k_values: dict\ None (see page 98), max_cluster: int (see page 98), min_cluster_size: int (see page 98) | KMeansCluster instance with configured parameters | TC-01: Valid Spark session → instance created.
TC-02: No Spark session → ValueError raised.
TC-03: Default params → max_cluster=8, min_cluster_size=2. |
| 2 | Vectorize WSS Data | Scales WSS by 1e12, fills nulls with 0, converts WSS to ML feature vector using VectorAssembler, creates composite key channel_sample_mbd (Channel_SampleID_MBD_ID). | SB: PySpark ML VectorAssembler, DataFrame transformations | API: VectorAssembler(inputCols=["wss"], outputCol="features"), fillna(), concat() | mbd_variance (see page 98) DataFrame with columns: Channel, SampleID, MBD_ID, CellID, outlets, wss | DataFrame with columns: Channel, SampleID, MBD_ID, CellID, outlets, wss (scaled), features (DenseVector), channel_sample_mbd | TC-04: WSS=1e12 → scaled to 1.0.
TC-05: WSS=NULL → filled with 0.
TC-06: channel_sample_mbd = "Retail_101_MBD01".
TC-07: features column is DenseVector type. |

| S R # | Functionality Name | Short Description | Design Elements - UI, SB, DB | Impacted Tables and API Used | Input | Output | Test Cases |
|-------|-------------------------------|---|---|--|--|--|--|
| 3 | Find Optimal k per MBD | For a single MBD, evaluates k from 2 to max_cluster using Silhouette Score (squared Euclidean). Disqualifies k values where any cluster has fewer outlets than min_cluster_size. Returns k with highest score. Defaults to 2 if all scores are 0. | SB: PySpark ML KMeans fitting per k, ClusteringEvaluator for Silhouette metric, outlet count validation per cluster | API: <code>KMeans(featuresCol="features", k=k)</code> (see page 98), <code>model.fit()</code> (see page 98), <code>model.transform()</code> (see page 98), <code>ClusteringEvaluator(metricName="silhouette", distanceMeasure="squared Euclidean")</code> , <code>evaluator.evaluate()</code> , <code>groupBy().agg(sum("outlets"))</code> | <code>vector_data_mbd</code> (single MBD filtered DataFrame), <code>max_clusters</code> : <code>int</code> , <code>min_cluster_size</code> : <code>int</code> (see page 98) | <code>k</code> : <code>int</code> (see page 98) – optimal number of clusters for this MBD | TC-08: MBD with 10 cells → k between 2-8.
TC-09: MBD with 2 cells → k=2.
TC-10: MBD with 1 cell → k=1. TC-11: All scores=0 → k defaults to 2.
TC-12: Cluster with 1 outlet, min_size=2 → score=0 for that k.
TC-13: records < 3 → returns record count directly. |
| 4 | Create k Dictionary | Iterates over all distinct channel_sample_mbd groups, calls find_k_values for each, and builds a dictionary mapping each MBD to its optimal k. | SB: Driver-side loop over distinct MBD keys, sequential Spark job execution per MBD | API: <code>select("channel_sample_mbd").distinct().collect()</code> , <code>filter()</code> , <code>find_k_values()</code> | <code>vector_data</code> (see page 98) (all MBDs DataFrame), <code>max_cluster</code> : <code>int</code> (see page 98), <code>min_cluster_size</code> : <code>int</code> (see page 98) | <code>k_values</code> : <code>dict</code> (see page 98) – e.g. <code>{"Retail_101_MBD01": 3, "Retail_101_MBD02": 5}</code> | TC-14: 3 MBDs → dict with 3 entries.
TC-15: Each MBD key format is Channel_SampleID_MBD_ID. TC-16: All MBDs present in output dict. |

| S R # | Functionality Name | Short Description | Design Elements - UI, SB, DB | Impacted Tables and API Used | Input | Output | Test Cases |
|-------|---------------------------------|---|--|--|---|---|--|
| 5 | Calculate Clusters | Fits KMeans per MBD using determined k values. For k=1 assigns IBDID=0. Unions all per-MBD results. Computes <code>ibd_cluster_center</code> as AVG(wss) per (channel_sample_mbd, IBDID) using window function. | SB: PySpark ML KMeans per MBD, DataFrame union, Window function for cluster center | API: <code>KMeans(featuresCol="features", predictionCol="IBDID", k=k)</code> (see page 98), <code>model.fit()</code> (see page 98), <code>model.transform()</code> (see page 98), <code>union()</code> (see page 98), <code>Window.partitionBy("channel_sample_mbd", "IBDID")</code> (see page 98), <code>f.avg("wss").over(window)</code> (see page 98) | <code>vector_data</code> (see page 98) (all MBDs DataFrame), <code>k_values</code> : dict (see page 98) | DataFrame with columns: Channel, SampleID, MBD_ID, CellID, outlets, wss, features, channel_sample_mbd, IBDID, ibd_cluster_center | TC-17: k=3 → 3 distinct IBDIDs for MBD. TC-18: k=1 → all cells IBDID=0. TC-19: MBD not in dict → k defaults to 2. TC-20: <code>ibd_cluster_center</code> = avg(wss) within cluster. TC-21: All MBDs present after union. |
| 6 | Transform (Orchestrator) | Entry point. Calls <code>vectorize</code> → <code>create_k_dict</code> (if needed) → <code>calc_clusters</code> → joins <code>mbd_variance</code> with clusters on (Channel, SampleID, MBD_ID, CellID). Renames IBDID to MBD+ (cluster_id+1). Sorts and drops internal columns. | SB: Pipeline orchestration, DataFrame join, column rename, sort, drop | API: <code>vectorize()</code> (see page 98), <code>create_k_dict()</code> (see page 98), <code>calc_clusters()</code> (see page 98), <code>join(on=[Channel, SampleID, MBD_ID, CellID], how="inner")</code> , <code>f.concat("MBD", f.col("IBDID")+1)</code> (see page 98), <code>sort()</code> , <code>drop()</code> (see page 98) | <code>mbd_variance</code> (see page 98) DataFrame with columns: Channel, SampleID, MBD_ID, MBD, CellID, min_contribution_channel_value, wss, outlets, avg_yearly_sales, wss_proportion, cumm_variance | DataFrame with columns: Channel, SampleID, MBD_ID, MBD, CellID, avg_yearly_sales, IBDID (e.g. "MBDName1"), outlets, wss, ibd_cluster_center | TC-22: <code>k_values=None</code> → auto-detection runs. TC-23: <code>k_values</code> provided → skips auto-detection. TC-24: IBDID format = "MBDName1", "MBDName2". TC-25: Output sorted by Channel, SampleID, MBD_ID, IBDID. TC-26: <code>channel_sample_mbd</code> and features columns dropped. TC-27: All cells from input present in output (inner join). TC-28: <code>avg_yearly_sales</code> carried from input. |

19.3 3. User Stories and dev plan

Definition of Done:

- Code is written, reviewed, and merged.
- Consistent coding standards should be followed
- For any of the repetitive logic, A function to be created generically and used.
- No critical or high-priority defects remain.
- For Code ready , Cocode, Sonar, Code coverage,Zephyr Test cases, logging and testing for all scenarios to be done and artefacts should be maintained.
- Security and performance checks to be completed.
- Test case scenarios with data should be maintained in confluence page.
- Automation to be done for each functionality.
- Data should be validated and matched with the uploaded/updated file at the backend tables.
- . After the Development Ready, Code should be reviewed by the TechLead and take the sign off.
- At the end of each functionality, Demo to be provided to stakeholders and get a signoff.

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|---------------------------------|--------------------------------------|---|------------|-----------------------|--|--|
| UI - Max and Cluster Size Field | UI , Frontend , KMeans (see page 98) | Add Max and Min Cluster Size input field to KMeans configuration page | React | 1 SP | As a user, I want to configure the maximum number of clusters per MBD so that I can control the upper bound of IBD groups created and also I want to configure the minimum number of outlets required per cluster so that I can ensure each IBD group has sufficient store coverage. | <p>Details: Add a numeric input field labeled "Max Cluster Size" to the KMeans configuration panel.</p> <p>AC: 1. Field displays with default value 8. 2. Accepts integer values from 2 to 20. 3. Shows validation error if value < 2 or > 20. 4. Shows validation error if non-integer entered. 5. Value persists after save. 6. Field is required.</p> <p>Details: Add a numeric input/dropdown field labeled "Min Cluster Size" to the KMeans configuration panel.</p> <p>AC: 1. Field displays with default value 2. 2. Accepts integer values from 1 to 10. 3. Shows validation error if value < 1 or > 10. 4. Shows validation error if Min Cluster Size ≥ Max Cluster Size. 5. Value persists after save. 6. Field is required.</p> |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|---|------------------------------------|--|------------|-----------------------|--|--|
| UI - Groups (k_values) Field and Cross-Field Validation | UI, Frontend, KMeans (see page 98) | Add Groups configuration input for per-MBD cluster count override
Implement cross-field validation for KMeans configuration | React | 1 SP | As a user, I want to optionally specify the number of groups (clusters) for each MBD so that I can override automatic detection when I have domain knowledge.
As a user, I want to see validation errors when my configuration is inconsistent so that I avoid invalid clustering runs. | Details: Add an editable input where users can specify k values per MBD. Include an "Auto-detect all" checkbox. AC: 1. Table lists all available MBD_IDs from the dataset. 2. Each row has a dropdown with values "Auto", 2, 3, ..., max_cluster. 3. Default is "Auto" for all MBDs. 4. "Auto-detect all" checkbox sets all rows to "Auto". 5. When all are "Auto", k_values is sent as null to backend. 6. Mixed selections (some Auto, some manual) are supported. 7. Manual k values must be between 2 and max_cluster.

Details: Implement validation rules across the three new fields. AC: 1. min_cluster_size < max_cluster validated. 2. Manual k_values entries ≤ max_cluster validated. 3. Error messages display inline next to fields. 4. Save button disabled when validation errors exist. 5. Validation triggers on field change and on save. |
| UI - Configuration | UI, Database | Insert the required entries into slot table to call Databrick | DB | 1 SP | | |
| UI - Throttler Util Service | UI calling throttler service. | UI needs to call a throttler util service and insert data into throttler table. | React / SB | 2 SP | As a developer, UI needs to send the request details to databrick by inserting data into throttler table. | |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|---|-------------------------|--|------------|-----------------------------------|--|--|
| UI - Provide option to run both Springboot and Databrick | UI | UI needs to provide an option to select Databrick or springboot and also make entries in Slot table | UI | 1 SP | | |
| SB - | SB Service - | SB needs to update min and max cluster size field to accomodate in existing SB service.
https://operationshk.nielseni.com/selfcollect/cdarapigateway/d2ovuejobs/VUEScreePlotSaveMBDs | SB | 1 SP or 0.5 SP (Pau nra j) | As a developer, SB needs to map 2 new fields and save in the required firdeds | This includes complete unit testing and integration testing to save these parametrs. |
| KMeans Clustering | Prototype Understanding | Identify missing functionality and understanding it | Databrick | 2 SP | As a developer, Need to understand the existing prototype and if there is anything missing, need to be identified and fixed in code. | |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|---|-----------------------------------|---|------------|-----------------------|---|---------------------------------------|
| Kmeans Implementation | IBD KMeans | As a developer,
The code to be implemented to receive the request and transform as required. | Databricks | 1 SP | As a developer, Pyspark code to be written to accept the input parameters and write logic for transformation. | |
| Validating the libraries and understanding ML function written in prototype. | Validating libraries and function | As a developer,
Need to understand used libraries to implement functionality | Databricks | 1 SP | As a developer, need to understand about below function
pyspark.sql.Session
pyspark.sql.DataFrame
pyspark.sql.functions
pyspark.ml ⁸¹ .feature.VectorAssembler
pyspark.ml ⁸² .clustering.KMeans
pyspark.ml ⁸³ .evaluation.ClusteringEvaluator
pyspark.sql.window.Window | |

81. <http://pyspark.ml>

82. <http://pyspark.ml>

83. <http://pyspark.ml>

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|---------------------------------------|---------------------------------------|--|---|-----------------------|---|--|
| Implementing Core KMeans Logic Part 1 | Backend, Python, KMeans (see page 98) | Implement a production-grade WSS-driven vectorization and clustering pipeline that uses VectorAssembler, silhouette -optimized k selection with minimum-outlet validation, builds k-value dictionaries across all channel-sample-MBD groups, performs per-MBD KMeans clustering with IBDID assignment, computes ibd_cluster_center via window-averaged WSS, and orchestrates the full workflow through transform() | Python, PySpark, REST API (Flask / FastAPI) | 5 SP | As a backend developer, I want the API to accept the three new clustering parameters so that UI-configured values reach the KMeans algorithm. | Details: Modify the API endpoint that triggers KMeans clustering to accept and validate the new parameters. AC: 1. API accepts max_cluster (see page 98) (int, optional, default 8). 2. API accepts min_cluster_size (see page 98) (int, optional, default 2). 3. API accepts k_values (see page 98) (dict or null, optional, default null). 4. Returns 400 if max_cluster < 2 or > 20. 5. Returns 400 if min_cluster_size < 1 or > 10. 6. Returns 400 if any k_value > max_cluster. 7. Parameters passed to <code>KMeansCluster.__init__()</code> . |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|---------------------------------------|--|---|-----------------|-----------------------|---|---|
| Implementing Core KMeans Logic Part 2 | Backend, PySpark, KMeans (see page 98) | Output to be generated in the attached format as per existing implementation based on sample and write in a file and compress it. | Python, PySpark | 3 SP | As a backend developer, I want the new API parameters to be correctly passed to the KMeansCluster constructor so that user-configured values control the clustering behavior. | Details: Update the service/orchestration layer that calls KMeansCluster to pass the new parameters. AC: 1. max_cluster (see page 98) from API
→ KMeansCluster(max_cluster=value) .
2. min_cluster_size (see page 98) from API
→ KMeansCluster(min_cluster_size=value) .
3. k_values (see page 98) from API
→ KMeansCluster(k_values=value) . 4. Default values used when parameters not provided. 5. Existing functionality unchanged when defaults used. |
| KMeans Data Validation | Backend, PySpark, KMeans (see page 98) | KMeans Data to be matched from the PYSPark to Function output and the data should be matched and verified with user. | Python, PySpark | 2 SP | | |
| Download KMeans Data | PySpark | The downloadable file to be generated as part of the KMeans once the KMeans is completed successfully. | PySpark | 2 SP | As a developer, The download logic need to be implemented in KMeans. | |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|----------------------------|---|--|------------|-----------------------|---|---|
| Unit Testing - UI, Backend | Testing, Frontend, KMeans (see page 98) | Write unit tests for new KMeans UI fields and validation and API parameters and KMeans integration | | 2 SP | As a QA engineer, I want comprehensive unit tests for the three new UI fields so that I can ensure input validation and field behavior are correct.
I want comprehensive unit tests for the backend parameter handling and KMeans clustering with new parameters so that I can verify correctness. | <p>Details: Write unit tests covering all UI scenarios. AC: 1. Test default values render correctly (8, 2, Auto). 2. Test valid input accepted for each field. 3. Test invalid input shows error (out of range, non-integer). 4. Test cross-field validation (min < max). 5. Test Groups table renders all MBDs. 6. Test "Auto-detect all" checkbox. 7. Test save button disabled on validation error. 8. Test payload format on submit. 9. Minimum 90% code coverage.</p> <p>Write unit tests for API validation, parameter passing, and KMeans behavior with various configurations. AC: 1. Test API accepts valid max_cluster values. 2. Test API rejects invalid max_cluster (< 2, > 20). 3. Test API accepts valid min_cluster_size. 4. Test API rejects invalid min_cluster_size. 5. Test cross-parameter validation (min < max). 6. Test k_values=None triggers auto-detection. 7. Test k_values provided skips auto-detection. 8. Test KMeans with max_cluster=3 only evaluates k=2,3. 9. Test KMeans with min_cluster_size=5 disqualifies small clusters. 10. Test default behavior unchanged. 11. Minimum 90% code coverage.</p> |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|----------------------------|--|--|-----------------------------------|-----------------------|--|--|
| Integration Testing | Testing, Integration, KMeans (see page 98) | Write integration tests for end-to-end KMeans flow with new parameters | pytest, PySpark, REST API testing | 2 SP | As a QA engineer, I want integration tests that verify the complete flow from UI configuration to KMeans output so that I can ensure all layers work together correctly. | Details: Test full flow: UI input → API call → KMeans execution → output validation. AC: 1. Test UI → API → KMeans with custom max_cluster. 2. Test UI → API → KMeans with custom min_cluster_size. 3. Test UI → API → KMeans with manual k_values. 4. Test UI → API → KMeans with all defaults. 5. Test saved config loads correctly on page refresh. 6. Test output DataFrame has correct columns. 7. Test IBDID naming format. 8. Test ibd_cluster_center computation. 9. Test with real-world sample data (at least 3 MBDs). |
| Regression Testing | Testing, Regression, KMeans (see page 98) | US-012: Execute regression tests to ensure existing KMeans functionality is unaffected | pytest, PySpark, Manual testing | 2 SP | As a QA engineer, I want to verify that existing KMeans functionality works identically with default parameters so that the new changes don't break anything. | Details: Run existing test suite and verify output matches baseline. AC: 1. All existing unit tests pass. 2. Output with default params (8, 2, None) matches pre-change output. 3. Silhouette score selection unchanged. 4. IBD naming unchanged. 5. Cluster center computation unchanged. 6. Edge cases (1 cell, 2 cells, NULL wss) unchanged. |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|---------------------------|------------------------------------|--|-------------------------------------|-----------------------|--|---|
| Database Migration | Backend, DB, Migration | US-014: Create database migration script for new configuration columns | SQL, Flyway/Alembic | 1 SP | As a DevOps engineer, I want a migration script that adds the new configuration columns so that the schema update is automated and reversible. | Details: Create migration script for new columns in the project/job config table. AC: 1. Migration adds <code>max_cluster INT</code> DEFAULT 8. 2. Migration adds <code>min_cluster_size INT</code> DEFAULT 2. 3. Migration adds <code>k_values JSONB</code> DEFAULT NULL. 4. Existing rows get default values. 5. Rollback script removes columns. 6. Migration tested on staging DB. |
| Staging Deployment | DevOps, Deployment | US-015: Deploy KMeans configuration changes to staging environment | Docker, Kubernetes, CI/CD | 1 SP | As a DevOps engineer, I want to deploy all KMeans changes to staging so that QA can test in a production-like environment. | Details: Deploy frontend, backend, and DB changes to staging. AC: 1. DB migration executed successfully. 2. Backend API deployed with new parameters. 3. Frontend deployed with new fields. 4. Smoke tests pass. 5. No errors in application logs. 6. Staging environment matches production config. |
| UAT Testing | Testing, UAT, KMeans (see page 98) | US-016: Conduct UAT for new KMeans configuration fields | Manual testing, Staging environment | 2 SP | As a business user, I want to test the new configuration fields in staging so that I can verify they meet my requirements before production. | Details: Business users test all three new fields with real data. AC: 1. Max Cluster Size field works as expected. 2. Min Cluster Size field works as expected. 3. Groups override works for specific MBDs. 4. Auto-detect produces reasonable cluster counts. 5. Output IBDs match business expectations. 6. Performance acceptable for production data volumes. 7. Sign-off from product owner. |

| Functionality | Label | Stories | Tech Stack | Story Points & Sprint | Story Description | Story Details and Acceptance Criteria |
|------------------------------|--------------------------------|---|------------------------------------|-----------------------|--|--|
| Production Deployment | DevOps, Deployment, Production | US-017: Deploy KMeans configuration changes to production | Docker, Kubernetes, CI/CD | 2 SP | As a DevOps engineer, I want to deploy all verified KMeans changes to production so that users can access the new configuration options. | Details: Deploy all changes to production with rollback plan. AC: 1. DB migration executed in production. 2. Backend deployed with new API parameters. 3. Frontend deployed with new UI fields. 4. Smoke tests pass in production. 5. Monitoring dashboards show no anomalies. 6. Rollback plan documented and tested. 7. Deployment during approved maintenance window. |
| Post-Deployment Verification | DevOps, Monitoring, Production | US-018: Verify production deployment and monitor for issues | Monitoring tools, Application logs | 1 SP | As a DevOps engineer, I want to verify the production deployment is stable so that I can confirm no issues for end users. | Details: Monitor production for 48 hours post-deployment. AC: 1. No errors in application logs related to KMeans. 2. API response times within SLA. 3. KMeans clustering completes successfully with new params. 4. Default behavior unchanged for existing jobs. 5. No data inconsistencies in output. 6. User-reported issues triaged within 4 hours. |

19.4 4. Sprint Summary

| Sprint | Stories | Total Story Points | Focus | Sprint considering 1 resource |
|-----------------|--|------------------------|---|---|
| Sprint 1 | US-001, US-002, US-003, US-004, US-005 | 7 SP | UI development + API changes | 1 |
| Sprint 2 | US-006,US-007, US-008, US-009, US-010,US-011, US-012 | 18 SP | Backend integration + Unit testing | 2 |
| Sprint 3 | US-013, US-014, US-015 | 4 SP | Integration testing + Documentation + Staging | 1 |
| Sprint 4 | US-016, US-017, US-018 | 7 SP | UAT + Production deployment + Monitoring | 1 |
| Total | | 36 Story Points | 5 Sprints | with 2 resources in 3 Sprints it should be completed
1- Databrick resource who takes care of complete implementation details
1 - Full stack developer who takes care of UI, DB changes, Integration testing and Deployment |

19.5 5. Story Point Distribution by Category

S08

| Category | Stories | Story Points | Sprint Completion |
|-------------|--|--------------|-------------------|
| Frontend/UI | US-001, US-002, US-003, US-004, US-005 | 7 SP | S08 - S10 |
| Backend/API | US-006,US-007, US-008, US-009, US-010,US-011, US-012 | 18 SP | S08 - S10 |

| Category | Stories | Story Points | Sprint Completion |
|---|------------------------|--------------|-------------------|
| Testing (Unit + Integration + Regression + UAT) | US-013, US-014, US-015 | 4 SP | S08 - S10 |
| DevOps/Deployment | US-016, US-017, US-018 | 7 SP | S08 - S10 |

20 VUE Auto IBD KMeans Job Tests

Current KMeans implementation uses KMeans clustering algorithm and uses silhouette score for all possible cluster within the k values.

20.1 Kmeans job with spring boot flow

Input k values used.

Auto Design IBDs

Max Number of Cells: 8 Min # of Stores: 2 Submit KMeans clustering job

| Channel ID | Sample ID | MBD ID | Name | Groups |
|------------|-----------|----------------|------------------------|--------|
| MONT | 20003424 | Dummy_20003424 | Dummy MBD | 2 |
| MONT | 20003424 | 1_20183971 | INA+INFC+C&C AREA III | 2 |
| MONT | 20003424 | 1_20183972 | INA+INFC+C&C AREA V | 2 |
| MONT | 20003424 | 1_20183973 | INA+INFC+C&C AREA II | 2 |
| MONT | 20003424 | 1_20183974 | INA+INFC+C&C AREA IV | 2 |
| MONT | 20003424 | 1_20183975 | INA+INFC+C&C AREA I | 2 |
| MONT | 20003424 | 1_20183976 | INA+INFC+C&C AREA VII | 2 |
| MONT | 20003424 | 1_20183977 | INA+INFC+C&C AREA VI | 2 |
| MONT | 20003424 | 1_20183978 | INA+INFC+C&C AREA VIII | 2 |

Select Job Engine

Job Engine: ☒ Spring Boot ☐ DataBricks

Submit Cancel

D20 Job Scheduler

D20 Job Scheduler D20 Upload/Download Scheduler D20 Common Scheduler

Filters: Region: Country: Project ID: 953239 Job ID: Insertion Date: Created by: Version ID: Status:

Application: OPTAL_REPLICA_VUE Job Type: Search Reset

| Job ID | Country | Application | Job Type | Description | Project ID | Version ID | Initiation Time | RunTime | Completion Time | Created By | Status | View Logs | Download |
|--------|---------|-------------|---------------|-------------|------------|------------|---------------------|---------|---------------------|--------------|-----------|-----------|----------|
| 155873 | BRAZIL | VUE | KMEANS_REPORT | test_kmeans | 953239 | 995587866 | 2026-05-13 08:50:18 | 37s | 2026-05-13 08:50:55 | Amit x Kumar | COMPLETED | | |

953239_155873_KMeansReport.zip⁸⁴

Files downloaded

84. https://adlm.nielseniq.com/confluence/download/attachments/933124968/953239_155873_KMeansReport.zip?api=v2&modificationDate=1779359703394&version=1

project_953239 > output_953239_SB_RUN > 953239_155873_KMeansReport

Search 953239_155873_KMeansReport

| Name | Type | Compressed size | Password pr... | Size | Ratio | Date modifie |
|---|---------------|-----------------|----------------|--------|-------|--------------|
| 953239_20003424_KMeans_IBDCellsReport | Text Document | 3 KB | No | 46 KB | 96% | 5/13/2026 8: |
| 953239_20003424_KMeans_IBDCharacteristicsReport | Text Document | 14 KB | No | 111 KB | 88% | 5/13/2026 8: |

IBD_Cell Data - 517

| | | | |
|--|-------------------|---------------|--------|
| 953239_20003424_KMeans_IBDCellsReport | 5/13/2026 8:50 AM | Text Document | 46 KB |
| 953239_20003424_KMeans_IBDCharacteristicsRe... | 5/13/2026 8:50 AM | Text Document | 111 KB |

```

git check • rough.txt • dbfs_patl • 953239_Cell_25 953239_ X
File Edit View H1 v i≡ v B I S ↺ ...
10093381 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD1
10071776 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD1
10071778 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD1
10071781 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD1
10093382 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD1
10083607 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD1
10083611 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD1
10071832 20003424 MONT
Ln 518, Col 10 9 of 46,240 characters Plain text 100% Unix (LF) UTF-8

```

IBD_Characteristic Data - 475

| Name | Date modified | Type | Size |
|--|-------------------|---------------|--------|
| 953239_20003424_KMeans_IBDCellsReport | 5/13/2026 8:50 AM | Text Document | 46 KB |
| 953239_20003424_KMeans_IBDCharacteristicsRe... | 5/13/2026 8:50 AM | Text Document | 111 KB |


```

rough. • dbfs.p • 953239_Cell 953239_2000 9532: X
File Edit View H1 v i≡ v B I S ↺ ...
352737.37802321 20040154-BR_STORE_TYPE_G2G 20040168-BR_G2G_QUEBRA_ST
20040154TRADICIONAL MIXTO GD_20040168REGMET JOAO PESSOA 1 38721.62206300
0.0000 4 2 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD2 1000
352737.37802321 20040154-BR_STORE_TYPE_G2G 20040168-BR_G2G_QUEBRA_ST
20040154FARMACIA PEQ INDEP_20040168INT BA 75 135097.42652400
49715784891824.1391 4 2 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD2 1000
352737.37802321 20040154-BR_STORE_TYPE_G2G 20040168-BR_G2G_QUEBRA_ST
20040154FARMACIA GDE INDEP_20040168REGMET SALVADOR 35 267593.60005600
46841675006596.5652 4 2 20003424 MONT
1_20183975 INA+INFC+C&C AREA I INA+INFC+C&C AREA I(1_20183975)_IBD2 1000
352737.37802321 20040154-BR_STORE_TYPE_G2G 20040168-BR_G2G_QUEBRA_ST
20040154SUPERETTE INDEP_20040168INT PB 15 1922908.25594200
5787972305227223.0757 4 2 20003424 MONT
Ln 476, Col 10 9 of 113,635 characters Plain text 100% Unix (LF) UTF-8

```

20.2 Kmeans job with databricks flow

The test happens with same input k values. However if the user provides k values then Kmeans job is not going to use K values calculation for that particular MBD. Since existing Kmeans was provided with K values, so Kmeans would use clustering algorithm for cluster size.

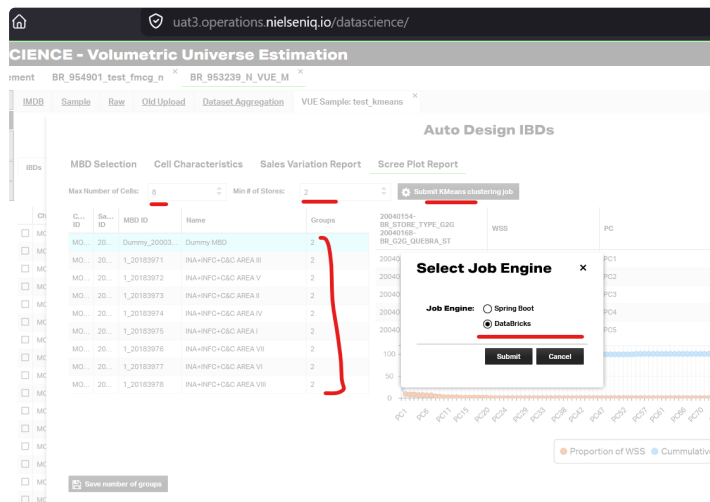
project_id 953239

country_code BR

scheduler_id 157428

transaction_id 895017935

input_dataset_id: 67



Databricks Run Log - <https://adb-6657211412833244.4.azuredatabricks.net/jobs/307492328156112/runs/600898347521718?o=6657211412833244>

UAT_VUE_K_MEANS run Succeeded

Output

```
Base Directory: dbfs:/opt/d20/vue/uat/data/vue_jobs/IBDS_tab/output/kmeans/953239/157428/vue_e_ibds_multi/part-00000-tid-808365384057795412-118c8d46-06ac-4ea6-8759-aaa737d1567d-314-1-c000.csv/
Data pushed to d20.br.vuekmeansibdcellsreportMulti_953239_67 successfully.
Data pushed to d20.br.VUEIBDCellsMulti_953239_67 successfully.
Data pushed to d20.br.vuekmeansibdcharacteristicsreportmulti_953239_67 successfully.
Data pushed to d20.br.VUEIBDsMulti_953239_67 successfully.
sample ids ['20003424']
Writing report data for sample id 20003424
All write operations completed successfully.
Final output files written successfully.
KMeans execution is successfully completed, updating status...
scheduler status {'schedulerId': 157428, 'bloblocation': 'abfss://d20@csusuatcdar.dfs.core.windows.net/hive/metastore/VUE/BR/953239/vue_kmeans/157428/kmeans_reports/,953239_157428_KMeansReport.zip', 'status': 'COMPLETED', 'log': 'KMeans execution is successfully completed', 'completionTime': '2026-05-21 09:31:04'}
status updated in vue_jobs_info table and throttler
KMeans execution is successfully completed, status updated.
Cleaning up temporary files in dbfs after successful execution.
KMeans files found. Deleting...
Successfully deleted KMeans files
```

Details

- Job ID: 307492328156112
- Job run ID: 600898347521718
- Task run ID: 563534432141454
- Run as: CCOE_Databricks_SP
- Launched: Manually
- Started: May 21, 2026, 02:53 PM
- Ended: May 21, 2026, 03:01 PM
- Duration: 7m 38s
- Status: Succeeded

Python wheel

- Package name: vue_kmeans
- Entry point: vue_kmeans

Total run time - 7 min 38 sec

execution time - 1 min 26 sec

Cluster UP time - 5 min 51 sec.

Reports Download

[953239_157428_KMeansReport.zip](#)⁸⁵

The screenshot shows the 'D20 Job Scheduler' interface. At the top, there are tabs for 'D20 Job Scheduler', 'D20 Upload/Download Scheduler', and 'D20 Common Scheduler'. Below these are filters for Region, Country, Project ID (953239), Job ID, and Insertion Date. A table lists jobs with columns: Job ID, Country, Application, Job Type, Description, Project ID, Version ID, Initiation Time, RunTime, Completion Time, Created By, Status, View Logs, Download, and Retriever. The job '953239_157428_KMeansReport.zip' is highlighted. A download notification box shows the file is 'Completed' and '16.6 KB'.

| Job ID | Country | Application | Job Type | Description | Project ID | Version ID | Initiation Time | RunTime | Completion Time | Created By | Status | View Logs | Download | Retriever |
|--------|---------|-------------|-----------------------|---|------------|------------|---------------------|---------|---------------------|--------------|-----------|-----------|----------|-----------|
| 157429 | BRAZIL | VUE | DOWNLOAD_PARQUET_D... | VUE_KMEANS_REPORT_DB-157428-test_kmeans | 953239 | 0 | 2026-05-21 09:38:10 | 21s | 2026-05-21 09:38:31 | Amit K Kumar | COMPLETED | | | |
| 157428 | BRAZIL | VUE | VUE_KMEANS_REPORT_DB | test_kmeans | 953239 | 895017935 | 2026-05-21 09:22:50 | 8m14s | 2026-05-21 09:31:04 | Amit K Kumar | COMPLETED | | | |
| 157427 | BRAZIL | VUE | ADD_V_NFR_NFR_SMB_01 | CDWorkflowAutomation | 953239 | 0 | 2026-05-21 04:53:16 | 1m10s | 2026-05-21 04:53:28 | chennuadhi | COMPLETED | | | |

Files in zip

The screenshot shows a file explorer window with the path 'Downloads > 953239_157428_KMeansReport'. It contains two files:

| Name | Type | Compressed size | Password pr... | Size | Ratio | Date modified |
|---|--------------------------------|-----------------|----------------|--------|-------|---------------|
| 953239_20003424_KMeans_IBDCellsReport | Microsoft Excel Comma Separ... | 3 KB | No | 52 KB | 96% | 5/21/2026 9:3 |
| 953239_20003424_KMeans_IBDCharacteristicsReport | Microsoft Excel Comma Separ... | 15 KB | No | 123 KB | 89% | 5/21/2026 9:3 |

20.2.1 Test with few kvalues as None for few mbds

If we don't pass kvalues for a mbd then kmeans job will calculate k values for it. Here it executed with 7 mbds with None values from input/UI which the Kmeans job calculated kvalues by Kmeans algorithm based on silhouette score for those have none kvalues.

Databricks Log

<https://adb-6657211412833244.4.azuredatabricks.net/jobs/307492328156112/runs/711187582524303?o=6657211412833244>

Input K values

```
[{Dummy_20003424:MONT:20003424:3},{1_20183978:MONT:20003424:2},
{1_20183971:MONT:20003424:None},{1_20183972:MONT:20003424:None},
{1_20183973:MONT:20003424:None},{1_20183974:MONT:20003424:None},
{1_20183975:MONT:20003424:None},{1_20183976:MONT:20003424:None},
{1_20183977:MONT:20003424:None}]
```

85. https://adlm.nielseni.com/confluence/download/attachments/933124968/953239_157428_KMeansReport.zip?api=v2&modificationDate=1779357908142&version=1

adb-6657211412833244.azuredatabricks.net/jobs/307492328156112/runs/711187582524303?o=665721141283 90%

Search data, notebooks, recents, and more... CTRL + P usnprodprocessing-amber-ws

Jobs & Pipelines > UAT_VUE_K_MEANS > Run 711187582524303 >

UAT_VUE_K_MEANS run Succeeded Edit task

Output

```
project_id 953239
country_code BR
scheduler_id 155897
transaction_id 589578670
env uat
region eastus2
scheduler_status {'schedulerId': 155897, 'blobLocation': '', 'status': 'INPROGRESS', 'log':
'VUE KMeans execution is started'}
status updated in vue_jobs_info table and throttler
KMeans execution is started, status updated...
sanitized and validated country_schema
vue_kmeans_files dbfs:/opt/d2o/vue/uat/data/vue_jobs/IBDS_tab/output/kmeans/953239/155897
kmeans_params_query SELECT parameters FROM d2o_common.vue_jobs_info
                        where projectid = %s and transactionid = %s

input_dataset_id: 67
k_values: [{Dummy_20003424:MONT:20003424:3},{1_20183978:MONT:20003424:2},{1_20183971:MONT:2
0003424:None},{1_20183972:MONT:20003424:None},{1_20183973:MONT:20003424:None},{1_20183974:M
ONT:20003424:None},{1_20183975:MONT:20003424:None},{1_20183976:MONT:20003424:None},{1_20183
977:MONT:20003424:None}]
max_cluster_size: 8
min_cluster_size: 2
job_name KMeans
```

Details

| | |
|-------------|------------------------|
| Job ID | 307492328156112 |
| Job run ID | 711187582524303 |
| Task run ID | 874453056705296 |
| Run as | CCOE_Databricks_SP |
| Launched | Manually |
| Started | May 19, 2026, 04:45 PM |
| Ended | May 19, 2026, 04:53 PM |
| Duration | 7m 50s |
| Status | Succeeded |

View run events View run libraries

Python wheel

| | |
|--------------|------------|
| Package name | vue_kmeans |
| Entry point | vue_kmeans |

Output K values

adb-6657211412833244.azuredatabricks.net/jobs/307492328156112/runs/711187582524303?o=665721141283 90%

Search data, notebooks, recents, and more... CTRL + P usnprodprocessing-amber-ws

Jobs & Pipelines > UAT_VUE_K_MEANS > Run 711187582524303 >

UAT_VUE_K_MEANS run Succeeded Edit task Repair run

Output

```
num_partitions 1
vue_mbd_multi_query select mbdid, mbdname, indexid, sampleid, period_type
                        from d2o_br.VUEMBDsMulti_953239_67

max available cores 160
estimated_rows 81
num_partitions... 1
num_partitions 1
Input preparation completed.
Starting KMeans transformers...
Initialized VUE KMeans Transformers.
k_values per MBD: {'MONT_20003424_Dummy_20003424': 3, 'MONT_20003424_1_20183978': 2, 'MONT_
20003424_1_20183971': 2, 'MONT_20003424_1_20183972': 3, 'MONT_20003424_1_20183973': 2, 'MON
T_20003424_1_20183974': 2, 'MONT_20003424_1_20183975': 2, 'MONT_20003424_1_20183976': 3, 'M
ONT_20003424_1_20183977': 4}
KMeans transformers completed.
kmeans_download_base_path abfss://d2o@csusuatcdar.dfs.core.windows.net/hive/metastore/VUE/B
R/953239/vue_kmeans/155897/kmeans_reports/
Writing final output files...
Preparing final output files...
Writing vue_ibds_multi to CSV...
Writing vue_ibd_cells_multi to CSV...
Writing vue_kmeans ibd characteristics report multi to CSV...
```

Details

| | |
|-------------|------------------------|
| Job ID | 307492328156112 |
| Job run ID | 711187582524303 |
| Task run ID | 874453056705296 |
| Run as | CCOE_Databricks_SP |
| Launched | Manually |
| Started | May 19, 2026, 04:45 PM |
| Ended | May 19, 2026, 04:53 PM |
| Duration | 7m 50s |
| Status | Succeeded |

View run events View run libraries

Python wheel

| | |
|--------------|------------|
| Package name | vue_kmeans |
| Entry point | vue_kmeans |

20.3 Data Validation with SB or DB func run

IBD_Cell Data - 517

| MBDID | MBDName | IBDName | Cell ID | sampleid | indexid |
|----------|----------|----------|----------|----------|---------|
| 1_201839 | INA+INFC | INA+INFC | 10071310 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071311 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071353 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071354 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071396 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071397 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071438 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071439 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071789 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071791 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071840 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071842 | 20003424 | MONT |
| 1_201839 | INA+INFC | INA+INFC | 10071843 | 20003424 | MONT |

IBD Characteristic data - 475

| MBDID | MBDName | IBDName | IBD Outlet | IBD Center | Combinati | Combin |
|----------|----------|----------|------------|------------|-----------|----------|
| 1_201839 | INA+INFC | INA+INFC | 20 | 3418521 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |
| 1_201839 | INA+INFC | INA+INFC | 171 | 722124.9 | 20040154 | 20040154 |

Reports Download counts are matching.

Data Validation between the pg DB tables. (ALL Data Counts Match)

-- SB RUN

select * from d2o_br.VUEIBDCellsMulti_953239_67; -- 517

select * from d2o_br.VUEIBDsMulti_953239_67; -- 18 UI also 18

select * from d2o_br.vuekmeansibdcharacteristicsreportmulti_953239_67 where jobid=155873; -- 475

select * from d2o_br.vuekmeansibdcellsreportMulti_953239_67 where jobid=155873; -- 517

-- DATABRICKS RUN

select * from d2o_br.VUEIBDCellsMulti_953239_67; -- 517

select * from d2o_br.VUEIBDsMulti_953239_67; -- 18 UI also 18

```
select * from d2o_br.vuekmeansibdcharacteristicsreportmulti_953239_67 where jobid=157428; -- 475
select * from d2o_br.vuekmeansibdcellsreportMulti_953239_67 where jobid=157428; -- 517
```

20.3.1 UI DATA after Databricks run

[illegible]

We can see there is data difference for few of columns like `ibd_outlets`, `ibd_center` because the grouping of `mbd` by their `ibds` grouped with different combination than in spring boot run. It has the difference due to cluster size calculated by KMeans algorithm with `mbd` to their combination ids.

For the ibd outlet difference we can clearly see that provided mbd has grouped with 2 ibds group.

ibclusterid assigned with mbd by kmeans falls into same number of groups 2 only but mbd --> valuesid falls into different group as compared to sb run. So why the outlet numbers are exchanged within two groups.

databricks run

| | A | B | C | D | E | F | G | H | I | J | K | L | M |
|----|------------|----------|--|---------|---------|----------|----------|--------|---------|-----------------------|--------|----------|---------|
| 1 | MBDID | MBDNa | IBDName | IBD Out | IBD Cer | Combin | Combin | Combin | Average | WSS | Groups | sample | indexid |
| 28 | 1_20183971 | INA+INFC | INA+INFC+C&C AREA III(1_20183971)_IBD2 | 4 | 4439779 | 20040154 | 20040154 | 4 | 4439779 | 4512035839237810.0000 | 2 | 20003424 | MONT |

sb run

| | A | B | C | D | E | F | G | H | I | J | K | L | M |
|-----|-----------|---|---------|------------|------------|--------------------|--------|--------|---------|----------|-----------------------|--------|----|
| 1 | MBDID | MBDNa | IBDName | IBD Outlet | IBD Center | Combin | Combin | Combin | Average | WSS | Iteration | Groups | sa |
| 255 | 1_2018397 | INA+INFC+INA+INFC+C&C AREA III(1_20183971)_IBD1 | | 7 | 8413356.83 | 20040154-20040154: | | | 3 | 12653478 | 2054723363259860.0000 | 4 | 2 |
| 265 | 1_2018397 | INA+INFC+INA+INFC+C&C AREA III(1_20183971)_IBD1 | | 7 | 8413356.83 | 20040154-20040154: | | | 3 | 5670439 | 52387741415379.1000 | 4 | 2 |
| 267 | 1_2018397 | INA+INFC+INA+INFC+C&C AREA III(1_20183971)_IBD1 | | 7 | 8413356.83 | 20040154-20040154: | | | 1 | 6916154 | 0.0000 | 4 | 2 |

File Edit View Run Help Python ☆ Last edit was now Run all DevNeeds Schedule

| | 1.2 IBDClusterID | 1.2 MBD_ID | 1.2 Combin... | 1.2 CombinationValueID | 1.2 outlets | 1.2 avg_yearl... | 1.2 SampleID | 1.2 Channel |
|---|------------------|------------|---------------|------------------------|-------------|------------------|--------------|-------------|
| 1 | 0 | 1_20183971 | 2 | 208 | 3 | 5670439.13252 | 20003424 | MONT |
| 2 | 0 | 1_20183971 | 2 | 89 | 3 | > 12653477.... | 20003424 | MONT |
| 3 | 0 | 1_20183971 | 2 | 257 | 1 | 6916153.86664 | 20003424 | MONT |

3 rows | 5.31s runtime Refreshed 1 minute ago

Just now (6s) 58

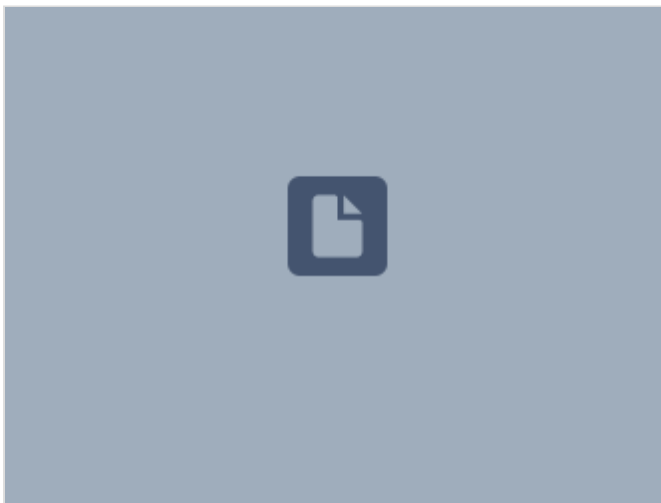
```
1 combination_clusters.filter((col('MBD_ID') == '1_20183971') & (col('IBDClusterID') == 1)).select("IBDClusterID", "MBD_ID", "CombinationID", "CombinationValueID", "outlets", "avg_yearly_sales", "SampleID", "Channel").display()
```

(42) Spark Jobs

Table +

| | 1.2 IBDClusterID | 1.2 MBD_ID | 1.2 CombinationID | 1.2 CombinationValueID | 1.2 outlets | 1.2 avg_yearly_sales | 1.2 SampleID |
|---|------------------|------------|-------------------|------------------------|-------------|----------------------|--------------|
| 1 | 1 | 1_20183971 | 2 | 127 | 4 | 4439779.471682 | 20003424 |

The following sheet has the complete data comparison details.



The performance has been improved by 53% from the previous run.

older - <https://adb-6657211412833244.4.azuredatabricks.net/jobs/307492328156112/runs/693278437038637?o=6657211412833244>

TOTAL - 10 min 32 Sec

CLUSTER UP TIME - 5 min 22 sec

RUN TIME - 4 min 39 sec

new - <https://adb-6657211412833244.4.azuredatabricks.net/jobs/307492328156112/runs/71187582524303?o=6657211412833244>

TOTAL - 7 min 50 Sec

CLUSTER UP TIME - 5 min 22 sec

RUN TIME - 2 min 7 sec